MP307 Practical 4 Continuous Population Models

Name: Cian Mac Liatháin ID: 11476078

In [76]:

```
%matplotlib notebook
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint
```

Q1.(*) Verhulst Logistic Model.

Consider the logistic population model

$$\frac{dP}{dt} = rP\left(1 - \frac{P}{K}\right)$$

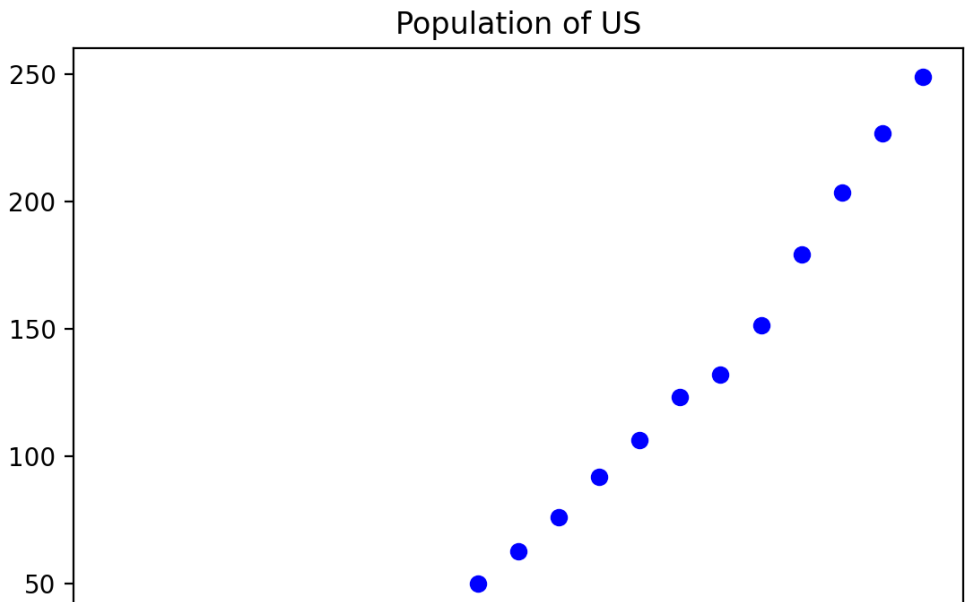
for $r, K > 0$ as a model of the US population from 1790 to 1990. The data for this are as given in the worksheet Q2. of `lab3_discrete_population_models.ipynb` as follows:

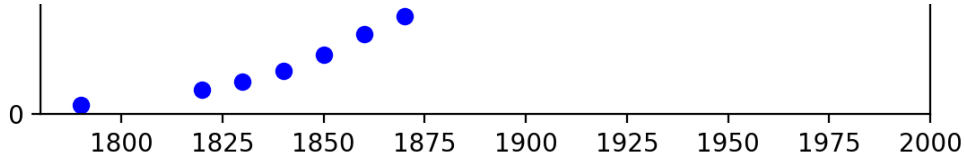
In [77]:

```
Year = [1790, 1820, 1830, 1840, 1850, 1860, 1870, 1880, 1890, 1900, 1910, 
        1940, 1950, 1960, 1970, 1980, 1990]
Pop= [3.79, 9.6, 12.9, 17.1, 23.2, 31.4, 38.6, 50.2, 62.9, 76.0, 92.0, 106
      132.0, 151.3, 179.3, 203.3, 226.54, 248.7]
```

In [78]:

```
# plot of data
plt.figure(1)
plt.plot(Year,Pop,'bo')
plt.axis([1780,2000,0,260])
plt.title('Population of US')
plt.show()
```





The Logistic differential equation has solution

$$P(t) = \frac{K}{1 + \left(\frac{K}{P_0} - 1\right) e^{-rt}}$$

In [79]:

```
#Logistic Model

# Python function for Logistic solution where t=0 is 1790
def P(t):
    P_0=3.79
    r=0.3 #  in inverse decades
    K=300.0
    ans=K/(1.+(K/P_0-1.)*np.exp(-r*t))
    return ans
```

In [80]:

```
P(0)
```

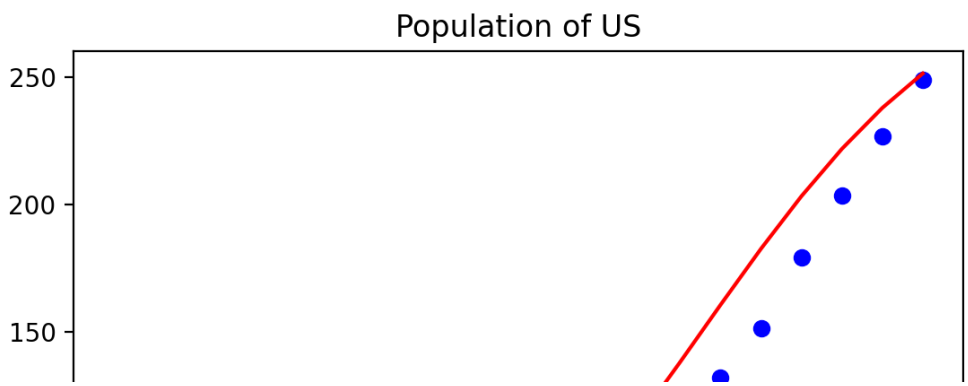
Out[80]: 3.79

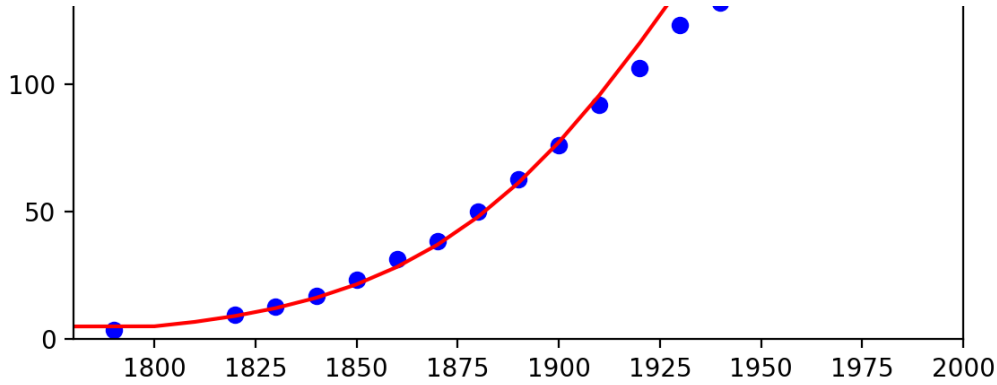
In [81]:

```
# plot Verhulst pop PV from 1790 to 1990 with 10 year intervals
PV=[0]*21 # list for Verhulst P initialized with 20 zeros
YV=[0]*21 # list for years initialized with 20 zeros

# choose 10 year intervals
for n in range(1,21):
    YV[n]=1790+10*n
    PV[n]=P(n)
```

In [82]:

```
plt.figure(2)
plt.plot(Year,Pop,'bo')
plt.plot(YV,PV,'r-')
plt.axis([1780,2000,0,260])
plt.title('Population of US')
plt.show()
```





In [ ]:

Q.2(*) Competitive Species Model

Consider two species with population sizes P_1, P_2 with growth rates r_1, r_2 and limiting population sizes of K_1, K_2 where

$$\frac{dP_1}{dt} = r_1 P_1 \left(1 - \frac{P_1 + P_2}{K_1}\right),$$
$$\frac{dP_2}{dt} = r_2 P_2 \left(1 - \frac{P_1 + P_2}{K_2}\right).$$

Analyse the behaviour of P_1, P_2 in the following cases, by plotting P_1, P_2 vs t and P_1 vs P_2:

(a). $r_1 = 1/10$, $r_2 = 1/10$, $K_1 = 100$, $K_2 = 50$ with $P_1(0) = 10$ and $P_2(0) = 15$.

(b). $r_1 = 1/10$, $r_2 = 1/10$, $K_1 = 100$, $K_2 = 50$ with $P_1(0) = 130$ and $P_2(0) = 200$.

(c). $r_1 = 1/10$, $r_2 = 1/100$, $K_1 = 40$, $K_2 = 50$ with $P_1(0) = 130$ and $P_2(0) = 20$.

(d). $r_1 = 1/10$, $r_2 = 1/100$, $K_1 = 50$, $K_2 = 60$ with $P_1(0) = 15$ and $P_2(0) = 10$.

In [112…

```
#(a)
# parameter values
r_1, r_2 = 1/10, 1/10

K_1, K_2 = 100, 50

P1_0, P2_0 = 10, 15 # initial populations
```

Define a function `dP_dt(P, t)` of a Python list `P=[P1,P2]` describing dP1/dt and dP2/dt.

Note: recall that indexing in Python goes 0,1,... so that Pop1 is stored in `P[0]` and Pop2 is stored in `P[1]`

In [113…

```
def dP_dt(P, t): # P=[P1,P2] list
    return [r_1*P[0]*(1 - (P[0]+P[1])/K_1), r_2*P[1]*(1 - (P[0]+P[1])/K_2)
```

Use `odeint` Python function to integrate the pair of odes subject to the initial conditions for time from `tmin` to `tmax` .

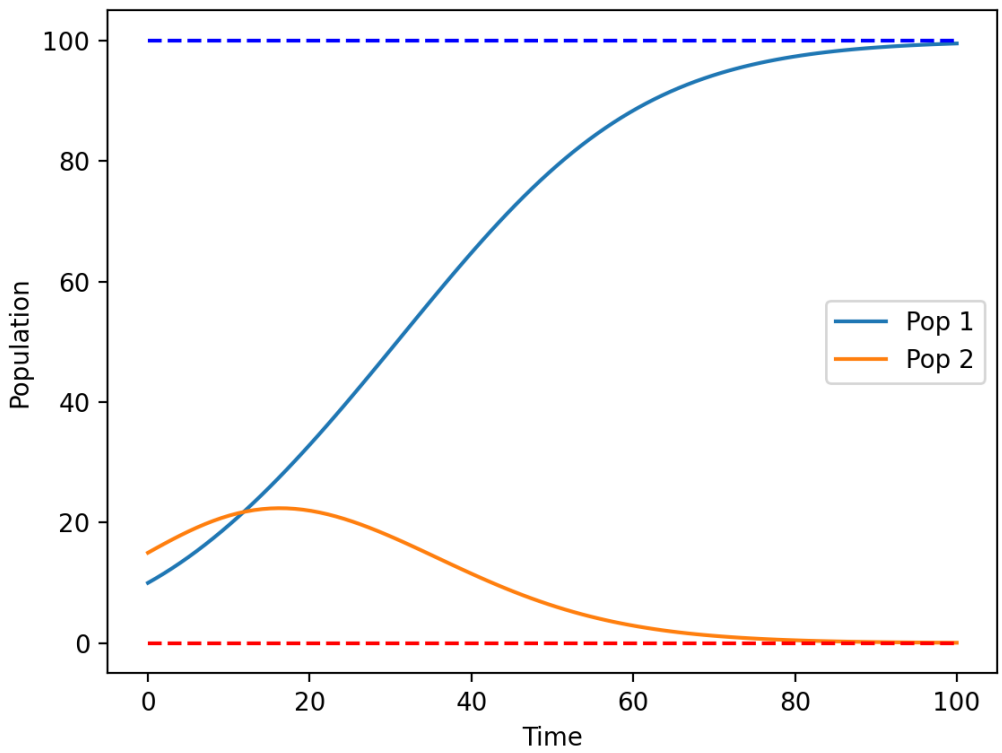
In [114…

```
tmin=0
tmax=100.

ts = np.linspace(tmin, tmax, 100) # 100 equally spaced t values from tmin t
P0 = [P1_0, P2_0] #initial conditions
Ps = odeint(dP_dt, P0, ts)
P1 = Ps[:,0] # values of P1 on ts values
P2 = Ps[:,1] # values of P2 on ts values
```

Plot P_1 and P_2 vs t and asymptotic lines.

In [115…

```
plt.figure(3)
plt.plot(ts, P1, "-", label="Pop 1")
plt.plot(ts, P2, "-", label="Pop 2")
plt.xlabel("Time")
plt.ylabel("Population")
plt.legend();
```



Notice that as $t \to \infty$ then $(P_1, P_2) \to (K_1, 0)$ for $K_1 > K_2$ and $(P_1, P_2) \to (0, K_2)$ for $K_1 < K_2$.

We can add the asymptotic lines to the above graph as follows

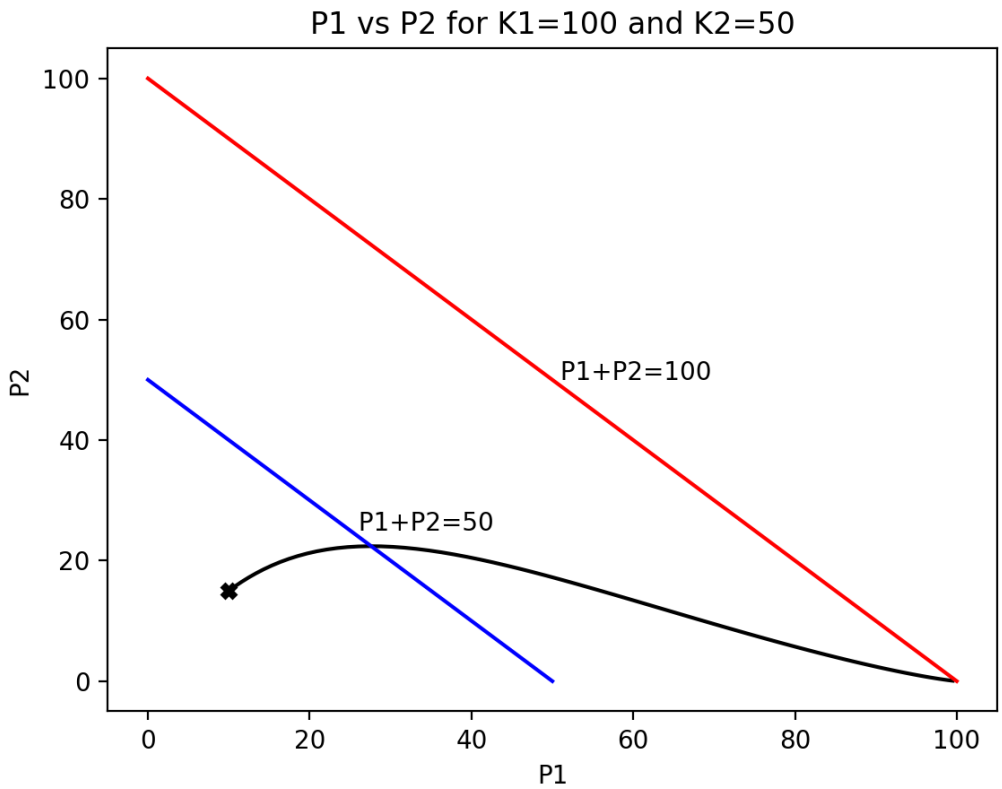
In [116…

```
P1_infty, P2_infty = K_1, 0
if K_2>K_1:
    P1_infty, P2_infty = 0,K_2
plt.plot([tmin,tmax],[P1_infty,P1_infty],"b--")
plt.plot([tmin,tmax],[P2_infty,P2_infty],"r--");
```

Plot P_1 vs P_2 with initial point marked with an x. We also plot the lines $P_1 + P_2 = K_1$ and $P_1 + P_2 = K_2$.

In [117…

```
plt.figure(4)

plt.plot(P1_0, P2_0,'kX') # mark initial point with x

plt.plot(P1, P2, 'k-')
plt.plot([0,K_1],[K_1,0],'r-')
plt.plot([0,K_2],[K_2,0],'b-')
plt.text(K_1/2, K_1/2,' P1+P2='+str(K_1))
plt.text(K_2/2, K_2/2,' P1+P2='+str(K_2))
plt.xlabel("P1")
plt.ylabel("P2")
plt.title("P1 vs P2 for K1="+str(K_1)+" and K2="+str(K_2));
```



In [120…

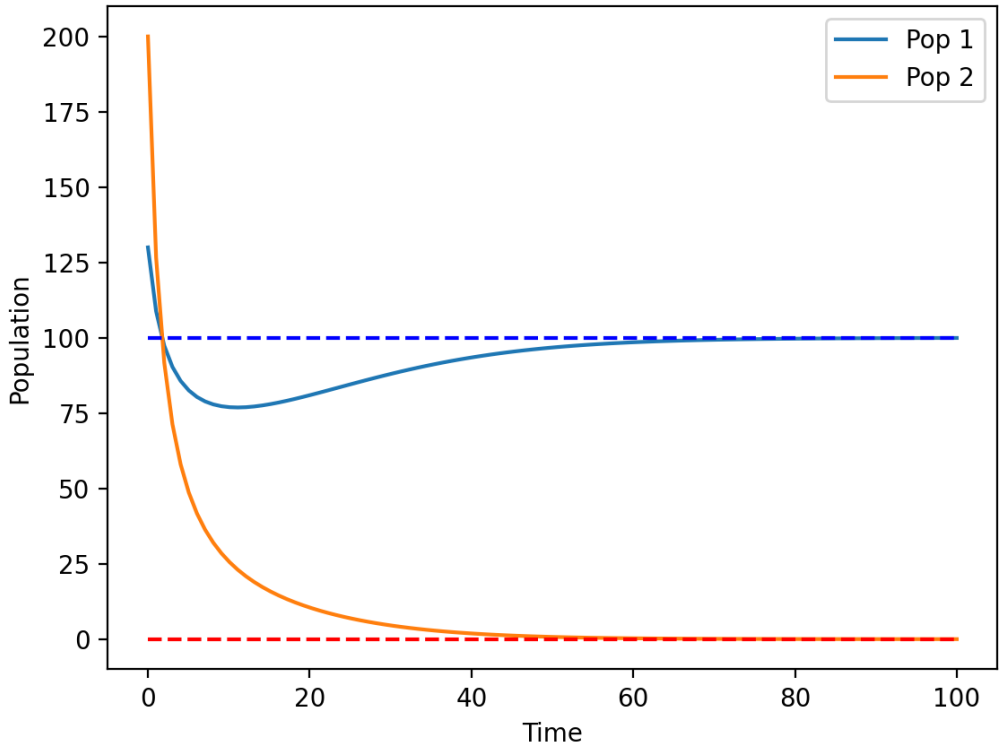
```
# (b)
# parameter values
r_1, r_2 = 1/10, 1/10

K_1, K_2 = 100, 50

P1_0, P2_0 = 130, 200 # initial populations
```

In [121…

```
tmin=0
tmax=100

ts = np.linspace(tmin, tmax, 100) # 100 equally spaced t values from tmin t
P0 = [P1_0, P2_0] #initial conditions
Ps = odeint(dP_dt, P0, ts)
P1 = Ps[:,0] # values of P1 on ts values
P2 = Ps[:,1] # values of P2 on ts values
```

In [122…

```
plt.figure(5)
plt.plot(ts, P1, "-", label="Pop 1")
plt.plot(ts, P2, "-", label="Pop 2")
plt.xlabel("Time")
plt.ylabel("Population")
plt.legend();
```

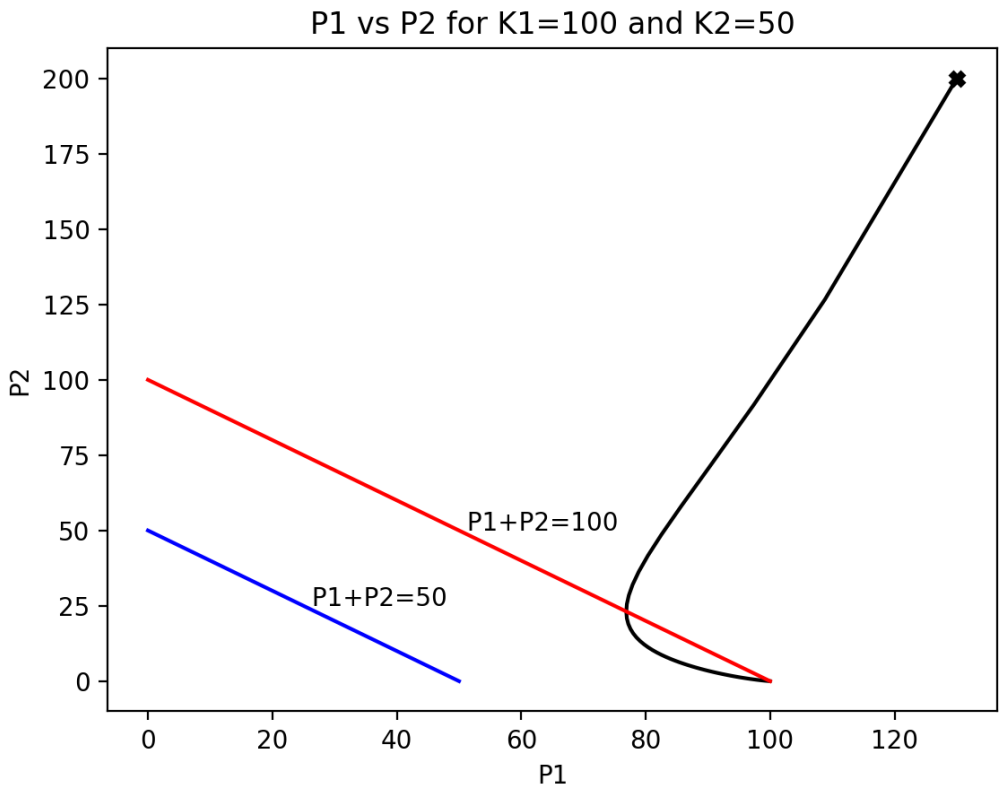


In [123…

```
P1_infty, P2_infty = K_1, 0
if K_2>K_1:
    P1_infty, P2_infty = 0,K_2
plt.plot([tmin,tmax],[P1_infty,P1_infty],"b--")
plt.plot([tmin,tmax],[P2_infty,P2_infty],"r--");
```

In [124…

```
plt.figure(6)

plt.plot(P1_0, P2_0,'kX') # mark initial point with x

plt.plot(P1, P2, 'k-')
plt.plot([0,K_1],[K_1,0],'r-')
plt.plot([0,K_2],[K_2,0],'b-')
plt.text(K_1/2, K_1/2,' P1+P2='+str(K_1))
plt.text(K_2/2, K_2/2,' P1+P2='+str(K_2))
plt.xlabel("P1")
plt.ylabel("P2")
plt.title("P1 vs P2 for K1="+str(K_1)+" and K2="+str(K_2));
```



In [125…

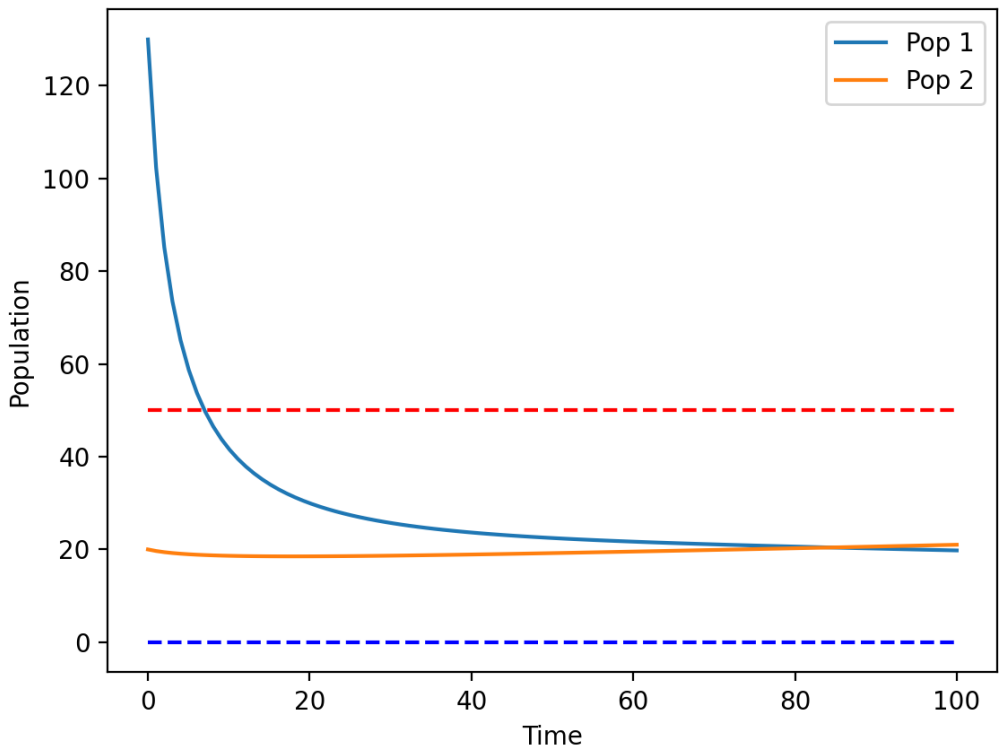
```
#(c)
# parameter values
r_1, r_2 = 1/10, 1/100

K_1, K_2 = 40, 50

P1_0, P2_0 = 130, 20 # initial populations
```

In [126…

```
P0 = [P1_0, P2_0] #initial conditions
Ps = odeint(dP_dt, P0, ts)
P1 = Ps[:,0] # values of P1 on ts values
P2 = Ps[:,1] # values of P2 on ts values
```

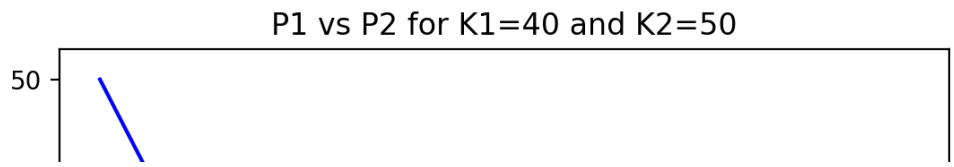
In [127…

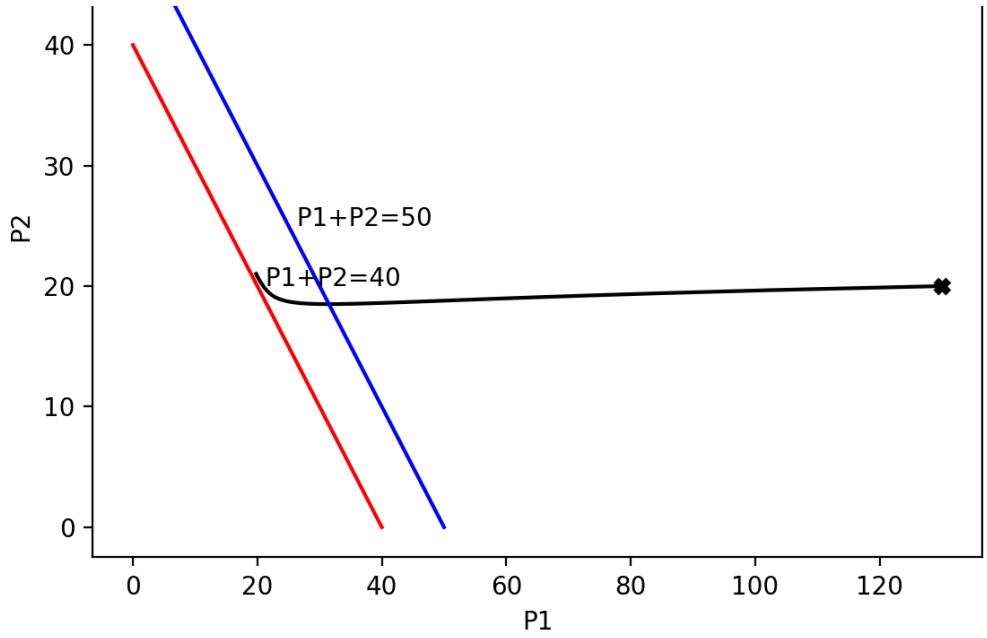
```
plt.figure(7)
plt.plot(ts, P1, "-", label="Pop 1")
plt.plot(ts, P2, "-", label="Pop 2")
plt.xlabel("Time")
plt.ylabel("Population")
plt.legend();
P1_infty, P2_infty = K_1, 0
if K_2>K_1:
    P1_infty, P2_infty = 0,K_2
plt.plot([tmin,tmax],[P1_infty,P1_infty],"b--")
plt.plot([tmin,tmax],[P2_infty,P2_infty],"r--");
```



In [128…

```
plt.figure(8)

plt.plot(P1_0, P2_0,'kX') # mark initial point with x

plt.plot(P1, P2, 'k-')
plt.plot([0,K_1],[K_1,0],'r-')
plt.plot([0,K_2],[K_2,0],'b-')
plt.text(K_1/2, K_1/2,' P1+P2='+str(K_1))
plt.text(K_2/2, K_2/2,' P1+P2='+str(K_2))
plt.xlabel("P1")
plt.ylabel("P2")
plt.title("P1 vs P2 for K1="+str(K_1)+" and K2="+str(K_2));
```





In [130…

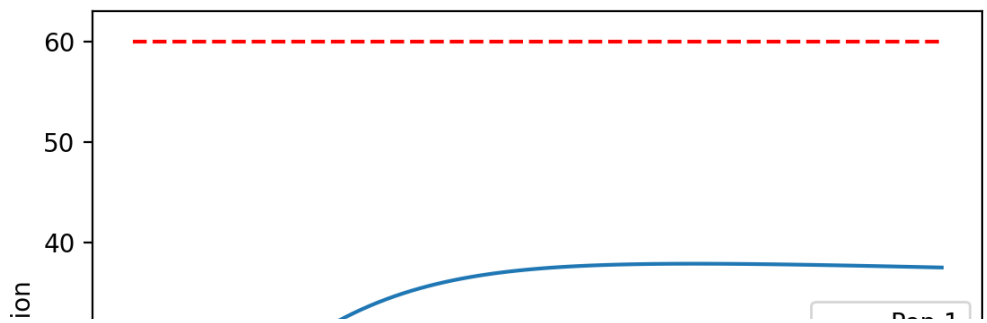
```
# (d)
# parameter values
r_1, r_2 = 1/10, 1/100

K_1, K_2 = 50, 60

P1_0, P2_0 = 15, 10 # initial populations
```

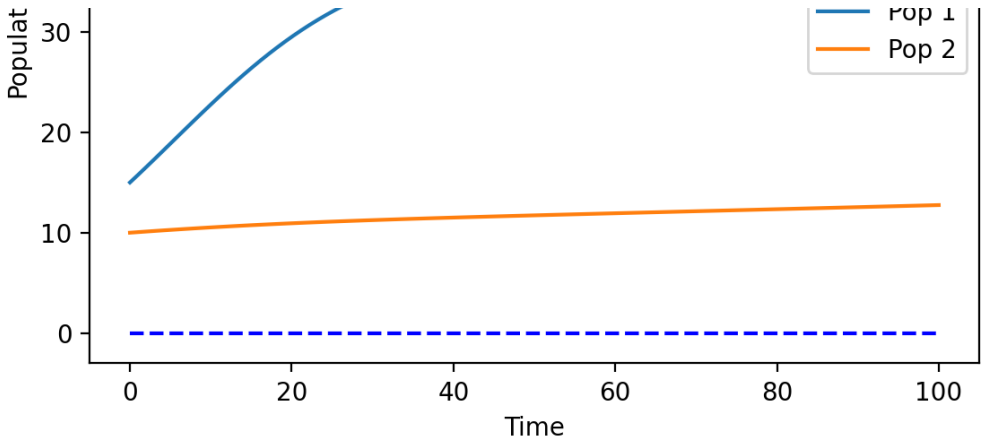
In [131…

```
P0 = [P1_0, P2_0] #initial conditions
Ps = odeint(dP_dt, P0, ts)
P1 = Ps[:,0] # values of P1 on ts values
P2 = Ps[:,1] # values of P2 on ts values
```

In [132…

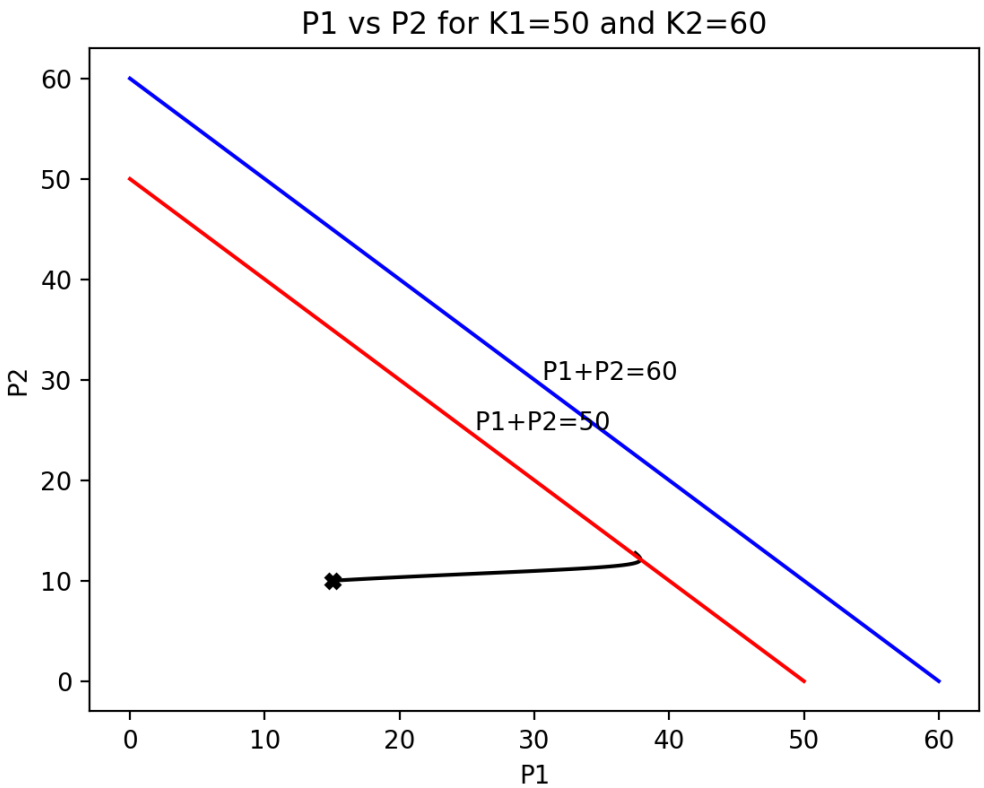
```
plt.figure(9)
plt.plot(ts, P1, "-", label="Pop 1")
plt.plot(ts, P2, "-", label="Pop 2")
plt.xlabel("Time")
plt.ylabel("Population")
plt.legend();
P1_infty, P2_infty = K_1, 0
if K_2>K_1:
    P1_infty, P2_infty = 0,K_2
plt.plot([tmin,tmax],[P1_infty,P1_infty],"b--")
plt.plot([tmin,tmax],[P2_infty,P2_infty],"r--");
```





In [133…

```
plt.figure(10)

plt.plot(P1_0, P2_0,'kX') # mark initial point with x

plt.plot(P1, P2, 'k-')
plt.plot([0,K_1],[K_1,0],'r-')
plt.plot([0,K_2],[K_2,0],'b-')
plt.text(K_1/2, K_1/2,' P1+P2='+str(K_1))
plt.text(K_2/2, K_2/2,' P1+P2='+str(K_2))
plt.xlabel("P1")
plt.ylabel("P2")
plt.title("P1 vs P2 for K1="+str(K_1)+" and K2="+str(K_2));
```



Q.3(*) Lotka-Volterra Predator/Prey System

Consider a prey species with population size x and a predator species with population size y where

$$\frac{dx}{dt} = x(a_1 - b_1 y),$$
$$\frac{dy}{dt} = y(-a_2 + b_2 x),$$

with $a_1, a_2, b_1, b_2 > 0$. Analyse the behaviour of the system for $a_1 = 3, a_2 = 5/2$ and $b_1 = 2, b_2 = 1$ by plotting x, y vs t, and x vs y in the following cases:

(a). $x(0) = 1$ and $y(0) = 1$.

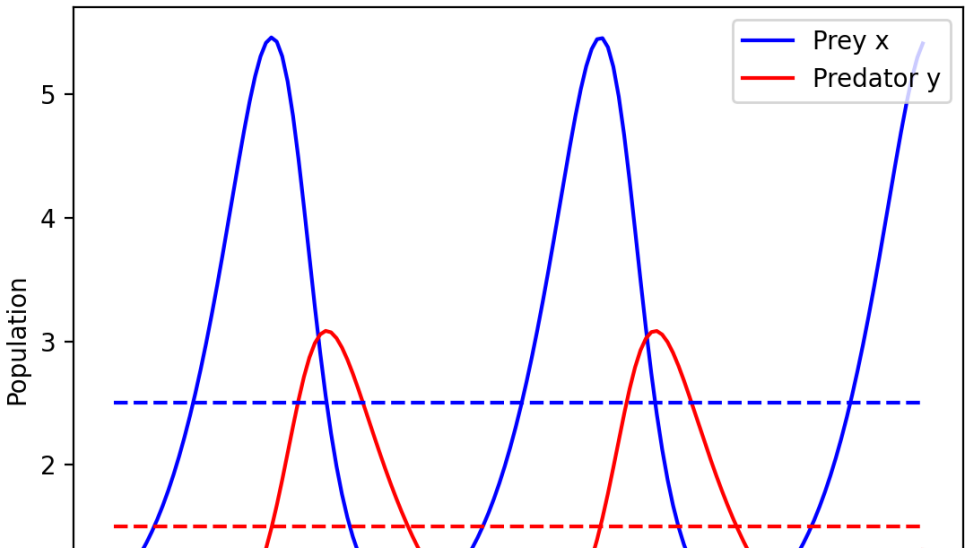
(b). $x(0) = 0.1 + 5/2$ and $y(0) = 0.1 + 3/2$. What behaviour do you observe?

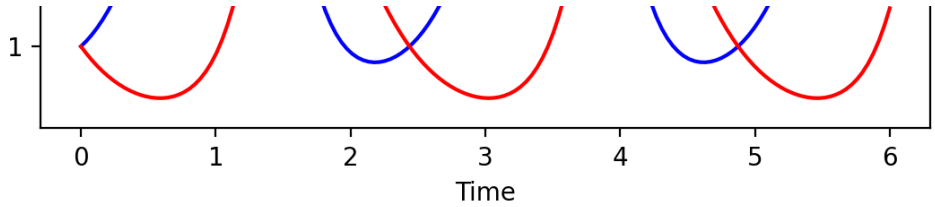
(c). $x(0) = 1$ and $y(0) = 5$.

Use similar coding to above

In [134…

```
#(a)
a_1, a_2, b_1, b_2 = 3, 5/2, 2, 1

x0, y0 = 1, 1 # initial populations

timerange=6.

def dP_dt(P, t): # P=[x,y] list
    return [P[0]*(a_1 - b_1*P[1]), P[1]*(-a_2 +b_2 *P[0])]

ts = np.linspace(0, timerange, 150)
P0 = [x0, y0] #initial conditions
Ps = odeint(dP_dt, P0, ts)
prey = Ps[:,0]
predators = Ps[:,1]

plt.figure(11)
plt.plot(ts, prey, "b-", label="Prey x")
plt.plot(ts, predators, "r-", label="Predator y")
plt.xlabel("Time")
plt.ylabel("Population")
plt.legend();
```





Equlibrium values $x_{eq} = a_2/b_2$ and $y_{eq} = a_1/b_1$. Notice that the maximum/minimum of x occurs when $y = y_{eq}$ and similarly, the maximum/minimum of y occurs when $x = x_{eq}$

In [135…

```
xeq , yeq =a_2/b_2, a_1/b_1
print("(xeq,yeq)=",(xeq,yeq))
```

```
(xeq,yeq)= (2.5, 1.5)
```

Look at values of x for $y = y_{eq}$ and of y for $x = x_{eq}$ in the last plot

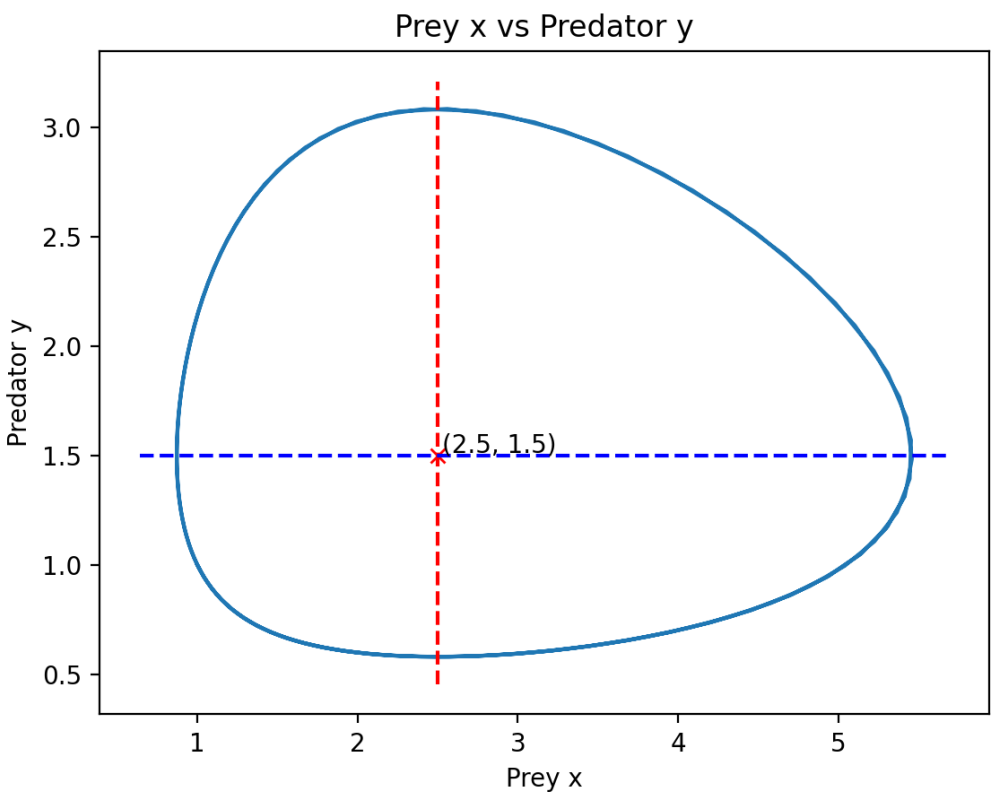
In [136…

```
plt.plot([0,timerange],[xeq,xeq],"b--")
plt.plot([0,timerange],[yeq,yeq],"r--");
```

Plot x vs y and show the equilibrium point. This plot shows that the system is periodic.

In [137…

```
plt.figure(12)
plt.plot(prey, predators, "-")
plt.plot([xeq],[yeq],'rx')
plt.text(xeq*(1.01),yeq*(1.01),str((xeq,yeq)))
plt.xlabel("Prey x")
plt.ylabel("Predator y")
plt.title("Prey x vs Predator y");
```



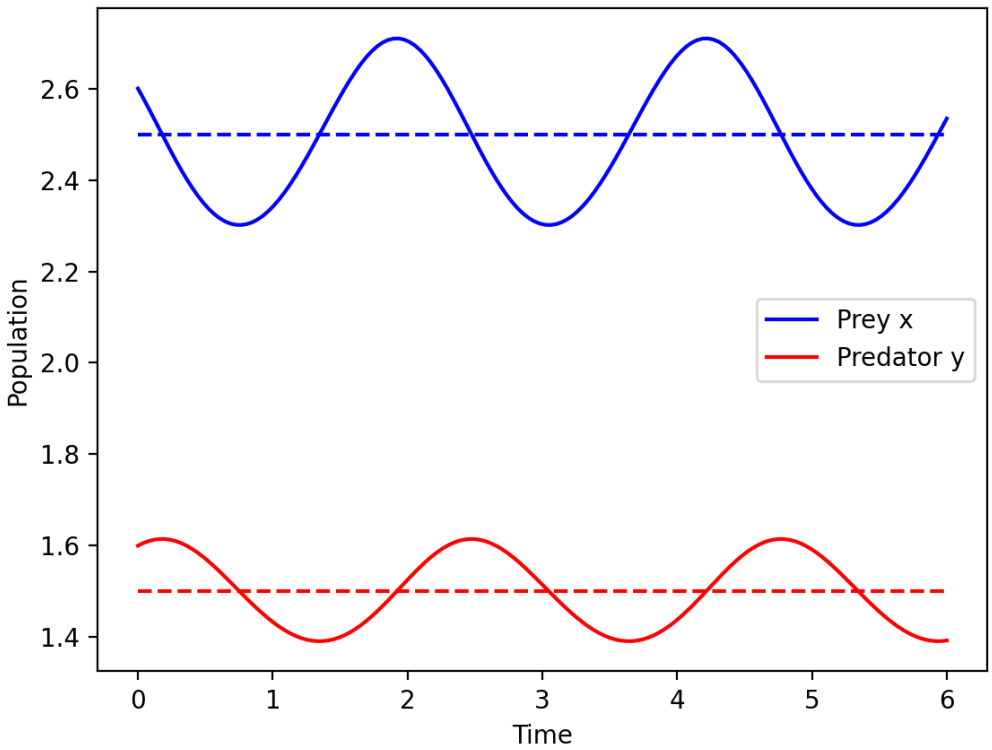
Add lines $x = x_{eq}$ and $y = y_{eq}$ to this plot

In [138…

```
miny=min(predators)
maxy=max(predators)
dely=maxy-miny
plt.plot([xeq,xeq],[miny-dely*0.05,maxy+dely*0.05],"r--")

minx=min(prey)
maxx=max(prey)
delx=maxx-minx
plt.plot([minx-delx*0.05,maxx+delx*0.05],[yeq,yeq],"b--");
```

In [139…

```
#(b)
a_1, a_2, b_1, b_2 = 3, 5/2, 2, 1

x0, y0 = 0.1+5/2, 0.1+3/2 # initial populations

timerange=6.

ts = np.linspace(0, timerange, 150)
P0 = [x0, y0] #initial conditions
Ps = odeint(dP_dt, P0, ts)
prey = Ps[:,0]
predators = Ps[:,1]

plt.figure(13)
plt.plot(ts, prey, "b-", label="Prey x")
plt.plot(ts, predators, "r-", label="Predator y")
plt.xlabel("Time")
plt.ylabel("Population")
plt.legend();
```

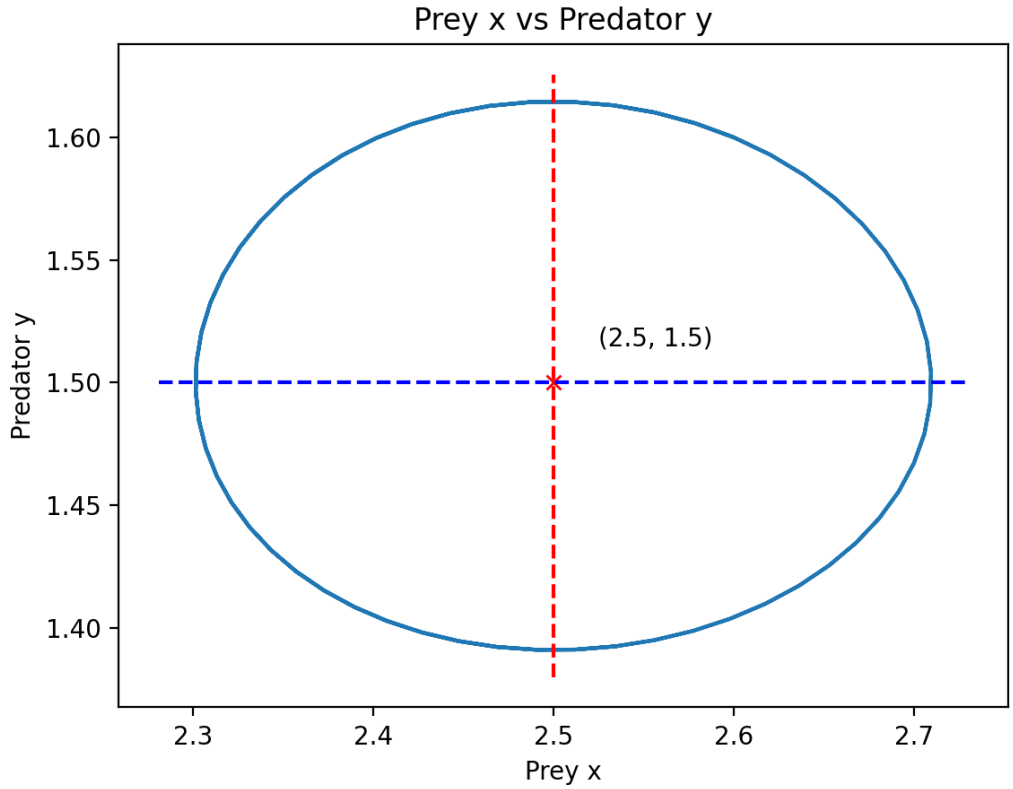


In [141…

```
plt.plot([0,timerange],[xeq,xeq],"b--")
plt.plot([0,timerange],[yeq,yeq],"r--");
```

In [142…

```
plt.figure(14)
plt.plot(prey, predators, "-")
plt.plot([xeq],[yeq],'rx')
plt.text(xeq*(1.01),yeq*(1.01),str((xeq,yeq)))
plt.xlabel("Prey x")
plt.ylabel("Predator y")
plt.title("Prey x vs Predator y");
```

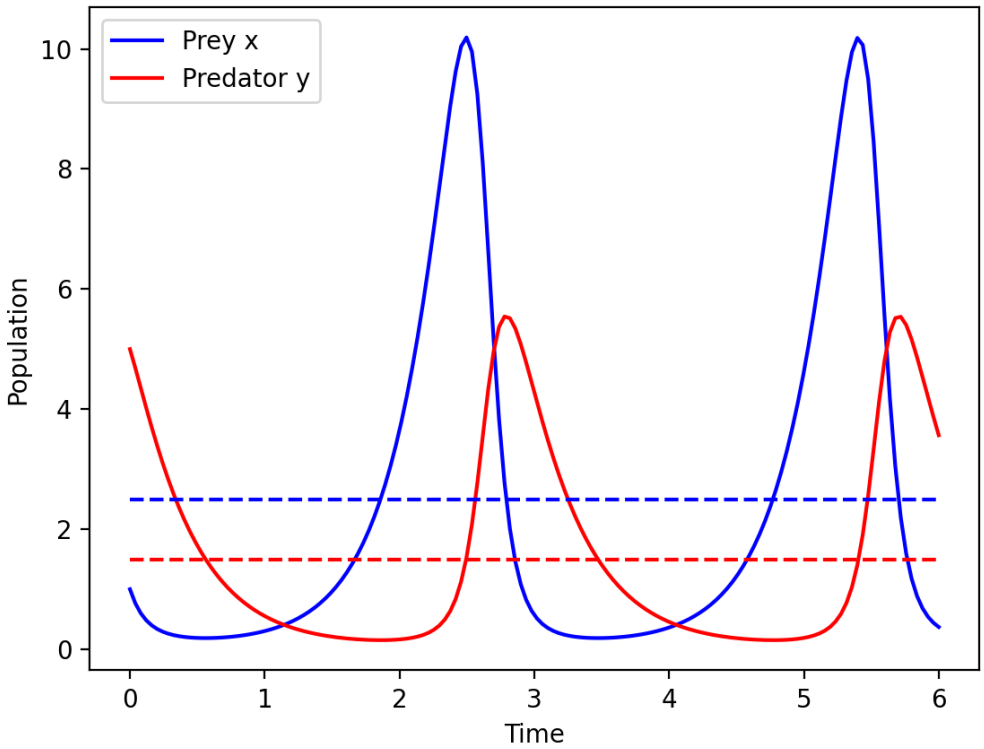


In [143…

```
miny=min(predators)
maxy=max(predators)
dely=maxy-miny
plt.plot([xeq,xeq],[miny-dely*0.05,maxy+dely*0.05],"r--")

minx=min(prey)
maxx=max(prey)
delx=maxx-minx
plt.plot([minx-delx*0.05,maxx+delx*0.05],[yeq,yeq],"b--");
```

In [144…

```
#(c)
a_1, a_2, b_1, b_2 = 3, 5/2, 2, 1

x0, y0 = 1, 5 # initial populations

timerange=6.

ts = np.linspace(0, timerange, 150)
P0 = [x0, y0] #initial conditions
Ps = odeint(dP_dt, P0, ts)
prey = Ps[:,0]
predators = Ps[:,1]

plt.figure(15)
plt.plot(ts, prey, "b-", label="Prey x")
plt.plot(ts, predators, "r-", label="Predator y")
plt.xlabel("Time")
plt.ylabel("Population")
plt.legend();
```

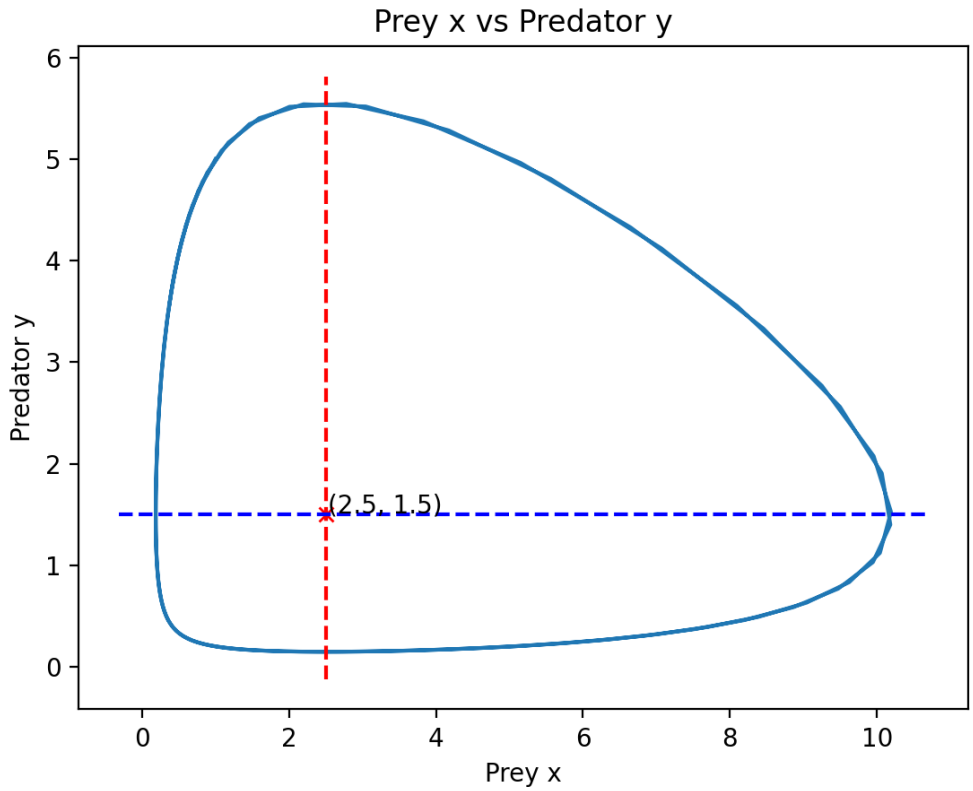


In [145…

```
plt.plot([0,timerange],[xeq,xeq],"b--")
plt.plot([0,timerange],[yeq,yeq],"r--");
```

In [146…

```
plt.figure(16)
plt.plot(prey, predators, "-")
plt.plot([xeq],[yeq],'rx')
plt.text(xeq*(1.01),yeq*(1.01),str((xeq,yeq)))
plt.xlabel("Prey x")
plt.ylabel("Predator y")
plt.title("Prey x vs Predator y");
```



In [147…

```
miny=min(predators)
maxy=max(predators)
dely=maxy-miny
plt.plot([xeq,xeq],[miny-dely*0.05,maxy+dely*0.05],"r--")

minx=min(prey)
maxx=max(prey)
delx=maxx-minx
plt.plot([minx-delx*0.05,maxx+delx*0.05],[yeq,yeq],"b--");
```

In [ ]:

MP307 Practical 3 Discrete Population Models

Name: Cian Mac Liatháin

ID: 11476078

In [1]:
```
%matplotlib notebook
import numpy as np
import matplotlib.pyplot as plt
```

Q.1(*) Geometric Population Model

Consider the geometric population model $P_{n+1} = (1+r)P_n$ for $n = 0, 1, \ldots$ with solution $P_n = P_0 e^{sn}$ for $s = \log(1+r)$ as a model of

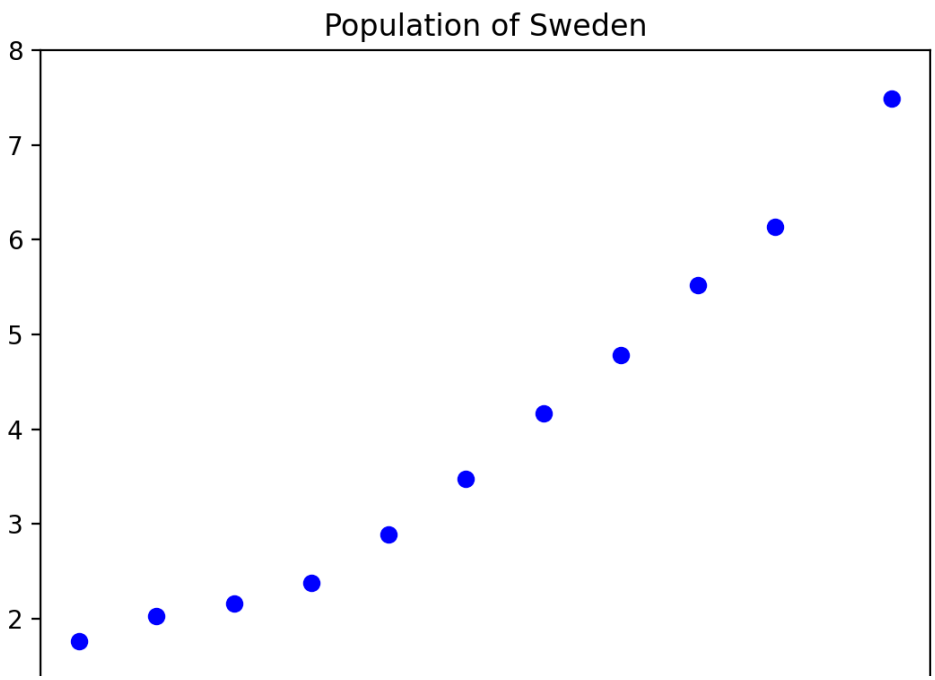
1. Sweden's population with 11 samples from 1750 to 1960
2. The US population with 19 samples from 1790 to 1990.

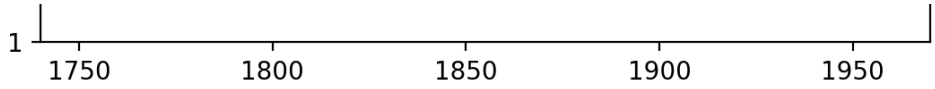
Population of Sweden

The data (in millions) are given in the lists `Year` and `Pop` below.

In [2]:
```
Year = [1750, 1770, 1790, 1810, 1830, 1850, 1870, 1890, 1910, 1930, 1960]
Pop= [1.76, 2.03, 2.158, 2.378, 2.888, 3.48, 4.17, 4.785, 5.522, 6.14, 7.49
```

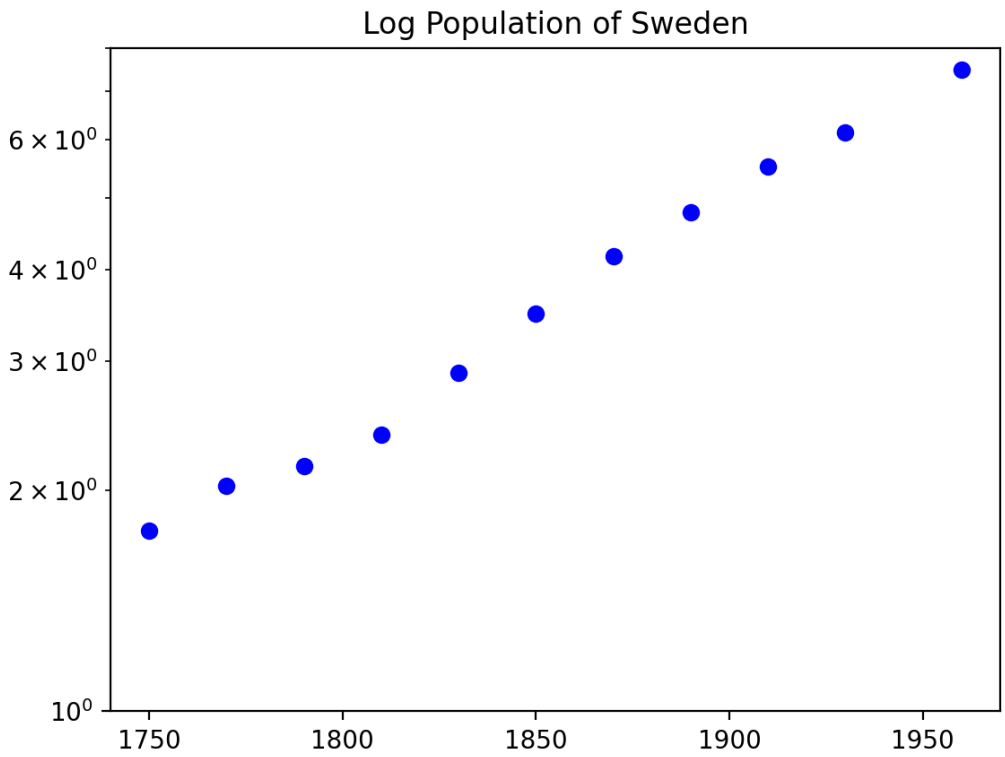
In [3]:
```
plt.figure(1)
plt.plot(Year,Pop,'bo')
plt.axis([1740,1970,1,8])
plt.title('Population of Sweden')
plt.show()
```





Look at t vs log(P):

In [4]:
```
plt.figure(2) # new figure
plt.plot(Year,Pop,'bo')
plt.yscale("log")
plt.axis([1740,1970,1,8])
plt.title('Log Population of Sweden')
plt.show()
```



The log plot suggest a linear relationship to between $\log P$ and t of the form $\log P(t) = st + b$ for some s, b. Thus $P(t) = P_0 e^{st}$ with $P_0 = \exp(b)$.

In [5]:
```
log_Pop=np.log(Pop) # take log of each entry in S_Pop
```

In [6]:
```
print(log_Pop)
```

```
[0.56531381 0.70803579 0.76918187 0.8662598  1.06056422 1.24703229
 1.42791604 1.56548602 1.70874011 1.81482474 2.0135688 ]
```

Use a least squares approximation to find a line as explained in the notes for matching data x_i to y_i.

In [7]:
```
N=len(Year) # no of elements in S_Year
sumx=0; sumy=0; sumxx=0; sumxy=0;
for i in range(N):
    sumx=sumx+Year[i]
    sumy=sumy+log_Pop[i]
    sumxx=sumxx+Year[i]**2
    sumxy=sumxy+log_Pop[i]*Year[i]
Xbar=sumx/N
Ybar=sumy/N
XXbar=sumxx/N
XYbar=sumxy/N
```

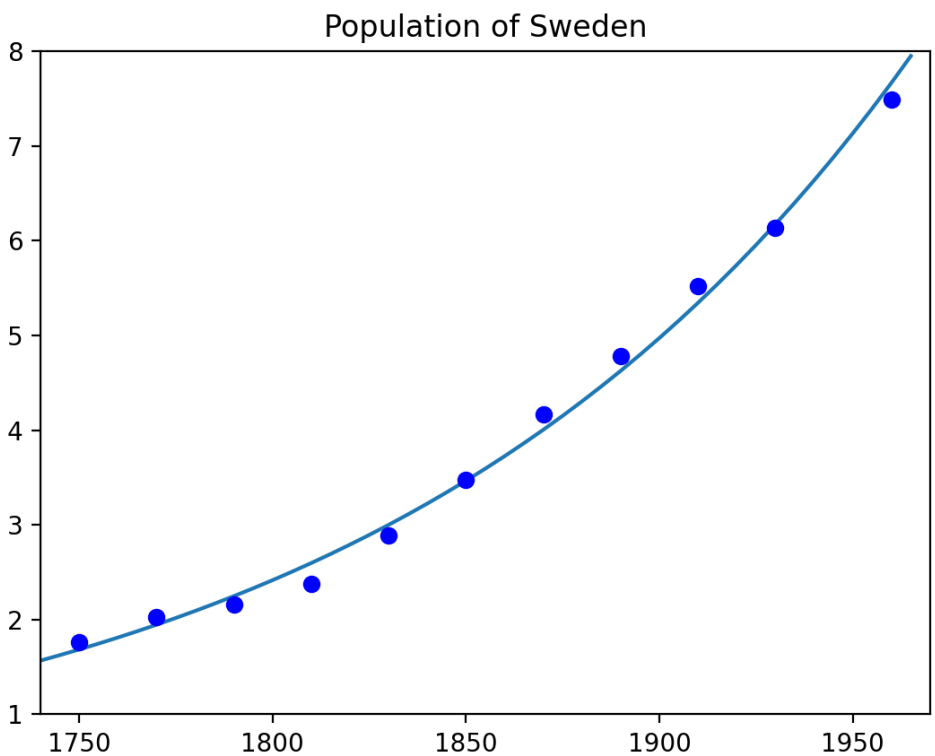
In [8]:
```
s=(XYbar-Xbar*Ybar)/(XXbar-Xbar**2)
b=(XXbar*Ybar-Xbar*XYbar)/(XXbar-Xbar**2)

P_0=np.exp(b)
print(s,P_0)
```

```
0.0072182711579564076 5.500654960458621e-06
```

Plot population $P(t) = P_0 e^{st}$ vs t.

In [9]:
```
t=np.arange(1740.0,1970.,5.)
P_LS=P_0*np.exp(s*t) # least squares approximation
```

In [10]:
```
plt.figure(3)
plt.plot(t,P_LS)
plt.plot(Year,Pop,'bo')
plt.axis([1740,1970,1,8])
plt.title('Population of Sweden')
plt.show()
```



In [ ]:

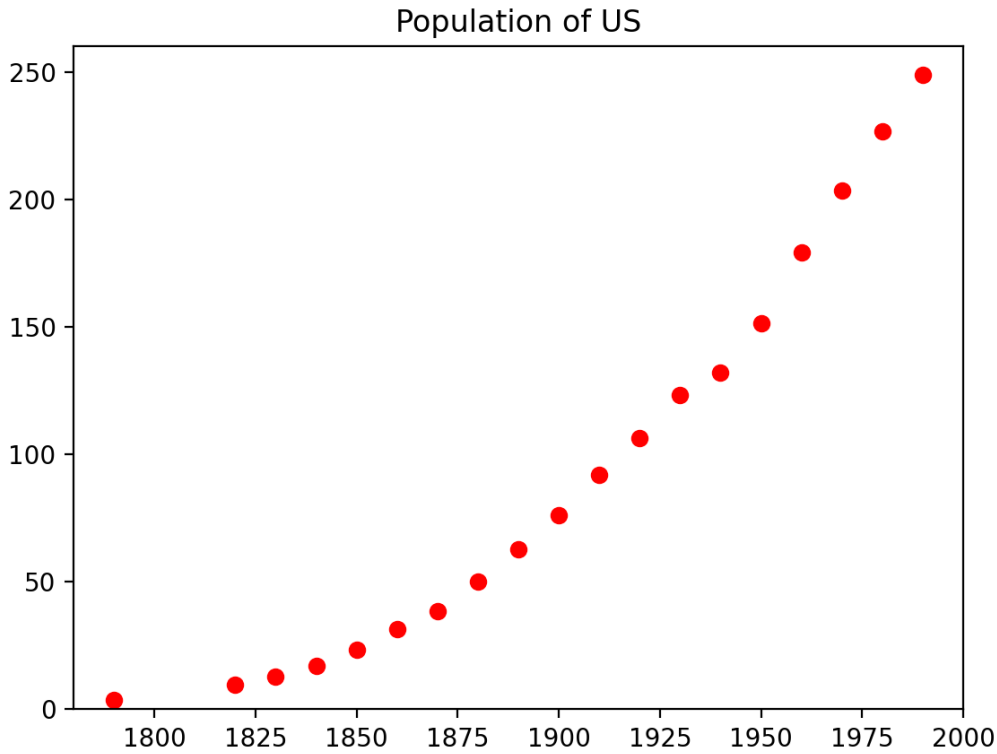
Repeat this analysis for the US population as follows:

In [11]:

```
Year = [1790, 1820, 1830, 1840, 1850, 1860, 1870, 1880, 1890, 1900, 1910, 1
Pop= [3.79, 9.6, 12.9, 17.1, 23.2, 31.4, 38.6, 50.2, 62.9, 76.0, 92.0, 106.
```

In [12]:

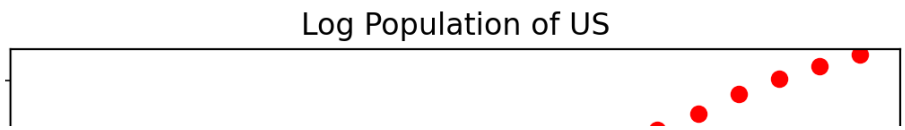
```
plt.figure(4)
plt.plot(Year,Pop,'ro')
plt.axis([1780,2000,0,260])
plt.title('Population of US')
plt.show()
```

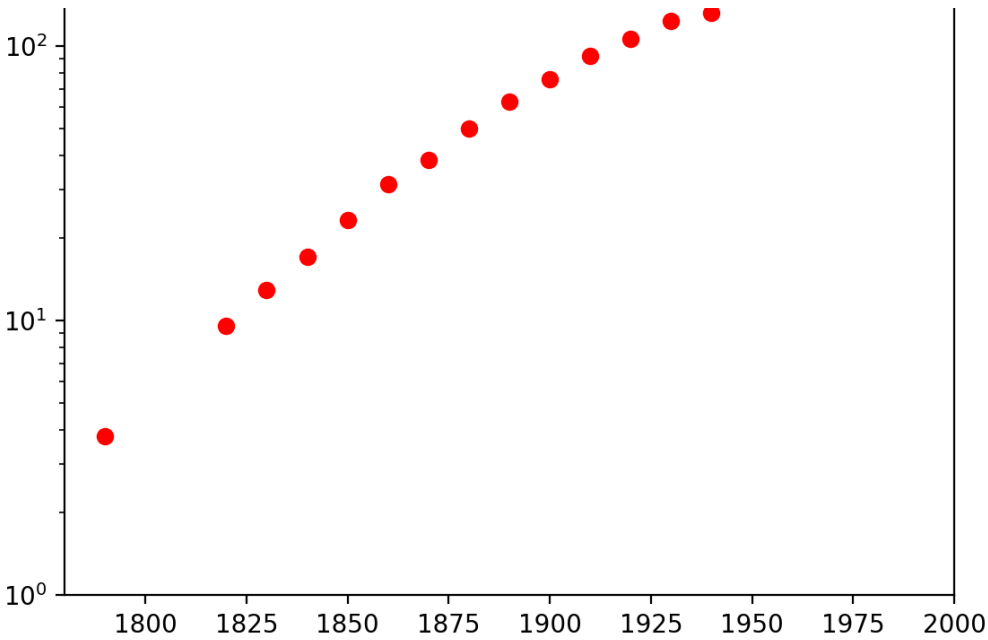


Look at t vs log(P):

In [15]:

```
plt.figure(5) # new figure
plt.plot(Year,Pop,'ro')
plt.yscale("log")
plt.axis([1780,2000,1,260])
plt.title('Log Population of US')
plt.show()
```





In [16]:
```
log_Pop=np.log(Pop)
```

In [17]:
```
print(log_Pop)
```

```
[1.33236602 2.2617631  2.55722731 2.83907846 3.14415228 3.44680789
 3.65325228 3.91601503 4.14154616 4.33073334 4.52178858 4.66814499
 4.81380905 4.88280192 5.01926462 5.18906038 5.31468272 5.42292153
 5.51624735]
```

In [18]:
```
N=len(Year) # no of elements in US_Year
sumx=0; sumy=0; sumxx=0; sumxy=0;
for i in range(N):
    sumx=sumx+Year[i]
    sumy=sumy+log_Pop[i]
    sumxx=sumxx+Year[i]**2
    sumxy=sumxy+log_Pop[i]*Year[i]
Xbar=sumx/N
Ybar=sumy/N
XXbar=sumxx/N
XYbar=sumxy/N
```

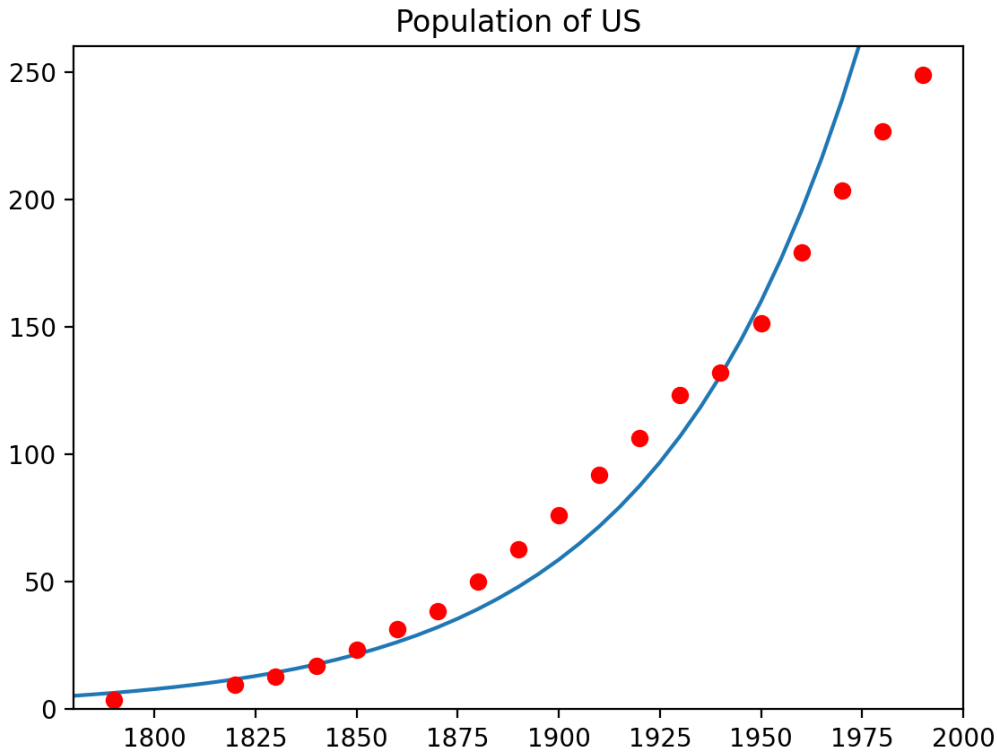
In [20]:
```
s=(XYbar-Xbar*Ybar)/(XXbar-Xbar**2)
b=(XXbar*Ybar-Xbar*XYbar)/(XXbar-Xbar**2)

P_0=np.exp(b)
print(s,P_0)
```

```
0.020051106626461583 1.6718326017274779e-15
```

In [21]:
```
t=np.arange(1780.0,2000.0,5.)
P_LS=P_0*np.exp(s*t) # least squares approximation
```

In [22]:

```
plt.figure(6)
plt.plot(t,P_LS)
plt.plot(Year,Pop,'ro')
plt.axis([1780,2000,0,260])
plt.title('Population of US')
plt.show()
```



Q.2 (*) Discrete Verhulst Model

The general discrete Verhulst model of population growth is as follows

$$P_{n+1} = \frac{1+r}{1+\frac{rP_n}{K}} P_n$$

where $P_n \to K$ as $n \to \infty$.

For the choice of parameters $P_0 = 3.79$, $r = 0.35$ and $K = 300$ compare the Verhulst model to the US population from 1790 to 1990. Explain what you see.

In [23]:

```
print(Year)
```

```
[1790, 1820, 1830, 1840, 1850, 1860, 1870, 1880, 1890, 1900, 1910, 1920, 19
30, 1940, 1950, 1960, 1970, 1980, 1990]
```

In [24]:

```
P_0=3.79
r=0.35
K=300.0
```

In [25]:

```
# plot Verhulst pop PV from 1790 to 1990 with 10 year intervals
PV=[0]*20 # list for Verhulst P initialized with 20 zeros
YV=[0]*20 # list for years initialized with 20 zeros

PV[0]=P_0
YV[0]=1790
# choose 10 year intervals
for n in range(1,20):
    YV[n]=1790+10*n
    PV[n]=(1+r)*PV[n-1]/(1+r*PV[n-1]/K)
```

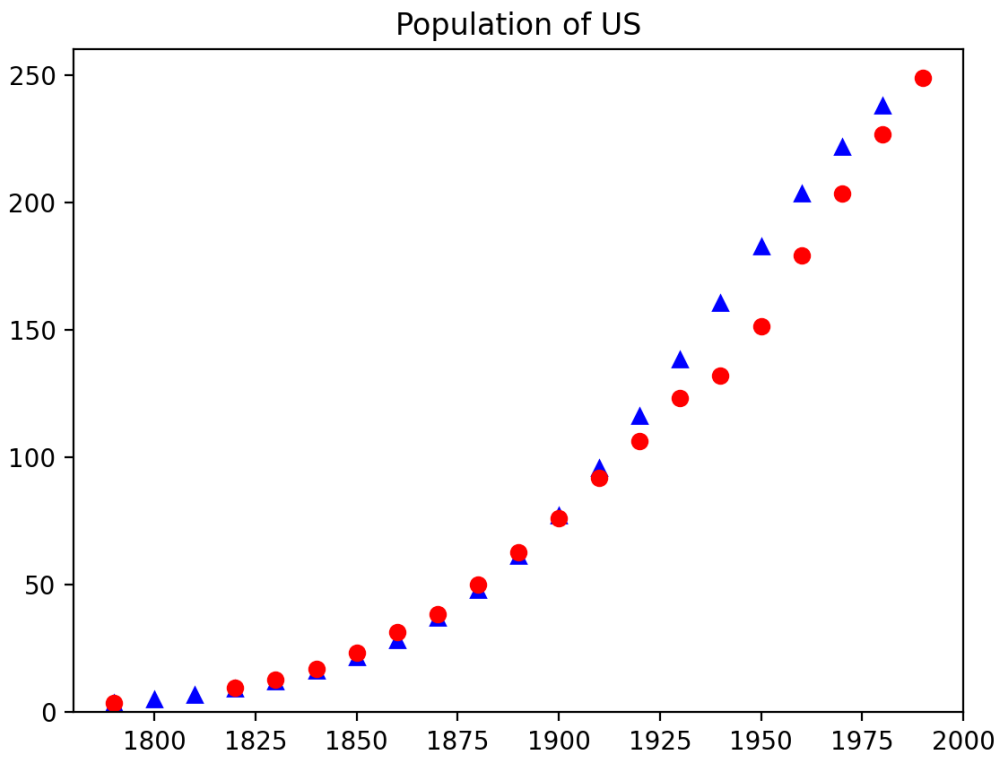
In [26]:

```
print(PV)
```

```
[3.79, 5.093976135520772, 6.836240197791433, 9.15590034404303, 12.229827865
107413, 16.278010802773355, 21.56575898309768, 28.39924806493404, 37.109457
41970735, 48.0188229724253, 61.386423256309854, 77.33325731948169, 95.76020
718123476, 116.28490230208658, 138.23136107104716, 160.69678033606021, 182.
69000801767106, 203.3004004123143, 221.83893811890553, 237.9088727981478]
```

In [27]:

```
plt.figure(7)
plt.plot(YV,PV,'b^')
plt.plot(Year,Pop,'ro')
plt.axis([1780,2000,0,260])
plt.title('Population of US')
plt.show()
```



We can see that the Verhulst model provides a very accurate estimation of US population from 1790 until 1925. The higher estimation than the actual US population from 1925 to 1990 may be due to unexpected factors effecting the population size like the First and Second World Wars and the Great Depression.

Q.3 (*) Cohort Model

Consider the human population divided into three age groups $0-14$, $15-39$ and ≥ 40 with population size P_i , $i=1,2,3$ and yearly birth rates of 0, 0.06 and 0 and death rates of 0.005, 0.01 and 0.015, respectively.

(a). Show that $\mathbf{P}(t+1)=\mathbf{A}\mathbf{P}(t)$ where

$$\mathbf{P}=\begin{bmatrix} P_1 \\ P_2 \\ P_3 \end{bmatrix}, \quad \mathbf{A}=\begin{bmatrix} 0.92867 & 0.06 & 0 \\ 0.06\,63 & 0.9504 & 0 \\ 0 & 0.0\,396 & 0.985 \end{bmatrix}.$$

(b). Find the largest eigenvalue of $\mathbf{A}$ and describe the long-term behaviour of this system.

(c). Consider the growth of the population with the following initial populations. How long does it take for the total population to double in each case?

(i) $P_1=200$, $P_2=P_3=400$.

(ii) $P_1=400$, $P_2=P_3=300$.

(iii) $P_1=200$, $P_2=500$ and $P_3=300$.

Explain what you see.

(a)

$$P_1(t+1)=b_1P_1(t)+\tfrac{14}{15}(1-d_1)P_1(t)+b_2P_2(t)+b_3P_3(t)=0.92867P_1(t)+0.06P_2($$

$$P_2(t+1)=\tfrac{1}{15}(1-d_1)P_1(t)+\tfrac{24}{25}(1-d_2)P_2(t)=0.0663P_1(t)+0.9504P_2(t)$$

$$P_3(t+1)=\tfrac{1}{25}(1-d_2)P_2(t)+(1-d_3)P_3(t)=0.0396P_2(t)+0.985P_3(t)$$

In [31]:

```
b=[0, 0.06 ,0]
d=[0.005, 0.01, 0.015]
```

In [32]:

```
A=np.mat([[b[0] + 14/15 - 14/15 * d[0], b[1], b[2]],
          [1/15*(1-d[0]), 24/25*(1-d[1]), 0],
          [0, 1/25*(1-d[1]), 1-d[2]]])
# recall Python indexing begins at 0
```

In [33]:

```
print(A)
```

```
[[0.92866667 0.06       0.         ]
 [0.06633333 0.9504     0.         ]
 [0.         0.0396     0.985      ]]
```

$P(t+1)=AP(t)$

Find eigenvalues and eigenvectors

In [34]:
```
X, V=np.linalg.eig(A)
print(X) # eigen values
print(V) # columns are right eigenvectors
```

```
[0.985      1.00354962 0.87551705]
[[ 0.          0.32180453  0.72790385]
 [ 0.          0.40162787 -0.64479684]
 [ 1.          0.85740124  0.23322312]]
```

(b) Can extract first, second, third eigenvalues/vectors as follows:

In [35]:
```
lambda0=X[0]
u0=V[:,0] # This is a 3 by 1 matrix
print(lambda0,u0)
```

```
0.985 [[0.]
 [0.]
 [1.]]
```

In [36]:
```
np.matmul(A,u0)/lambda0 # A u = lambda u
```

Out[36]:
```
matrix([[0.],
        [0.],
        [1.]])
```

In [37]:
```
lambda1=X[1]
u1=V[:,1]
print(lambda1,u1)
```

```
1.0035496159842685 [[0.32180453]
 [0.40162787]
 [0.85740124]]
```

In [38]:
```
np.matmul(A,u1)/lambda1 # A u = lambda u
```

Out[38]:
```
matrix([[0.32180453],
        [0.40162787],
        [0.85740124]])
```

In [39]:
```
lambda2=X[2]
u2=V[:,2]
print(lambda2,u2)
```

```
0.8755170506823979 [[ 0.72790385]
 [-0.64479684]
 [ 0.23322312]]
```

In [40]:
```
np.matmul(A,u2)/lambda2 # A u = lambda u
```

Out[40]:
```
matrix([[ 0.72790385],
        [-0.64479684],
        [ 0.23322312]])
```

In [41]:
```
max(lambda0, lambda1, lambda2)
```

Out[41]:
```
1.0035496159842685
```

We can see that the largest eigenvalue is lambda1 with a value of 1.0035496159842685.

As this is greater than 1, slow exponential growth will occur in this model

(c) Initial poplulation $P_1 = P_2 = P_3 = 100$.

In [42]:
```
P0=[100,100,100]
```

In [43]:
```
np.matmul(A,P0) # population after one year
```

Out[43]:
```
matrix([[ 98.86666667, 101.67333333, 102.46      ]])
```

In [44]:
```
n=20 # 20 years
An=np.linalg.matrix_power(A,n) # matrix A^n
print(An)
```

```
[[0.48657128 0.47022728 0.        ]
 [0.51986238 0.65689805 0.        ]
 [0.24436408 0.51787528 0.73913643]]
```

In [62]:
```
Pn=np.matmul(An,P0) # population after n years
print(Pn)
```

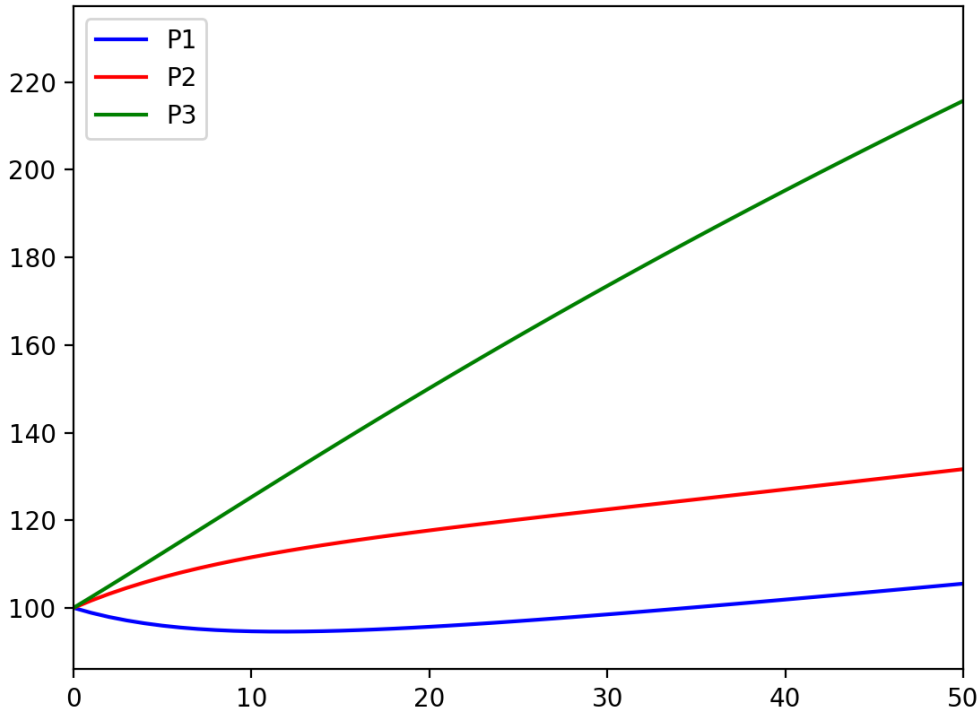
```
[[ 95.67985614 117.67604324 150.13757869]]
```

In [64]:
```
Pn.sum()
```

Out[64]:
```
363.49347806660535
```

Define lists P1, P2, P3 giving population at time 0,1,2,3, etc

In [51]:
```
nit=50 # nit is no of time steps

#initialise list to zero

P1=[0]*(nit+1)
P2=[0]*(nit+1)
P3=[0]*(nit+1)

#Given initial population

P1[0],P2[0],P3[0]=100,100,100

# compute P1,P2,P3
for n in range(1,nit+1):
    AP=np.matmul(A,[P1[n-1],P2[n-1],P3[n-1]]) # AP[n-1]
    P1[n],P2[n],P3[n]=AP[0,0],AP[0,1],AP[0,2]

# plotting

t=np.arange(0,nit+1,1) #  time steps
plt.figure()
P1plot,=plt.plot(t,P1,'b-')
P2plot,=plt.plot(t,P2,'r-')
P3plot,=plt.plot(t,P3,'g-')
plt.legend((P1plot,P2plot,P3plot),('P1','P2','P3'))
Pmax=max(max(P1),max(P2),max(P3))
Pmin=min(min(P1),min(P1),min(P1))
plt.axis([0,nit,Pmin/1.1,Pmax*1.1])
plt.show()
```



Compare populations to eigenvector for largest eigenvalue

In [52]:
```
print(P2[50]/P1[50],u1[1,0]/u1[0,0]) # compare first and second cohorts to
```

```
1.247744063179402 1.248049155293366
```

In [53]:
```
print(P3[50]/P1[50],u1[2,0]/u1[0,0]) # compare first and third cohorts to e
```

```
2.0444670328229053 2.664354161915348
```

(c)

(i) $P_1 = 200, P_2 = P_3 = 400$

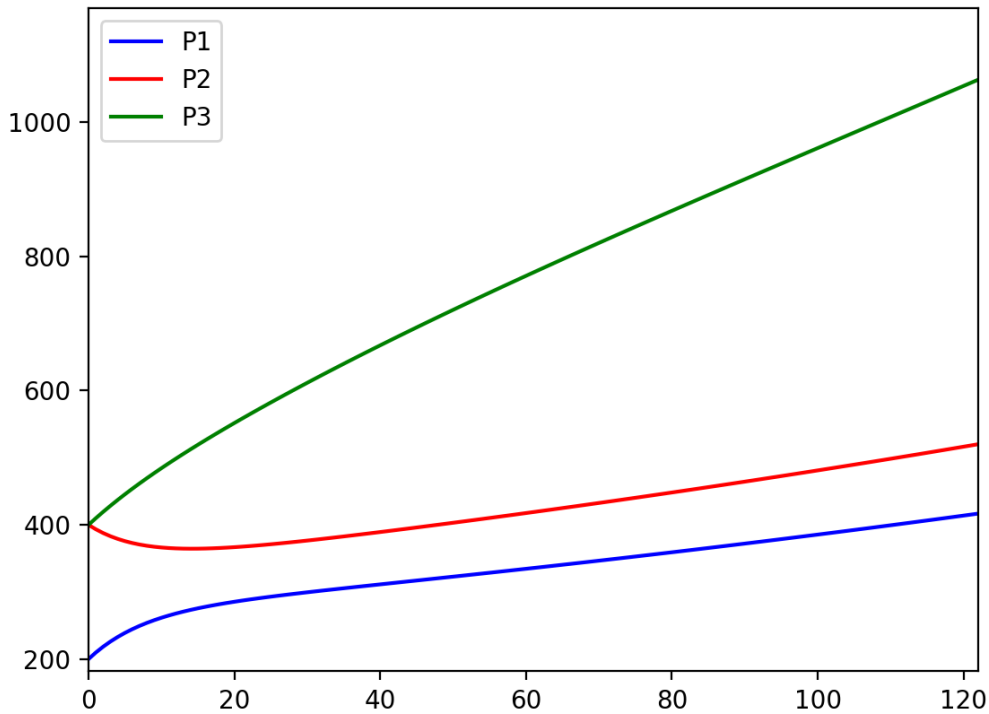
In [54]:
```
P0=[200,400,400]

totalPopulation = sum(P0)
```

In [55]:
```
years = 0
for n in range(300):
    An=np.linalg.matrix_power(A,n) # matrix A^n
    Pn=np.matmul(An,P0) # population after n years
    if Pn.sum() > totalPopulation*2:
        years = n
        break;
print("The total population doubles after ", n, " years.")
```

```
The total population doubles after  122  years.
```

In [56]:

```
nit=122 # nit is no of time steps

#initialise list to zero

P1=[0]*(nit+1)
P2=[0]*(nit+1)
P3=[0]*(nit+1)

#Given initial population

P1[0],P2[0],P3[0]=P0[0], P0[1], P0[2]

# compute P1,P2,P3
for n in range(1,nit+1):
    AP=np.matmul(A,[P1[n-1],P2[n-1],P3[n-1]]) # AP[n-1]
    P1[n],P2[n],P3[n]=AP[0,0],AP[0,1],AP[0,2]

# plotting

t=np.arange(0,nit+1,1) #  time steps
plt.figure()
P1plot,=plt.plot(t,P1,'b-')
P2plot,=plt.plot(t,P2,'r-')
P3plot,=plt.plot(t,P3,'g-')
plt.legend((P1plot,P2plot,P3plot),('P1','P2','P3'))
Pmax=max(max(P1),max(P2),max(P3))
Pmin=min(min(P1),min(P1),min(P1))
plt.axis([0,nit,Pmin/1.1,Pmax*1.1])
plt.show()
```



In [57]:

```
print(P2[122]/P1[122],u1[1,0]/u1[0,0]) # compare first and second cohorts t
```

```
1.248049187924413 1.248049155293366
```

In [58]:

```
print(P3[122]/P1[122],u1[2,0]/u1[0,0]) # compare first and third cohorts to
```

```
2.5512020932531168 2.664354161915348
```

(c)

(ii) $P_1 = 400, P_2 = P_3 = 300$

In [62]:

```
P0=[400,300,300]

totalPopulation = sum(P0)
print("Starting population: ",totalPopulation)
```

```
Starting population:  1000
```

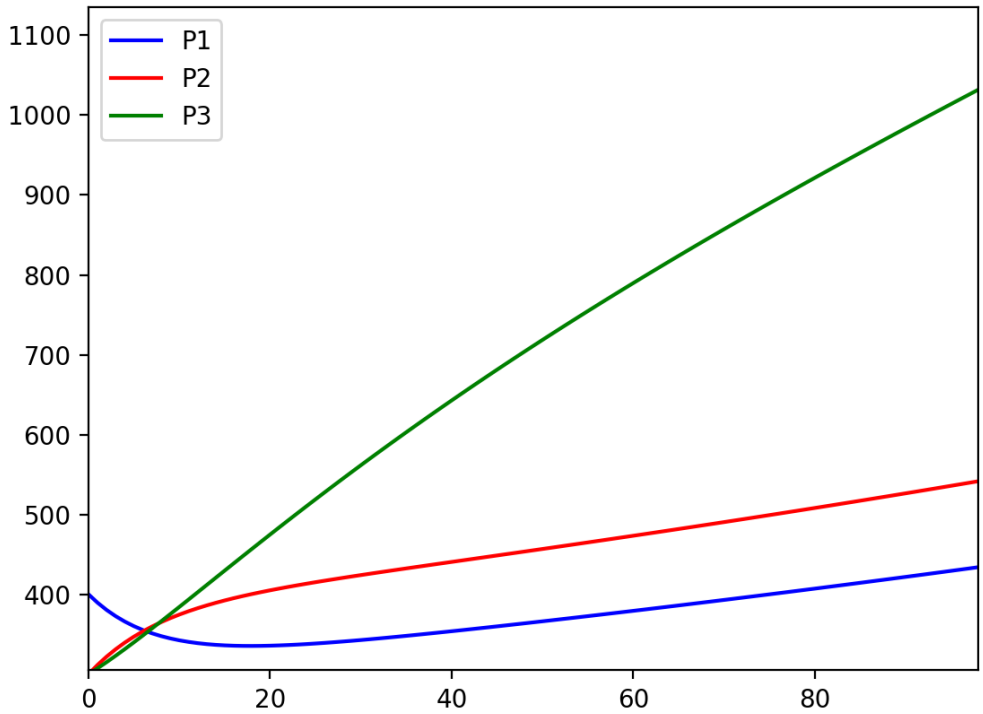
In [63]:

```
years = 0
for n in range(300):
    An=np.linalg.matrix_power(A,n) # matrix A^n
    Pn=np.matmul(An,P0) # population after n years
    if Pn.sum() > totalPopulation*2:
        years = n
        break;
print("The total population doubles after ", n, " years.")
```

```
The total population doubles after  98  years.
```

In [64]:

```
nit=98 # nit is no of time steps

#initialise list to zero

P1=[0]*(nit+1)
P2=[0]*(nit+1)
P3=[0]*(nit+1)

#Given initial population

P1[0],P2[0],P3[0]=P0[0], P0[1], P0[2]

# compute P1,P2,P3
for n in range(1,nit+1):
    AP=np.matmul(A,[P1[n-1],P2[n-1],P3[n-1]]) # AP[n-1]
    P1[n],P2[n],P3[n]=AP[0,0],AP[0,1],AP[0,2]

# plotting

t=np.arange(0,nit+1,1) #  time steps
plt.figure()
P1plot,=plt.plot(t,P1,'b-')
P2plot,=plt.plot(t,P2,'r-')
P3plot,=plt.plot(t,P3,'g-')
plt.legend((P1plot,P2plot,P3plot),('P1','P2','P3'))
Pmax=max(max(P1),max(P2),max(P3))
Pmin=min(min(P1),min(P1),min(P1))
plt.axis([0,nit,Pmin/1.1,Pmax*1.1])
plt.show()
```



In [66]:
```
print(P2[98]/P1[98],u1[1,0]/u1[0,0]) # compare first and second cohorts to
```

```
1.248048146439516 1.248049155293366
```

In [67]:
```
print(P3[98]/P1[98],u1[2,0]/u1[0,0]) # compare first and third cohorts to e
```

```
2.377783277474929 2.664354161915348
```

(iii) $P_1 = 200, P_2 = 500, P_3 = 300$

In [69]:
```
P0=[200,500,300]

totalPopulation = sum(P0)
print("Starting population: ",totalPopulation)
```

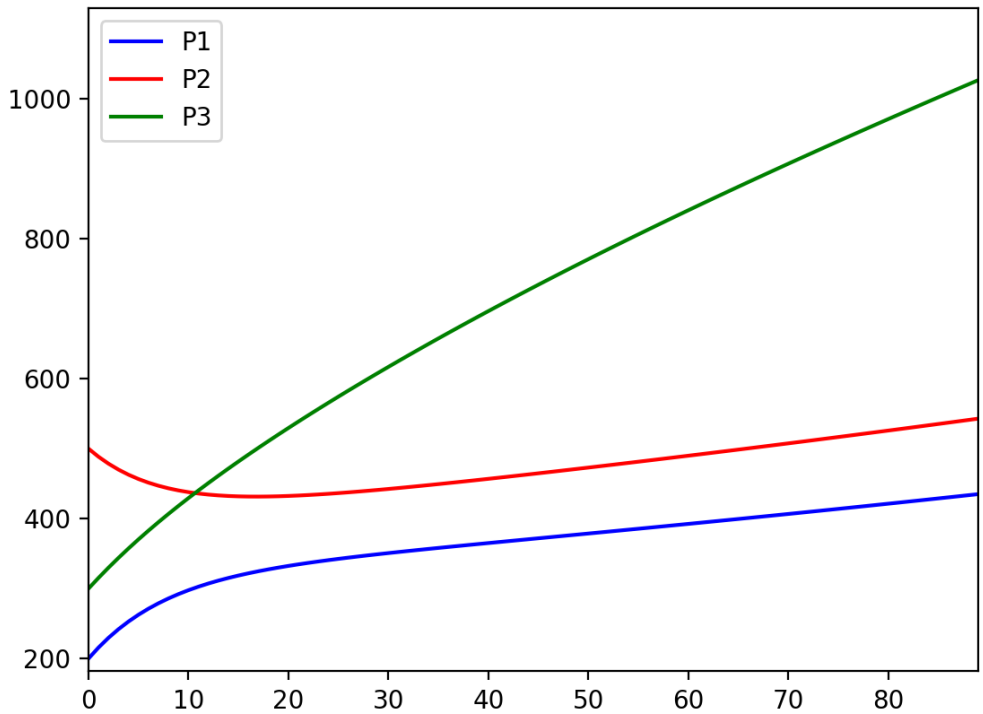
```
Starting population:  1000
```

In [70]:
```
years = 0
for n in range(300):
    An=np.linalg.matrix_power(A,n) # matrix A^n
    Pn=np.matmul(An,P0) # population after n years
    if Pn.sum() > totalPopulation*2:
        years = n
        break;
print("The total population doubles after ", n, " years.")
```

```
The total population doubles after  89  years.
```

In [71]:

```
nit=89 # nit is no of time steps

#initialise list to zero

P1=[0]*(nit+1)
P2=[0]*(nit+1)
P3=[0]*(nit+1)

#Given initial population

P1[0],P2[0],P3[0]=P0[0], P0[1], P0[2]

# compute P1,P2,P3
for n in range(1,nit+1):
    AP=np.matmul(A,[P1[n-1],P2[n-1],P3[n-1]]) # AP[n-1]
    P1[n],P2[n],P3[n]=AP[0,0],AP[0,1],AP[0,2]

# plotting

t=np.arange(0,nit+1,1) #  time steps
plt.figure()
P1plot,=plt.plot(t,P1,'b-')
P2plot,=plt.plot(t,P2,'r-')
P3plot,=plt.plot(t,P3,'g-')
plt.legend((P1plot,P2plot,P3plot),('P1','P2','P3'))
Pmax=max(max(P1),max(P2),max(P3))
Pmin=min(min(P1),min(P1),min(P1))
plt.axis([0,nit,Pmin/1.1,Pmax*1.1])
plt.show()
```



In [73]:

```
print(P2[89]/P1[89],u1[1,0]/u1[0,0]) # compare first and second cohorts to
```

```
1.248053340193975 1.248049155293366
```

In [74]:

```
print(P3[89]/P1[89],u1[2,0]/u1[0,0]) # compare first and third cohorts to e
```

```
2.3601791730634036 2.664354161915348
```

We can see from the three different initial populations that regardless of initial population, over time P1, P2 and P3 grow exponentially, in line with the dominant eigenvalue of 1.004

Q.4 (*) Chaotic Models

Consider the Ricker model of Salmon population size as follows

$$P_{n+1} = re^{-\frac{P_n}{K}} P_n$$

where r is the rate and K is another parameter. For simplicity choose $K = 500$ and consider the behaviour of this system for

(a). $r < 1$, with initial population of 1000.

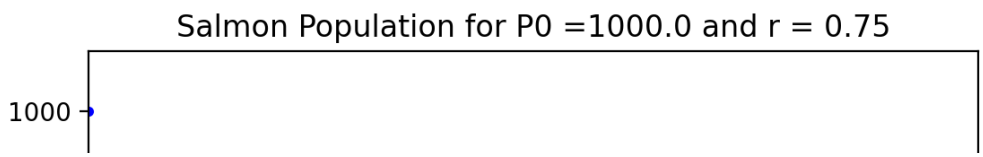
(b). $1 < r < e^2$, for initial population 100, 500 and 1000.

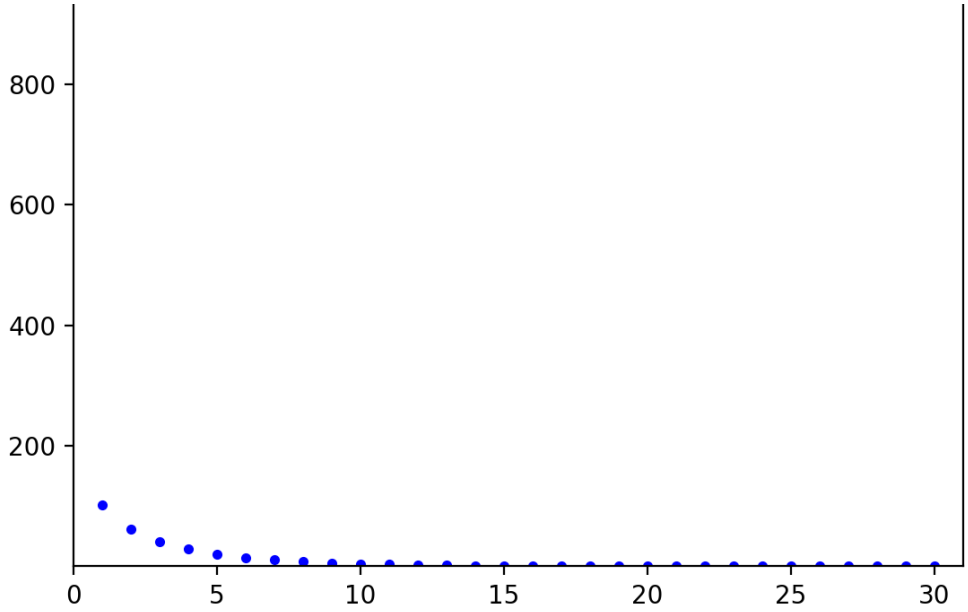
(c). $r > e^2$, for initial population 100, 500 and 1000.

Explain what you see.

In [75]:

```
# initial values
P_0=1000.
K=500.
r=0.75

nit=30 # nit is no of time steps

t=np.arange(0,nit+1,1) #  time steps

P=[0]*(nit+1) # list for P initialized with nit zeros

P[0]=P_0 # given initial population
for n in range(1,nit+1):
    P[n]=r*P[n-1]*np.exp(-P[n-1]/K)

# plotting
plt.figure()
plt.plot(t,P,'b.')
Pmax=max(P)
Pmin=min(P)
plt.axis([0,nit+1,Pmin/1.1,Pmax*1.1])
plt.title('Salmon Population for P0 =' +str(P_0)+ ' and r = ' +str(r))
plt.show()
```





Population dies out for $r < 1$.

Explore the other values of r where one can find convergence to non-zero population for $1 < r < e^2$ and period doubling phenomena and chaos for $r > e^2$.

In [76]:

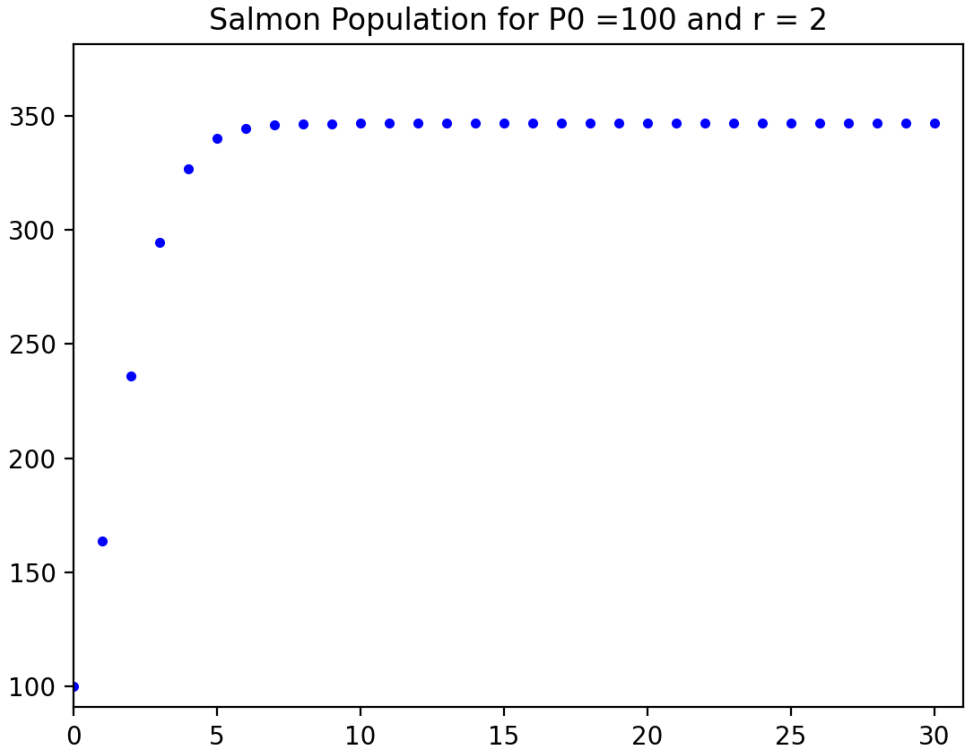
```
e2=np.exp(2)
print(e2)
```

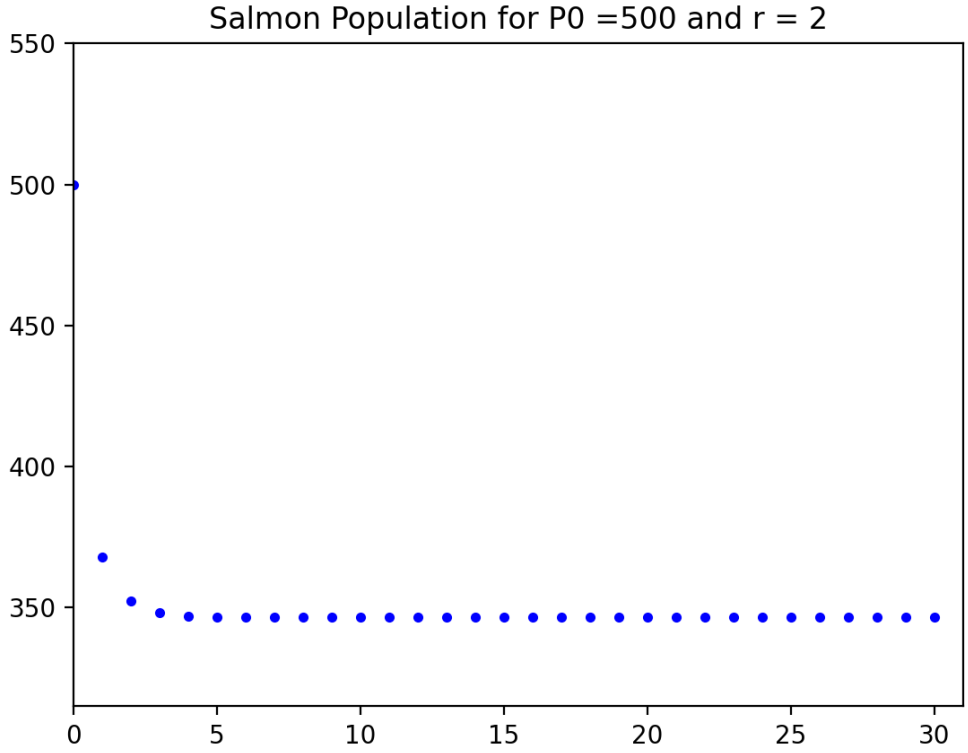
```
7.38905609893065
```

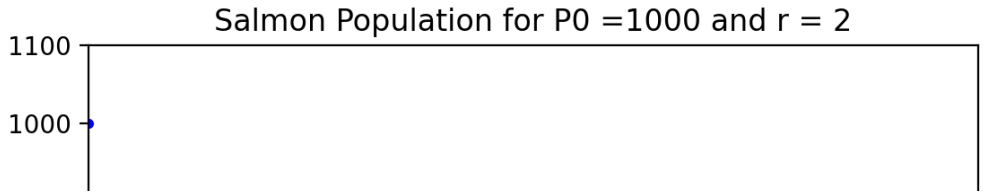
(b). $1 < r < e^2$, for initial population 100, 500 and 1000.

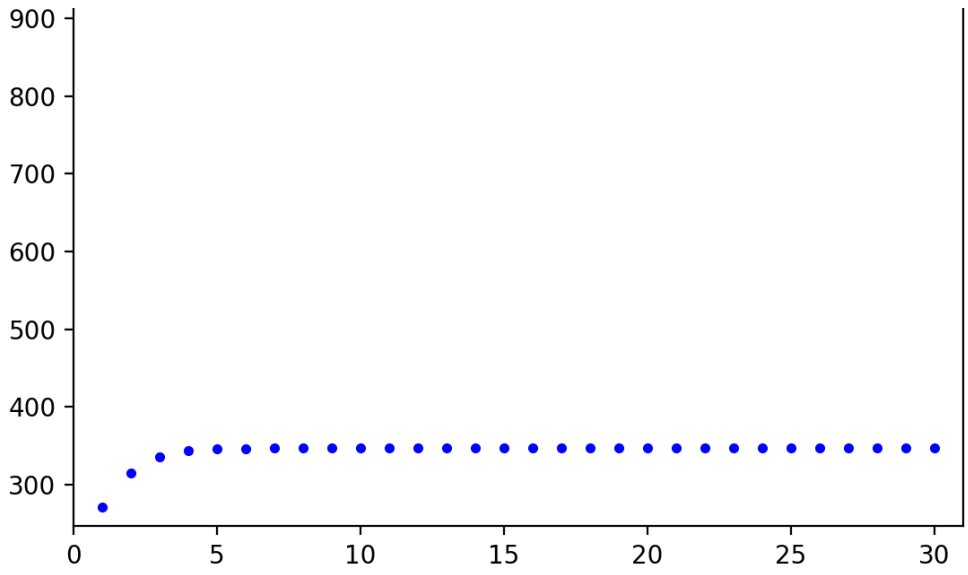
In [103…

```
#r = 2
populations = [100, 500, 1000]
for i in range(3):
    # initial values
    P_0=populations[i]
    K=500
    r=2

    nit=30 # nit is no of time steps

    t=np.arange(0,nit+1,1) #  time steps

    P=[0]*(nit+1) # list for P initialized with nit zeros

    P[0]=P_0 # given initial population
    for n in range(1,nit+1):
        P[n]=r*P[n-1]*np.exp(-P[n-1]/K)

    # plotting
    plt.figure()
    plt.plot(t,P,'b.')
    Pmax=max(P)
    Pmin=min(P)
    plt.axis([0,nit+1,Pmin/1.1,Pmax*1.1])
    plt.title('Salmon Population for P0 =' +str(P_0)+ ' and r = ' +str(r))
    plt.show()
```


Salmon Population for P0 =100 and r = 2
350
300
250
200
150
100
0
5
10
15
20
25
30


Salmon Population for P0 =500 and r = 2
550
500
450
400
350
0
5
10
15
20
25
30

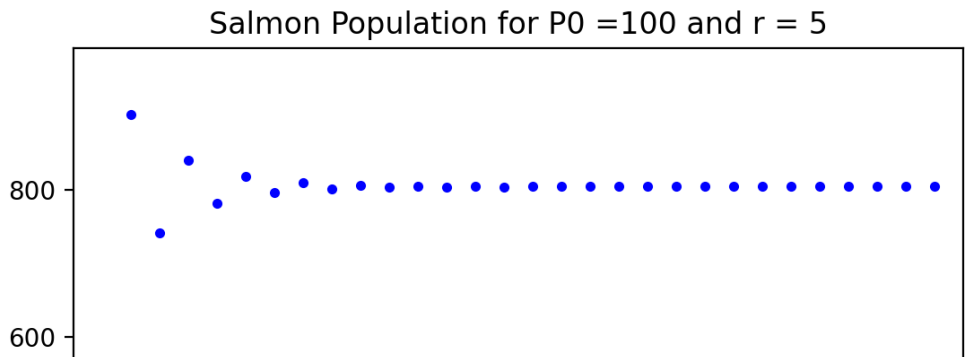

Salmon Population for P0 =1000 and r = 2
1100
1000

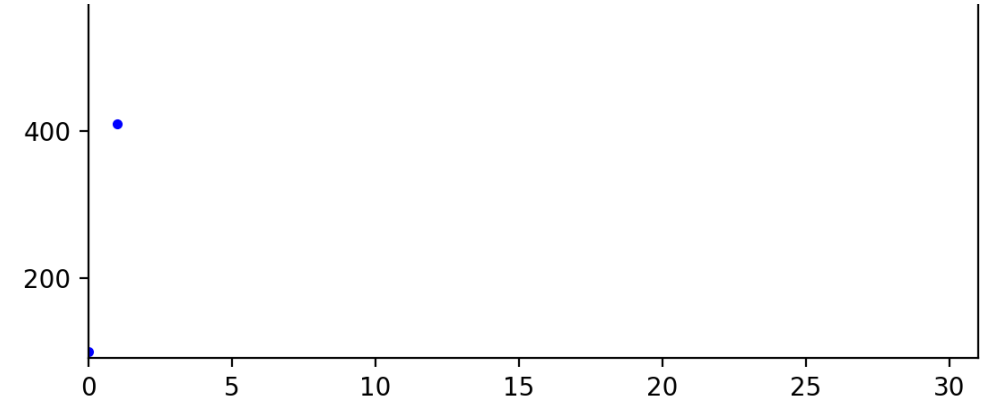


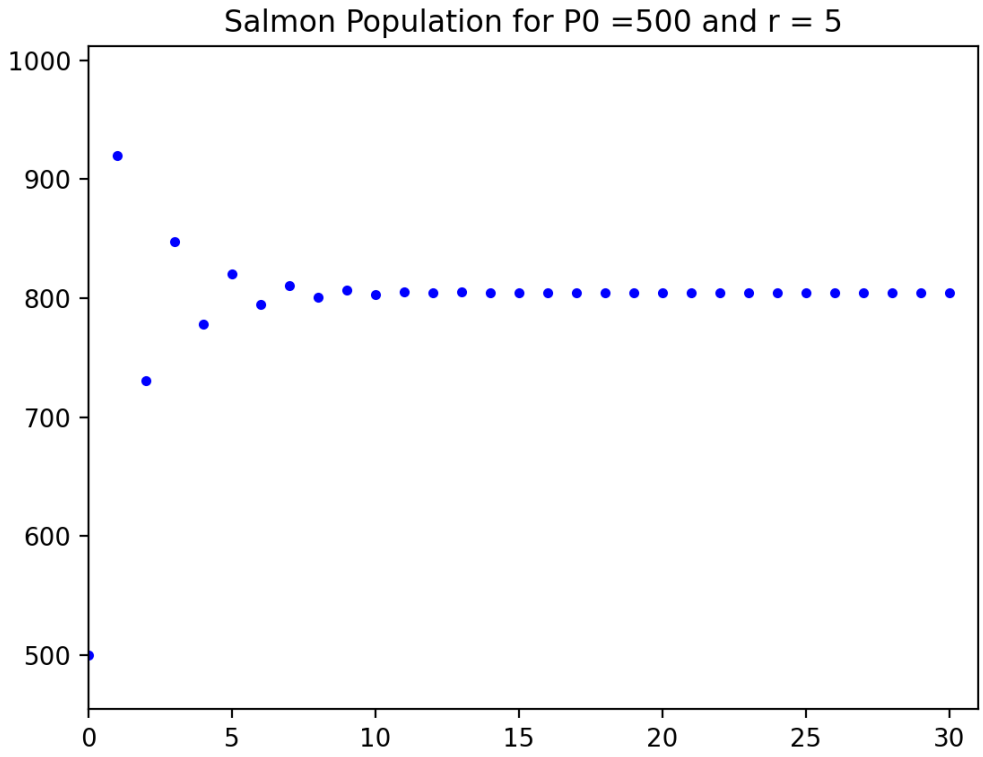
When R_0 = 2, the population converges to $P^* = K \ln R_0 = (500)(\ln 2) = 346.57$

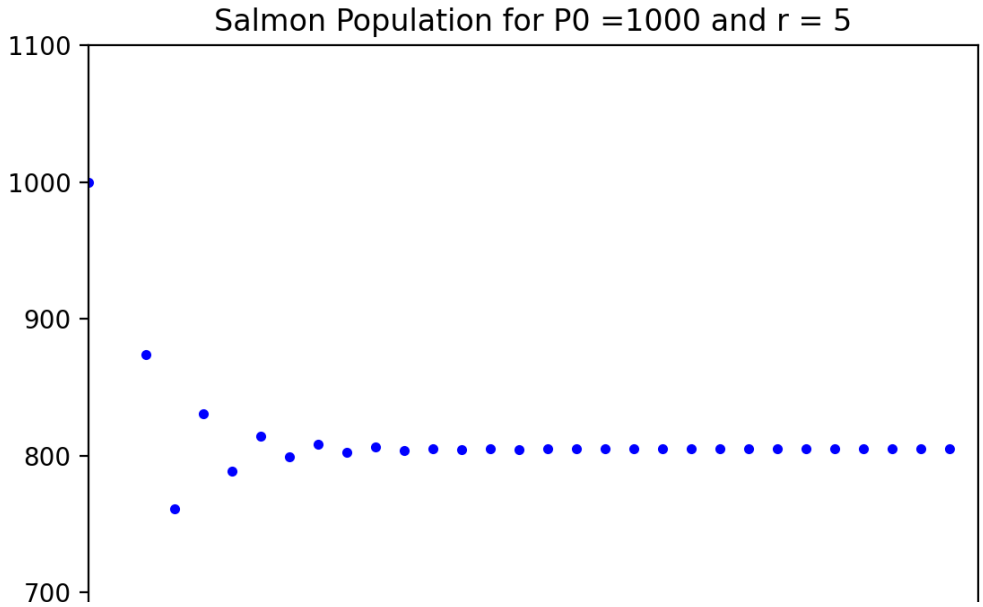
In [108…

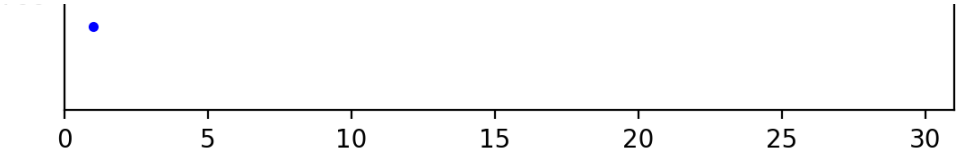
```
#r = 5
populations = [100, 500, 1000]
for i in range(3):
    # initial values
    P_0=populations[i]
    K=500
    r=5

    nit=30 # nit is no of time steps

    t=np.arange(0,nit+1,1) #  time steps

    P=[0]*(nit+1) # list for P initialized with nit zeros

    P[0]=P_0 # given initial population
    for n in range(1,nit+1):
        P[n]=r*P[n-1]*np.exp(-P[n-1]/K)

    # plotting
    plt.figure()
    plt.plot(t,P,'b.')
    Pmax=max(P)
    Pmin=min(P)
    plt.axis([0,nit+1,Pmin/1.1,Pmax*1.1])
    plt.title('Salmon Population for P0 =' +str(P_0)+ ' and r = ' +str(r))
    plt.show()
```




400
200
0
5
10
15
20
25
30


Salmon Population for P0 =500 and r = 5
1000
900
800
700
600
500
0
5
10
15
20
25
30


Salmon Population for P0 =1000 and r = 5
1100
1000
900
800
700

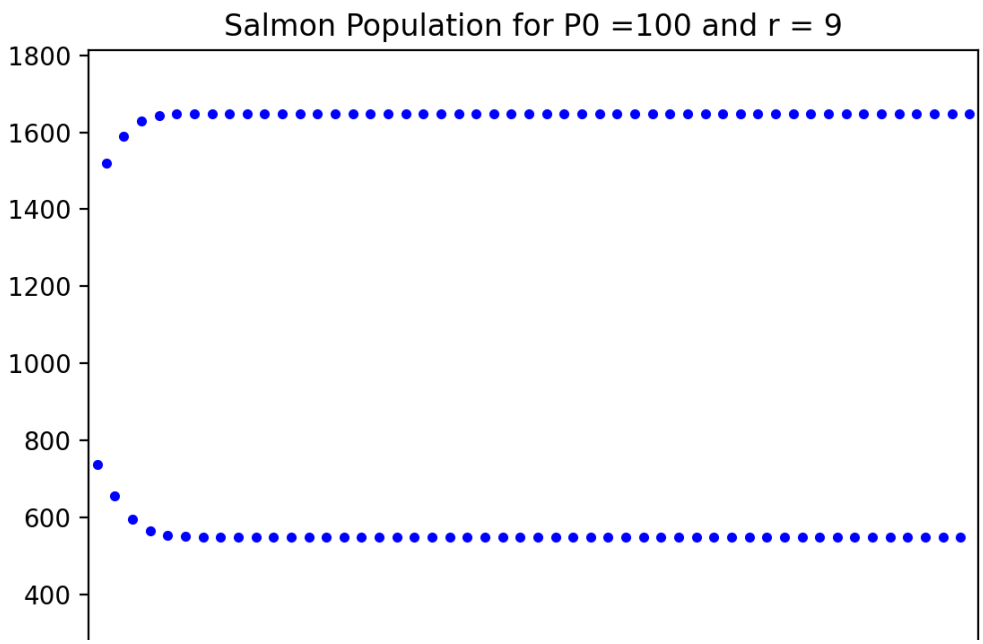


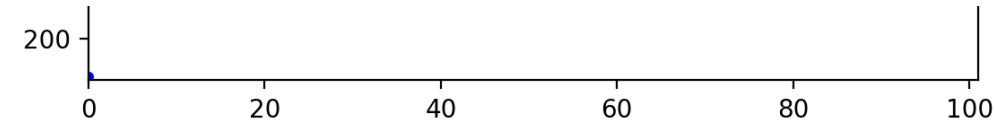
When R_0 = 5, the population converges to $P^* = K \ln R_0 = (500)(\ln 5) = 804.72$

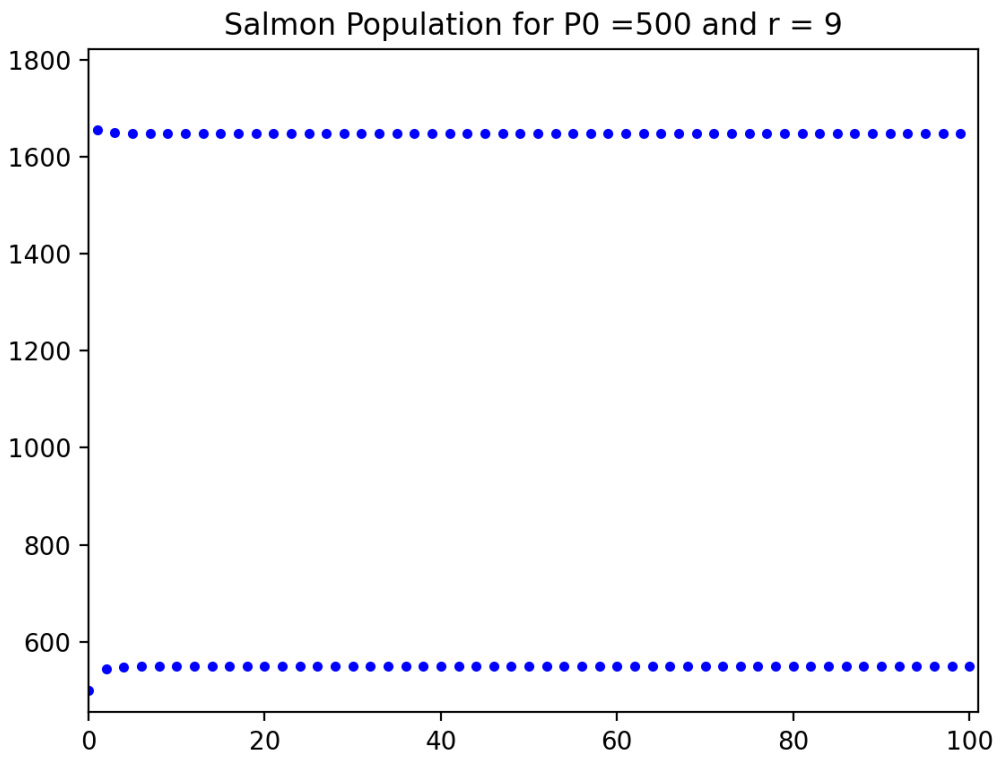
(c). $r > e^2$, for initial population 100, 500 and 1000.

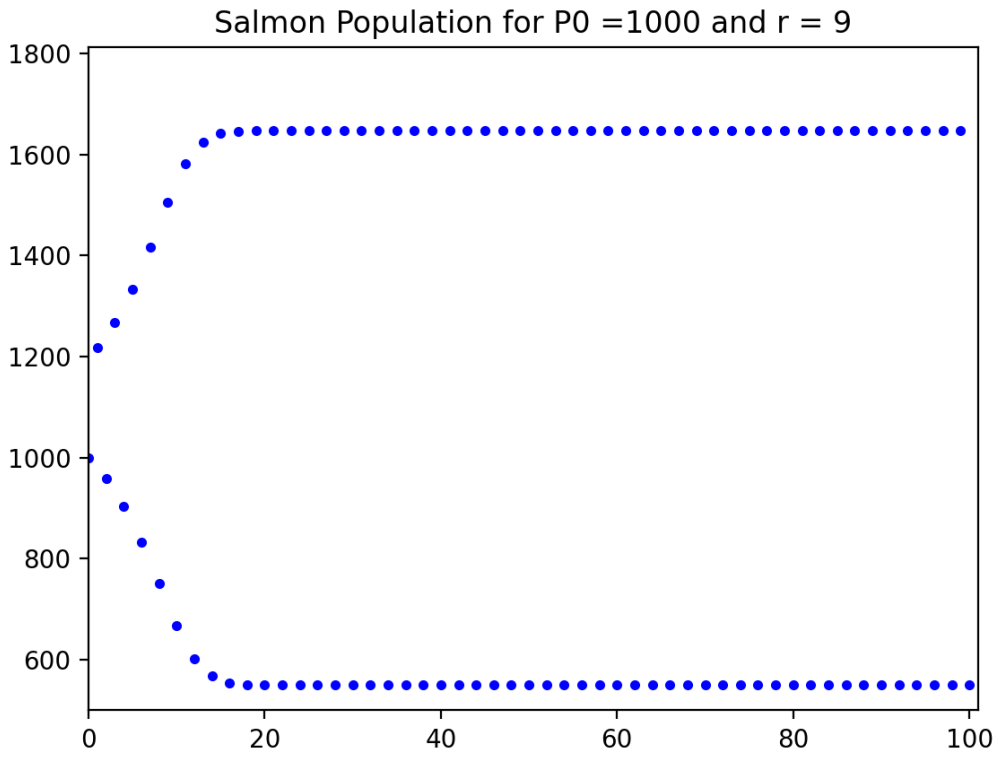
In [112…

```
#r = 9
populations = [100, 500, 1000]
for i in range(3):
    # initial values
    P_0=populations[i]
    K=500
    r=9


    nit=100 # nit is no of time steps

    t=np.arange(0,nit+1,1) #  time steps

    P=[0]*(nit+1) # list for P initialized with nit zeros

    P[0]=P_0 # given initial population
    for n in range(1,nit+1):
        P[n]=r*P[n-1]*np.exp(-P[n-1]/K)


    # plotting
    plt.figure()
    plt.plot(t,P,'b.')
    Pmax=max(P)
    Pmin=min(P)
    plt.axis([0,nit+1,Pmin/1.1,Pmax*1.1])
    plt.title('Salmon Population for P0 =' +str(P_0)+ ' and r = ' +str(r))
    plt.show()
```







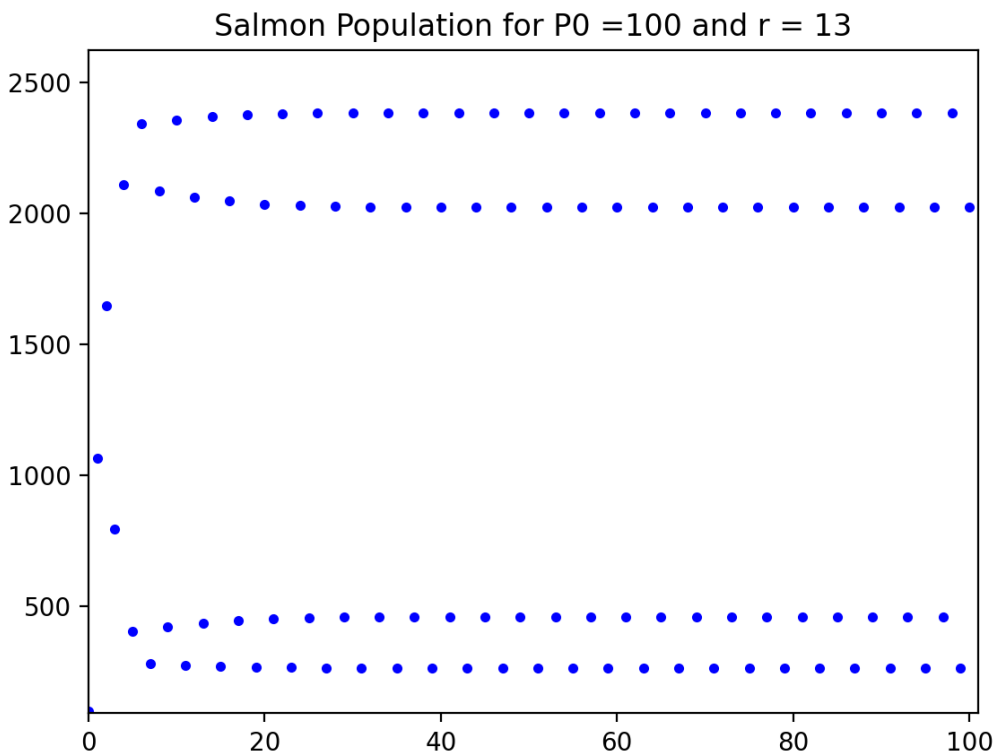


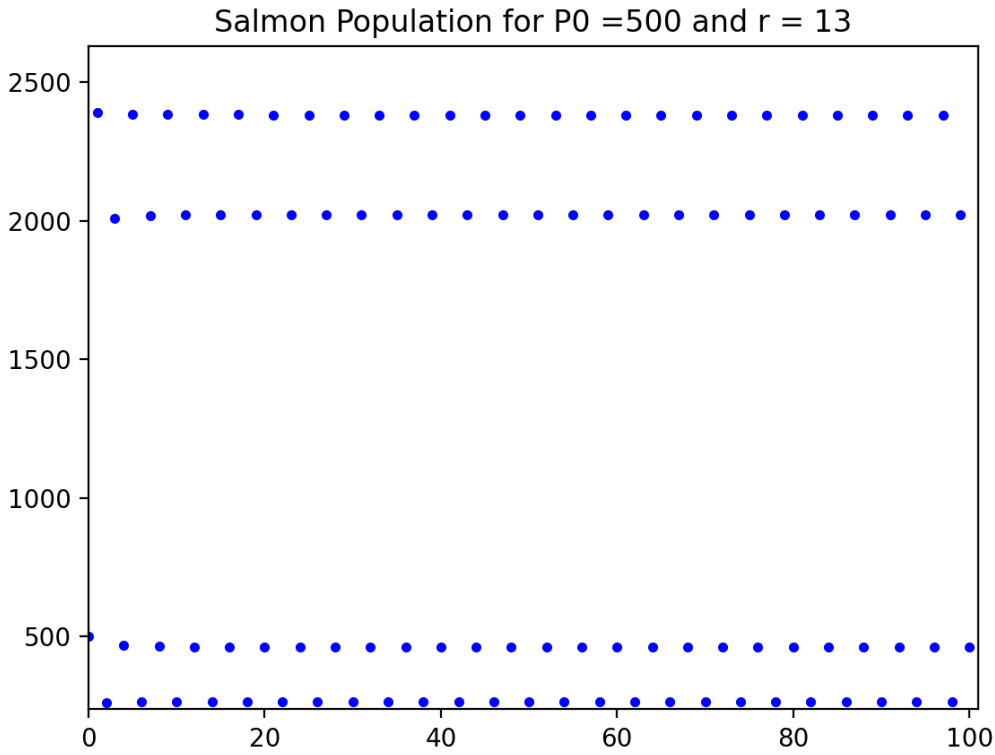
When R_0 = 9 we see that period doubling occurs with $1647.92 \approx F(549.31)$ and

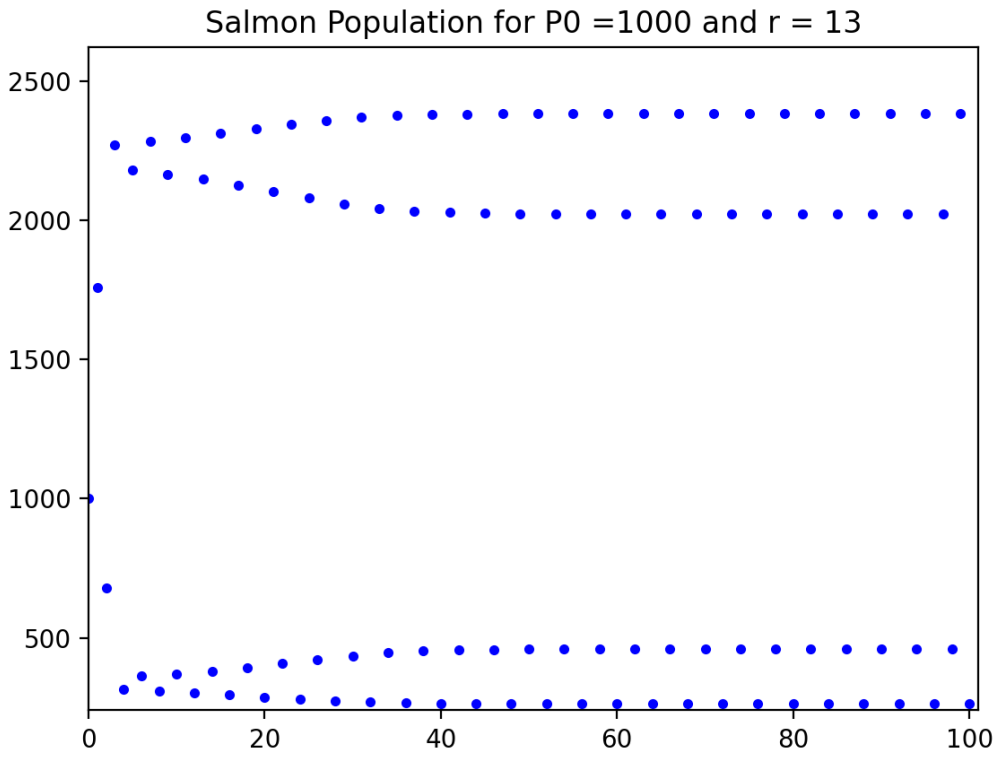
$549.31 \approx F(1647.92)$ where the population stabilises at either 1648 or 549 depending on the previous year.

In [116…

```
#r = 13
populations = [100, 500, 1000]
for i in range(3):
    # initial values
    P_0=populations[i]
    K=500
    r=13


    nit=100 # nit is no of time steps

    t=np.arange(0,nit+1,1) #  time steps

    P=[0]*(nit+1) # list for P initialized with nit zeros

    P[0]=P_0 # given initial population
    for n in range(1,nit+1):
        P[n]=r*P[n-1]*np.exp(-P[n-1]/K)


    # plotting
    plt.figure()
    plt.plot(t,P,'b.')
    Pmax=max(P)
    Pmin=min(P)
    plt.axis([0,nit+1,Pmin/1.1,Pmax*1.1])
    plt.title('Salmon Population for P0 =' +str(P_0)+ ' and r = ' +str(r))
    plt.show()
```



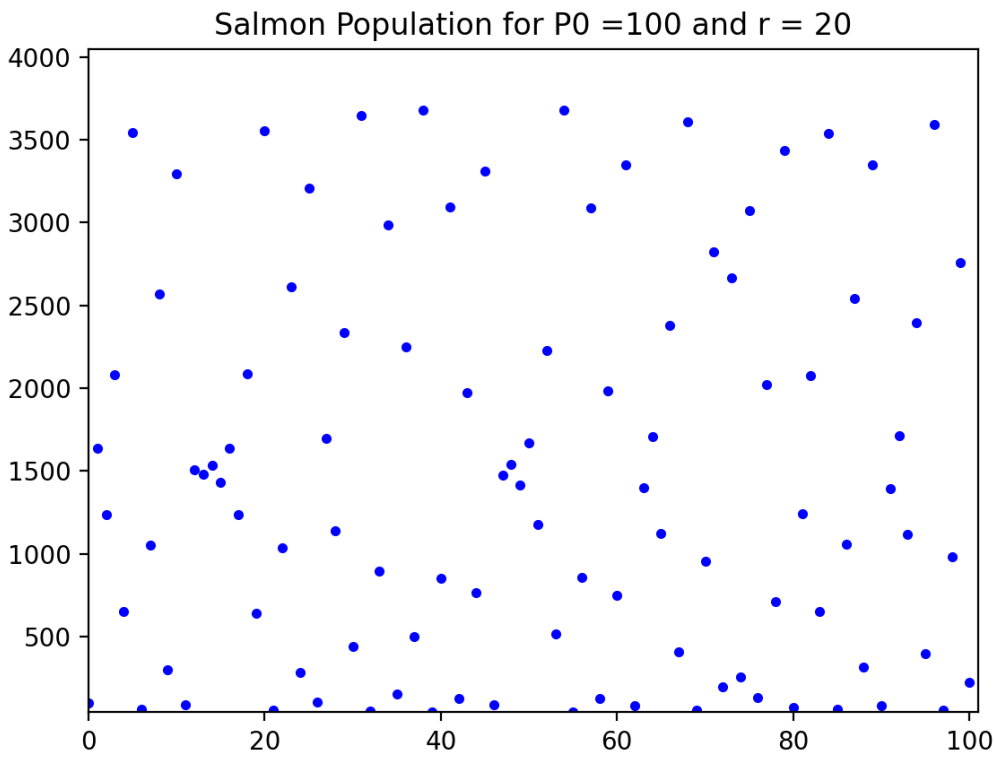


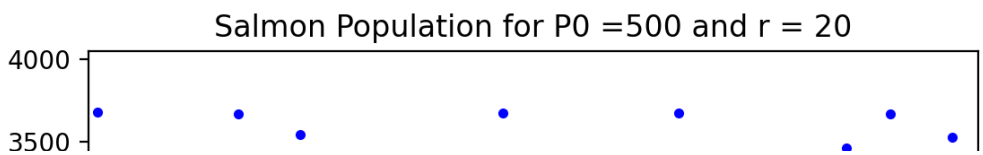


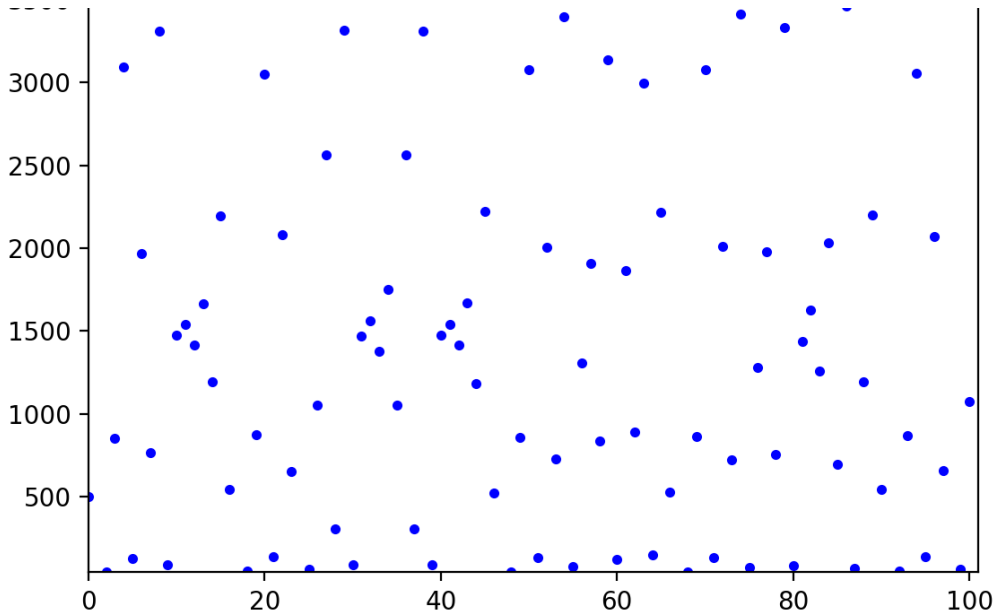
When R_0 = 13 we see further period doubling occuring.

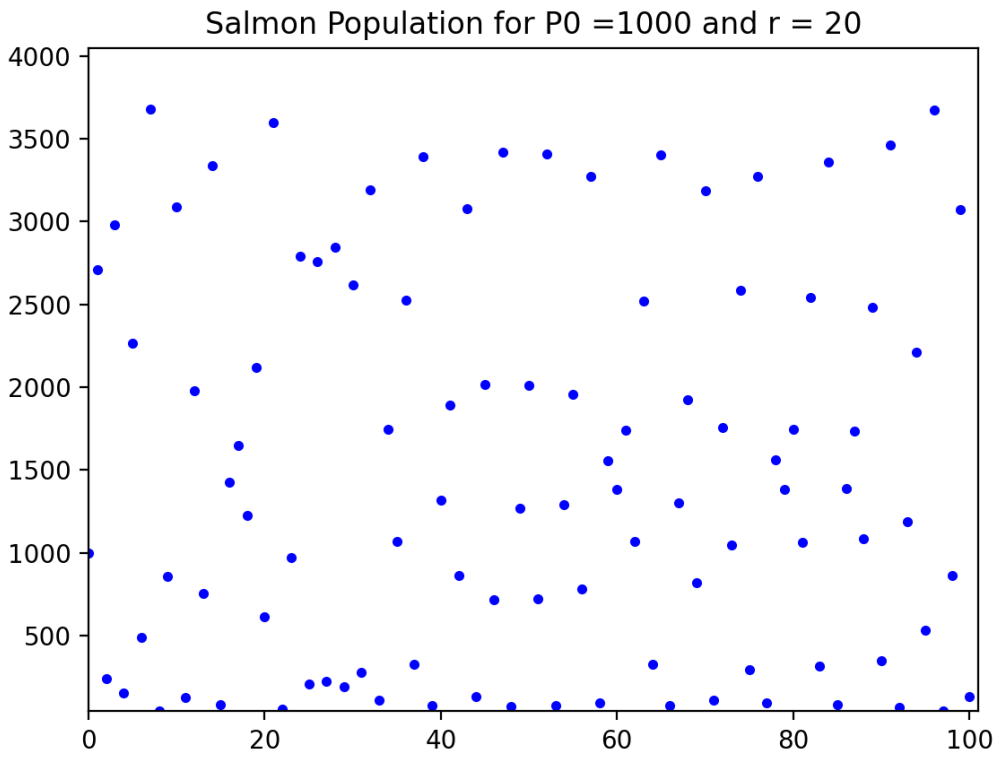
In [102…

```
#r = 20
populations = [100, 500, 1000]
for i in range(3):
    # initial values
    P_0=populations[i]
    K=500
    r=20

    nit=100 # nit is no of time steps

    t=np.arange(0,nit+1,1) #  time steps

    P=[0]*(nit+1) # list for P initialized with nit zeros

    P[0]=P_0 # given initial population
    for n in range(1,nit+1):
        P[n]=r*P[n-1]*np.exp(-P[n-1]/K)

    # plotting
    plt.figure()
    plt.plot(t,P,'b.')
    Pmax=max(P)
    Pmin=min(P)
    plt.axis([0,nit+1,Pmin/1.1,Pmax*1.1])
    plt.title('Salmon Population for P0 =' +str(P_0)+ ' and r = ' +str(r))
    plt.show()
```









When R_0 = 20 we see unpredictable chaotic behaviour.

MP307 Practical 2 Queueing Theory II

Name: Cian Mac Liatháin

ID: 11476078

Python functions

The Python functions contained in this notebook are:

1. `Nearneigh(pup,pdown,r)` . This generates the transition matrix `P` for a simple nearest neighbour model with maximum size `r` and probability `pup` of one step up transition and `pdown` for one step down transition. The output is the transition matrix.

2. `Equilibrium(P)` . This computes the equilibrium probabilities for a given transition matrix `P` . If the system is not ergodic then an error message appears. The output is an array.

3. `Queue(P,n0,nit)` . This simulates a queue for a given transition matrix `P` with initial queue size `n0` for `nit` iterations. The output is a list of simulated queue data.

4. `Qplot(qdata,pi)` . This generates an animation of the queue for any input data list `qdata` . The second argument is optional and consists of the equilibrium probabilities `pi` (if they exist and have been calculated via `Equilibrium(P)` above or otherwise). The output is a Python animation where 50 plots are shown for the normalised frequency of events which can be compared to a plot of the equilibrium probabilities `pi` if provided.

In [60]:

```
%matplotlib notebook
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as patches
import matplotlib.path as path
import matplotlib.animation as animation
from matplotlib.patches import Rectangle
import random as ran
```

In [61]:

```
def Nearneigh (pup, pdown, r):
    # This generates the transition matrix P for a nearest neighbour model
    # maximum size r and probability pup of one step up transition and pdow
    # for one step down transition.
    # The output is the transition matrix P.
    P = np.zeros((r + 1, r + 1))
    P[0, 0] = 1 - pup
    P[0, 1] = pup
    P[r , r ] = 1 - pdown
    P[r , r - 1] = pdown
    for i in range(1, r):
        P[i, i - 1] = pdown
        P[i , i] = 1 - pdown - pup
        P[i , i + 1] = pup
    return(P)
```

Example.

Find the transition matrix for a queue of maximum size 5 with prob customer arrival=0.2 and prob customer servicing=0.3

In [62]:

```
P=Nearneigh(0.2,0.3,5)
print(P)
```

```
[[0.8 0.2 0.  0.  0.  0. ]
 [0.3 0.5 0.2 0.  0.  0. ]
 [0.  0.3 0.5 0.2 0.  0. ]
 [0.  0.  0.3 0.5 0.2 0. ]
 [0.  0.  0.  0.3 0.5 0.2]
 [0.  0.  0.  0.  0.3 0.7]]
```

In [63]:

```
def Equilibrium (P):
    # This computes the equilibrium probabilities pi for a given transition
    # If the system is not ergodic then an error message appears.
    # The output is an array.
    degenflag = 0
    n = len(P)
    pi = np.zeros((n))
    for i in range(n):
        if float(sum(P[i,:]))!= 1.0:
            print("ERROR: row",i,"of P does not sum to 1 !")
    eigenvals, eigenvects=np.linalg.eig(P.transpose()) # eigendata for P tr
    for k in range(n):
        if abs(eigenvals[k]-1)< 1.0e-14:
            degenflag = degenflag + 1
            k1=k # label for eigenvalue 1.
    if 1 < degenflag:
        print("ERROR: P is not Ergodic")
    else:
        v=eigenvects[:,k1]
        pi=v/sum(v)
    return(pi.real)
```

In [64]:

```
pi=Equilibrium(P)
print(pi)
```

```
[0.36541353 0.24360902 0.16240602 0.10827068 0.07218045 0.0481203 ]
```

In [65]:

```
def Queue (P, n0, nit):
# This simulates a queue for a given transition matrix P with initial queue
# The output is a list of simulated queue data.
    nP = len(P)
# pp is a relabelled transition matrix with row elements in decreasing orde
# This is used to speed up the simulation process
# The relabelling is stored in the matrix loc.
    pp = np.zeros((nP,nP))
    for i in range(nP):
        for j in range(nP):
            pp[i, j] = P[i, j]
    loc = np.array([list(range(nP))]*nP)
    for i in range(nP):
        for j in range(nP-1):
            maxpp = pp[i, j]
            kmax = j
            for k in range(j, nP):
                if maxpp < pp[i, k]:
                    maxpp = pp[i, k]
                    kmax = k
                    pp[i, j], pp[i, k] = pp[i, k], pp[i, j]
                    loc[i, j], loc[i, k] = loc[i, k], loc[i, j]
    n = n0
    qseq = [n0]
    if n0 < 0 or nP-1 < n0:
        print("ERROR: inputted initial Q size : Qsize<0 or Qsize>",nP-1)
    for m in range(nit):
        x = ran.random() # generates uniform random no on [0,1]
        k = 0
        x1 = pp[n, k]
        while (x1 < x):
            k = k + 1
            x1 = x1 + pp[n, k]
        n = loc[n, k]
        qseq = qseq + [n]
    return(qseq)
```

In [66]:

```
print(P) # For the transition matrix in our example above
```

```
[[0.8 0.2 0.  0.  0.  0. ]
 [0.3 0.5 0.2 0.  0.  0. ]
 [0.  0.3 0.5 0.2 0.  0. ]
 [0.  0.  0.3 0.5 0.2 0. ]
 [0.  0.  0.  0.3 0.5 0.2]
 [0.  0.  0.  0.  0.3 0.7]]
```

Simulate 1000 steps with intial state 2 for given P

In [67]:

```
qdata=Queue(P,2,1000)
print(qdata)
```

```
[2, 2, 2, 1, 1, 0, 0, 0, 0, 0, 1, 1, 2, 2, 3, 3, 3, 2, 3, 4, 5, 5, 5, 4, 4,
4, 3, 4, 4, 3, 3, 2, 2, 2, 2, 2, 1, 1, 2, 2, 2, 3, 2, 2, 1, 2, 2, 2, 3, 3,
4, 3, 3, 2, 3, 2, 2, 3, 3, 2, 2, 2, 2, 3, 4, 3, 4, 4, 4, 5, 5, 5, 5, 5, 5,
5, 5, 5, 4, 4, 3, 3, 3, 3, 3, 3, 2, 1, 1, 2, 3, 3, 4, 3, 3, 3, 3, 4, 3, 2,
2, 3, 3, 3, 2, 3, 2, 2, 3, 2, 2, 2, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1,
0, 1, 2, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1,
0, 1, 0, 0, 0, 0, 1, 0, 1, 1, 1, 1, 2, 3, 2, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0,
0, 0, 0, 0, 0, 1, 2, 3, 3, 3, 3, 3, 2, 3, 4, 3, 4, 4, 4, 3, 3, 2, 3, 3, 3,
3, 4, 5, 4, 3, 2, 2, 2, 2, 2, 2, 2, 1, 1, 2, 2, 3, 2, 2, 2, 2, 2, 2, 2, 1,
```

```
1, 1, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1, 2, 3, 4, 3, 2,
2, 3, 3, 3, 2, 2, 2, 2, 2, 2, 1, 0, 0, 1, 2, 3, 2, 2, 2, 2, 1, 1, 1, 1, 1,
2, 1, 1, 1, 2, 2, 2, 1, 2, 1, 1, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 1, 2, 2,
2, 3, 4, 4, 3, 2, 1, 1, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 2, 2, 3, 2, 1, 1, 0, 0, 1, 1, 1, 1, 0, 0, 0,
0, 0, 0, 1, 1, 2, 1, 2, 2, 2, 2, 1, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 2, 3, 3,
3, 3, 3, 3, 2, 2, 2, 2, 3, 3, 3, 2, 2, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0,
0, 0, 1, 1, 2, 2, 1, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0,
0, 0, 0, 1, 1, 2, 2, 3, 4, 4, 3, 4, 4, 4, 4, 3, 3, 2, 2, 1, 2, 2, 2, 3, 2,
1, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 2, 2, 1, 1, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 2, 2, 1, 0,
0, 1, 1, 0, 1, 1, 1, 0, 0, 0, 1, 2, 2, 1, 0, 0, 0, 1, 0, 0, 0, 1, 2, 2, 3,
2, 2, 2, 2, 2, 2, 3, 3, 4, 3, 3, 4, 4, 3, 3, 3, 2, 2, 2, 2, 2, 2, 1, 1, 0,
1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
1, 2, 2, 2, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 1, 1,
2, 2, 2, 1, 1, 1, 1, 2, 2, 2, 2, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 2, 2, 1, 2,
2, 2, 2, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 1,
2, 2, 1, 0, 0, 0, 0, 1, 2, 2, 1, 0, 1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 2, 2, 2,
2, 2, 3, 3, 2, 1, 1, 0, 0, 0, 0, 0, 1, 2, 2, 3, 3, 3, 3, 3, 3, 2, 3, 3, 2,
2, 2, 1, 1, 2, 2, 2, 3, 3, 3, 2, 2, 2, 2, 2, 2, 2, 3, 4, 5, 4, 5, 5, 5, 4,
4, 3, 3, 3, 3, 3, 2, 2, 2, 1, 0, 1, 0, 1, 1, 2, 1, 0, 0, 0, 0, 0, 0, 0, 0,
1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 2, 2, 3, 3, 3, 2, 2, 3,
3, 4, 4, 3, 4, 4, 4, 3, 4, 5, 4, 5, 4, 3, 4, 3, 2, 2, 1, 1, 2, 2, 1, 1, 2,
2, 1, 1, 1, 2, 3, 3, 2, 2, 3, 3, 3, 3, 3, 2, 2, 2, 1, 2, 1, 1, 1, 2, 2, 2,
2, 3, 2, 2, 2, 2, 1, 1, 2, 3, 3, 2, 1, 1, 1, 2, 2, 1, 1, 0, 0, 0, 0, 1, 1,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1,
1, 0, 0, 0, 1, 2, 2, 3, 3, 3, 3, 2, 3, 2, 2, 3, 2, 3, 2, 1, 2, 2, 1, 0, 1,
1]
```

In [68]:

```
len(qdata) # notice there are 1001 data pts for 1000 steps
```

Out[68]:

```
1001
```

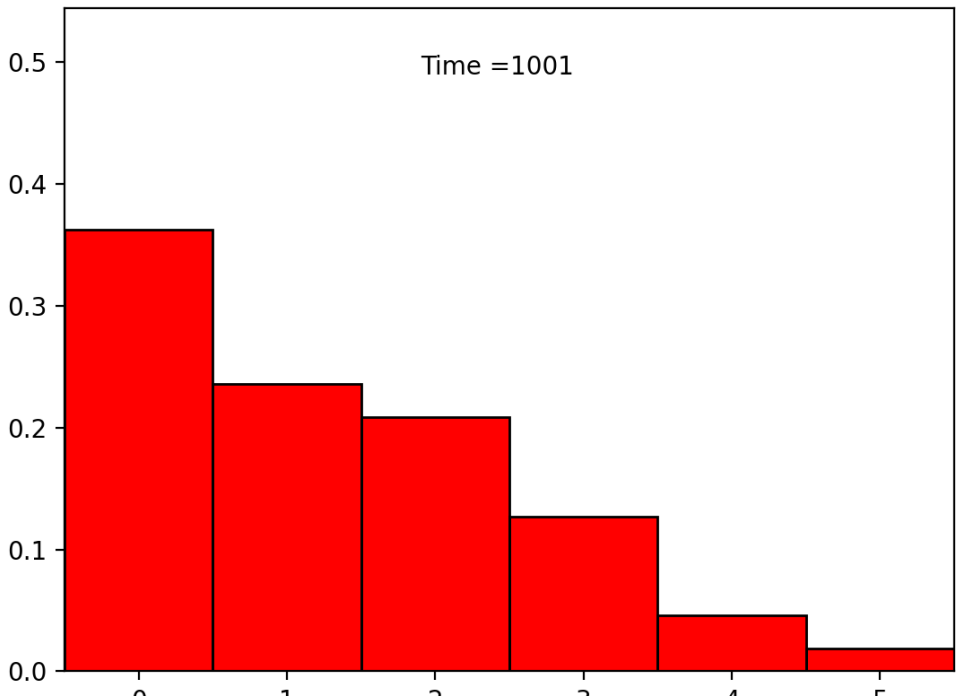
In [69]:

```
def Qplot(qdata,pi=0):
    # This generates up to 50 animations of the queue for any input data li
    # The second argument is optional and consists of the equilibrium proba
    # (if they exist and have been calculated via Equilibrium(P) above).
    # The output is an animation of 50 plots of the normalised frequency of
    # This is compared to the equilibrium probabilities pi if provided.

    # Nsim animations
    Nsim=50

    Npoints=len(qdata)
    qmax=max(qdata)

    shift = lambda x: x-1/2
    inbins=list(map(shift,list(range(qmax+2)))) # bins -1/2,1/2,3/2,...
    n,bins  = np.histogram(qdata, bins=inbins)

    freq=np.array(n)/Npoints

    # get the corners of the rectangles for the histogram

    left = bins[:-1]
    right = bins[1:]
    bottom = np.zeros(len(left))
    top = bottom + freq
    nrects = len(left)

    # Here comes the tricky part -- we have to set up the vertex and path c
    # arrays using `.Path.MOVETO`, `.Path.LINETO` and `.Path.CLOSEPOLY` for

    #* We need 1 ``MOVETO`` per rectangle, which sets the initial point.
    #* We need 3 ``LINETO``'s, which tell Matplotlib to draw lines from
    #  vertex 1 to vertex 2, v2 to v3, and v3 to v4.
    #* We then need one ``CLOSEPOLY`` which tells Matplotlib to draw a line
    #  the v4 to our initial vertex (the ``MOVETO`` vertex), in order to cl
    #  polygon.

    #<div class="alert alert-info"><h4>Note</h4><p>The vertex for ``CLOSEPO
    #  in the ``verts`` array to keep the codes aligned with the vertices.<

    nverts = nrects * (1 + 3 + 1)
    verts = np.zeros((nverts, 2))
    codes = np.full(nverts, path.Path.LINETO)
    codes[0::5] = path.Path.MOVETO
    codes[4::5] = path.Path.CLOSEPOLY
    verts[0::5, 0] = left
    verts[0::5, 1] = bottom
    verts[1::5, 0] = left
    verts[1::5, 1] = top
    verts[2::5, 0] = right
    verts[2::5, 1] = top
    verts[3::5, 0] = right
    verts[3::5, 1] = bottom

    #To animate the histogram, we use the ``qanimate`` function, which upda
    #histogram (in this case, only the heights of each rectangle).

    def qanimate(i):
        # simulate data
        Ni_points=int(Npoints*i/Nsim)
        datai=qdata[1:Ni_points] # first Ni_points
        ni, bin = np.histogram(datai, bins=inbins)
```

```
    patch = None

    fig, ax = plt.subplots()
    barpath = path.Path(verts, codes)
    patch = patches.PathPatch(
        barpath, facecolor='red', edgecolor='black', alpha=1)
    ax.add_patch(patch)

    time_text = ax.text(0.4, 0.9, '', transform=ax.transAxes)

    # draw rectangle for equilibrium probs pi
    if not type(pi) is int:
        for m in range(len(pi)):
            heightpi=pi[m]
            widthpi=1
            rectpi = Rectangle((m-1/2, 0), widthpi, heightpi, fc='white', e
            ax.add_patch(rectpi)

    ax.set_xlim(-1/2, qmax+1/2)
    ymax=max(freq)*1.5
    ax.set_ylim(0, ymax)

    ani = animation.FuncAnimation(fig, qanimate, Nsim+1,repeat=False, blit=
    plt.show()
    return ani
```

In [70]:

```
Qplot(qdata) # animation of given qdata without pi given
```



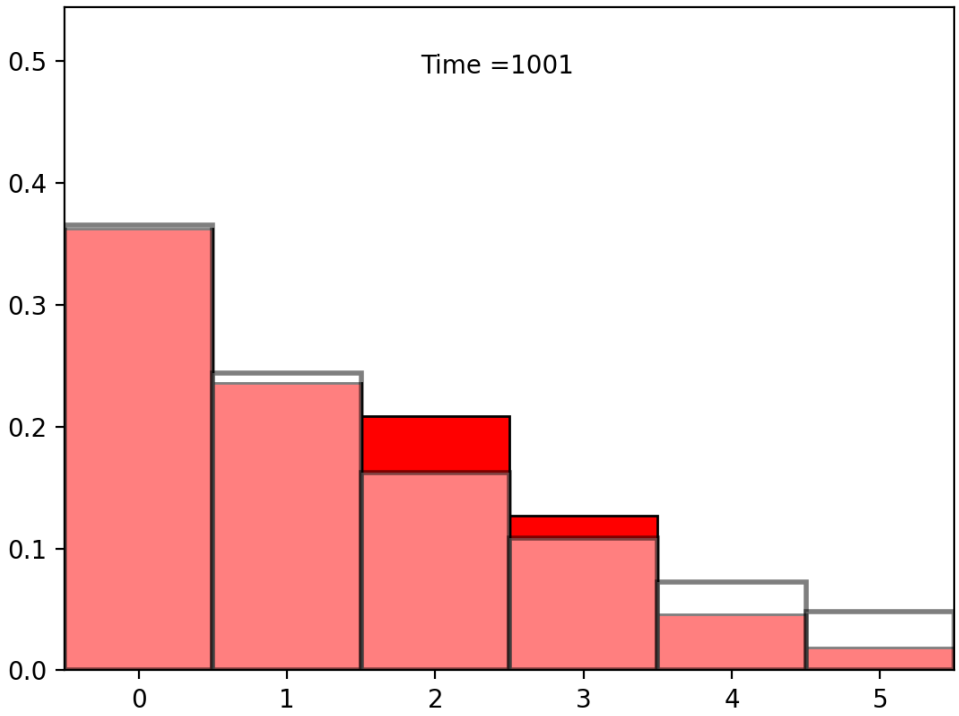
```
0         1         2         3         4         5
```

Out[70]: <matplotlib.animation.FuncAnimation at 0x7fd540537d90>

In [71]:
```
print(pi)
```

```
[0.36541353 0.24360902 0.16240602 0.10827068 0.07218045 0.0481203 ]
```

In [72]:
```
Qplot(qdata,pi) # animation of given qdata with pi given
```



Out[72]: <matplotlib.animation.FuncAnimation at 0x7fd5408b3c70>

Q.1

Simulate the Markov process with transition matrix:

$$\begin{bmatrix} 1/2 & 0 & 1/2 & 0 & 0 & 0 \\ 0 & 1/2 & 0 & 1/2 & 0 & 0 \\ 0 & 0 & 1/2 & 0 & 1/2 & 0 \\ 0 & 1/4 & 0 & 1/2 & 0 & 1/4 \\ 1/4 & 0 & 1/4 & 0 & 1/2 & 0 \\ 0 & 0 & 0 & 1/2 & 0 & 1/2 \end{bmatrix}$$

This is the same example as in Q.3 of Practical 1. Observe the long-time behaviour for initial queue size 0 and initial queue size 1.

Observe what happens if the first row of the transition matrix is changed to

```
[1/4, 1/4, 1/2, 0, 0, 0]
```

In [73]:

```
P=np.array([
    [1/2,0,1/2,0,0,0],
    [0,1/2,0,1/2,0,0],
    [0,0,1/2,0,1/2,0],
    [0,1/4,0,1/2,0,1/4],
    [1/4,0,1/4,0,1/2,0],
    [0,0,0,1/2,0,1/2]
    ])
print(P)
```

```
[[0.5  0.   0.5  0.   0.   0.  ]
 [0.   0.5  0.   0.5  0.   0.  ]
 [0.   0.   0.5  0.   0.5  0.  ]
 [0.   0.25 0.   0.5  0.   0.25]
 [0.25 0.   0.25 0.   0.5  0.  ]
 [0.   0.   0.   0.5  0.   0.5 ]]
```

In [74]:

```
Equilibrium(P)
```

```
ERROR: P is not Ergodic
```

Out[74]:

```
array([0., 0., 0., 0., 0., 0.])
```

In [75]:

```
qdata=Queue(P,0,1000)  # 1000 iterations with initial state 0
```

In [76]:

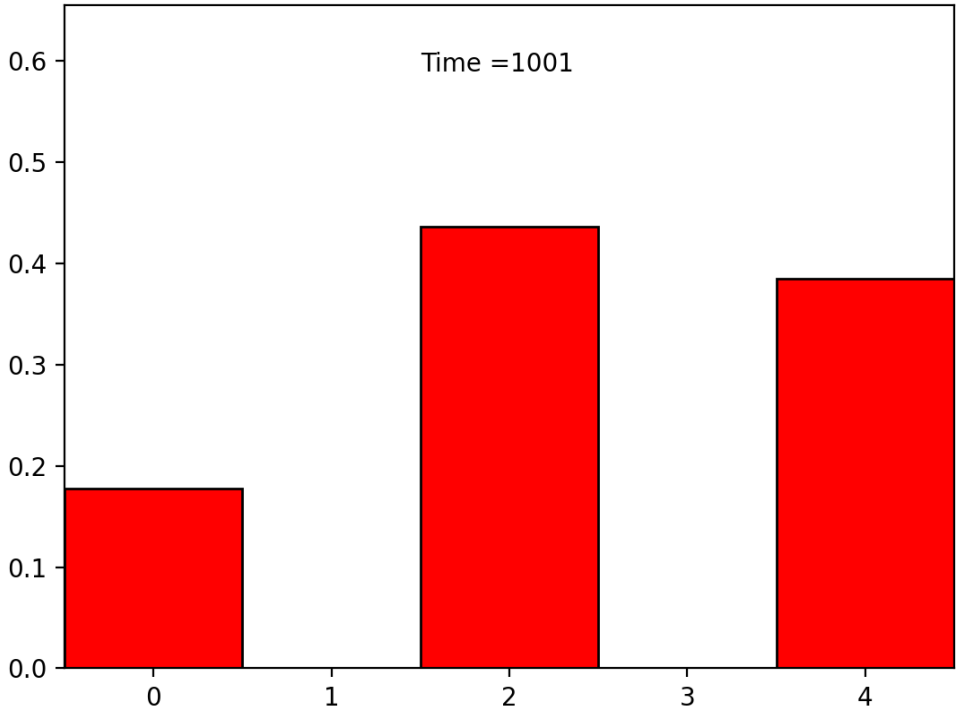
```
print(qdata)
```

```
[0, 2, 4, 2, 2, 4, 2, 4, 2, 2, 2, 2, 2, 4, 0, 2, 2, 2, 2, 4, 4, 2, 2, 2, 4,
2, 4, 0, 0, 2, 4, 4, 2, 2, 4, 4, 0, 0, 0, 0, 2, 4, 4, 4, 4, 4, 2, 2, 4, 4,
2, 2, 2, 4, 2, 2, 2, 2, 4, 4, 2, 2, 2, 4, 4, 0, 0, 2, 4, 2, 2, 4, 2, 4, 4,
2, 2, 4, 4, 4, 4, 2, 2, 2, 4, 4, 4, 0, 0, 0, 2, 2, 4, 4, 2, 4, 4, 2, 2, 4,
2, 2, 2, 4, 2, 2, 2, 2, 2, 2, 4, 4, 2, 4, 4, 4, 4, 4, 2, 2, 2, 2, 2, 4, 2,
4, 0, 2, 2, 4, 2, 2, 4, 2, 2, 2, 2, 2, 2, 4, 0, 0, 2, 2, 4, 2, 2, 4, 2, 2,
4, 0, 2, 4, 4, 4, 0, 0, 0, 2, 2, 2, 2, 4, 4, 4, 2, 4, 4, 2, 2, 4, 0, 0, 0,
2, 4, 4, 4, 0, 2, 2, 2, 4, 4, 4, 2, 2, 4, 4, 2, 4, 4, 4, 4, 4, 0, 2, 2, 2,
4, 2, 2, 4, 4, 0, 2, 4, 0, 0, 2, 4, 4, 0, 0, 0, 2, 4, 0, 0, 2, 2, 2, 2, 4,
2, 2, 4, 4, 4, 4, 2, 2, 4, 4, 2, 4, 4, 4, 0, 0, 2, 4, 2, 2, 4, 0, 0, 2, 4,
2, 4, 4, 0, 0, 0, 2, 4, 4, 0, 0, 0, 0, 2, 2, 4, 4, 4, 0, 2, 4, 4, 0, 2, 2,
2, 2, 2, 2, 2, 4, 4, 0, 2, 2, 2, 2, 2, 4, 0, 2, 2, 2, 4, 4, 4, 2, 4, 4, 4,
4, 4, 0, 2, 2, 2, 2, 2, 2, 4, 4, 2, 2, 4, 0, 0, 2, 2, 2, 2, 2, 2, 4, 4, 4,
4, 0, 0, 2, 2, 2, 4, 4, 4, 0, 0, 0, 2, 2, 2, 4, 4, 4, 2, 4, 4, 0, 0, 2, 4,
4, 4, 4, 2, 4, 2, 4, 0, 0, 0, 0, 2, 2, 2, 2, 2, 2, 2, 2, 4, 0, 0, 0, 2, 4,
2, 2, 2, 4, 2, 4, 0, 0, 0, 0, 0, 2, 2, 2, 2, 2, 2, 2, 2, 4, 0, 0, 2, 4, 2,
4, 0, 2, 4, 4, 4, 4, 4, 4, 4, 4, 2, 2, 2, 4, 4, 0, 2, 4, 4, 2, 4, 2, 2, 4,
0, 2, 2, 4, 0, 2, 4, 0, 2, 2, 4, 4, 0, 0, 2, 4, 4, 2, 4, 4, 4, 4, 4, 4, 0,
0, 2, 4, 4, 4, 4, 2, 2, 2, 2, 2, 2, 2, 4, 2, 2, 4, 0, 0, 2, 4, 4, 0, 2, 2,
2, 2, 2, 4, 2, 4, 2, 4, 4, 4, 0, 2, 4, 4, 0, 2, 4, 2, 4, 4, 0, 0, 0, 2, 2,
4, 0, 0, 0, 2, 4, 0, 0, 0, 2, 2, 2, 2, 4, 4, 0, 2, 2, 4, 0, 0, 2, 2, 4, 2,
2, 2, 2, 4, 2, 2, 2, 2, 2, 4, 4, 0, 2, 4, 4, 4, 0, 2, 4, 0, 0, 2, 4, 0, 2,
2, 2, 2, 4, 2, 2, 2, 2, 4, 4, 0, 0, 2, 4, 4, 4, 4, 4, 0, 2, 2, 2, 4, 2, 2,
2, 4, 4, 4, 4, 0, 2, 2, 4, 4, 0, 2, 2, 4, 4, 0, 2, 4, 2, 2, 4, 4, 4, 4, 4,
0, 2, 2, 2, 2, 4, 4, 4, 0, 0, 2, 4, 2, 2, 4, 0, 2, 4, 0, 2, 2, 4, 2, 2, 4,
4, 2, 4, 4, 0, 2, 2, 2, 2, 2, 4, 4, 2, 2, 2, 4, 4, 0, 2, 4, 4, 0, 0, 2, 4,
2, 2, 4, 0, 0, 2, 4, 2, 4, 0, 2, 2, 4, 2, 4, 4, 4, 0, 2, 2, 2, 4, 4, 0, 2,
4, 2, 4, 2, 2, 4, 4, 4, 4, 0, 2, 2, 4, 4, 4, 4, 4, 2, 4, 2, 2, 2, 2, 4, 4,
2, 2, 4, 2, 2, 2, 2, 4, 2, 4, 2, 4, 2, 4, 4, 4, 4, 4, 2, 4, 2, 4, 4, 4, 0,
2, 2, 2, 4, 4, 0, 2, 4, 2, 4, 0, 0, 0, 2, 4, 0, 2, 2, 2, 4, 0, 2, 2, 2, 2,
4, 4, 4, 0, 2, 2, 4, 0, 2, 2, 4, 4, 4, 4, 4, 4, 0, 2, 2, 2, 2, 2, 2, 2, 2,
2, 2, 4, 2, 4, 2, 2, 2, 2, 2, 2, 2, 2, 2, 4, 4, 0, 2, 4, 4, 2, 2, 2, 4, 4,
2, 4, 2, 2, 2, 4, 4, 2, 4, 0, 0, 2, 4, 4, 0, 0, 2, 2, 2, 2, 4, 4, 4, 2, 2,
2, 4, 4, 4, 2, 2, 2, 4, 0, 0, 0, 0, 0, 2, 2, 4, 4, 4, 2, 2, 2, 2, 4, 4, 2,
4, 2, 2, 2, 4, 4, 0, 0, 0, 0, 0, 2, 2, 2, 4, 4, 4, 2, 4, 2, 4, 2, 2, 2, 4,
0, 2, 4, 4, 4, 4, 4, 4, 4, 2, 4, 4, 4, 4, 0, 2, 2, 2, 2, 4, 4, 4, 0, 0, 0,
2, 2, 4, 4, 0, 0, 2, 4, 4, 0, 2, 2, 2, 4, 4, 4, 2, 4, 4, 4, 4, 2, 2, 2, 4,
2, 4, 4, 4, 4, 4, 0, 2, 2, 2, 2, 4, 2, 4, 2, 2, 2, 4, 2, 2, 4, 4, 4, 4, 4,
0, 0, 2, 2, 4, 2, 2, 4, 4, 0, 2, 4, 0, 0, 0, 0, 0, 2, 2, 4, 4, 4, 4, 4, 4,
```

```
2, 2, 2, 4, 0, 2, 4, 4, 0, 0, 2, 2, 4, 0, 0, 2, 4, 0, 0, 0, 0, 0, 0, 0, 0,
```

In [77]:

```
Qplot(qdata)
```



Out[77]:

```
<matplotlib.animation.FuncAnimation at 0x7fd542812f40>
```

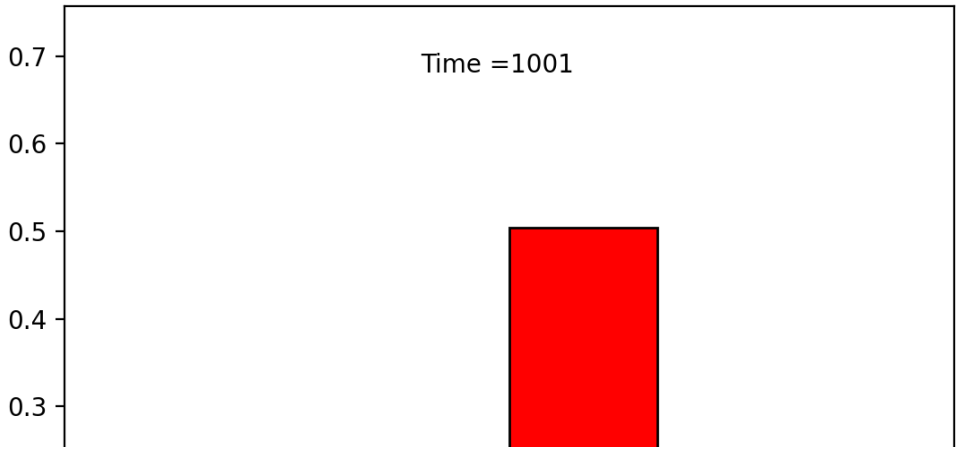
We can see that when the initial queue size is 0, the queue alternates between queue sizes 0, 2 and 4. This shows that the queue is not ergodic, as the sizes of 1, 3 and 5 are never reached.

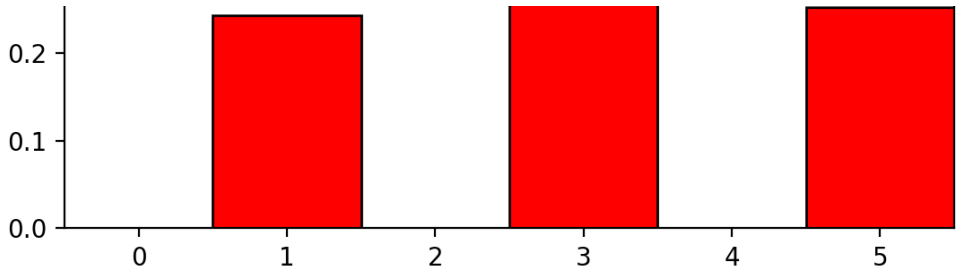
In [78]:

```
qdata=Queue(P,1,1000)  # 1000 iterations with initial state 1
```

In [79]:

```
Qplot(qdata)
```





Out[79]: <matplotlib.animation.FuncAnimation at 0x7fd543268700>

We can see that when the initial queue size is 1, the queue alternates between queue sizes 1, 3 and 5. This shows that the queue is not ergodic, as the sizes of 0,2, and 4 are never reached.

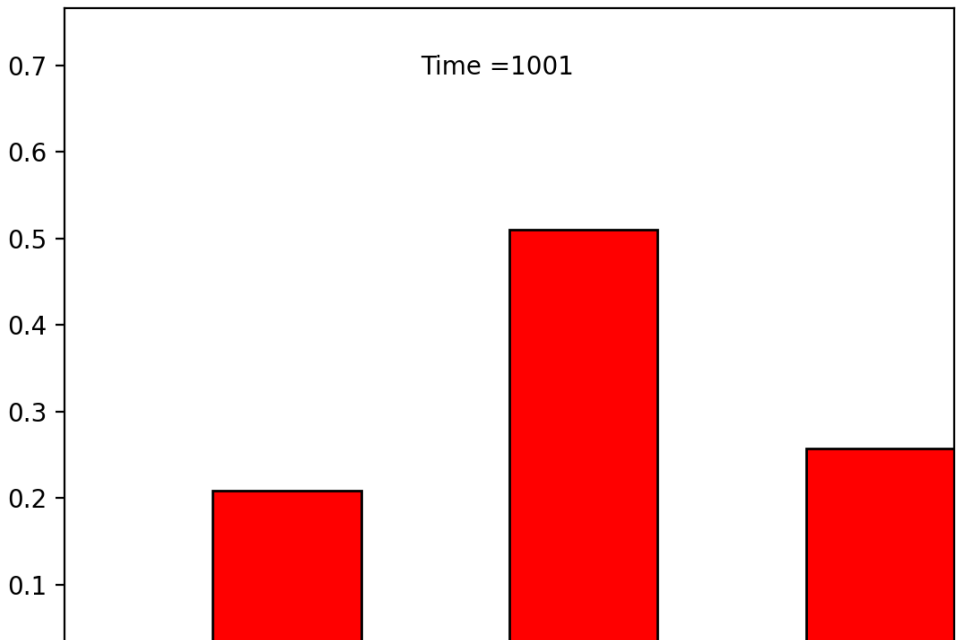
This behaviour is what is expected after Equilbrium(P) showing that the queue is not ergodic as there is not a single eigenvector of value 1.

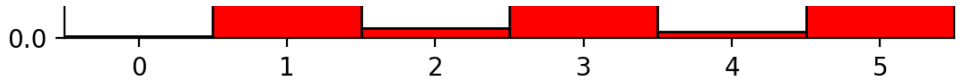
In [80]:
```
#Change first row of the transition matrix

P[0] = [0.25,0.25, 0.5, 0, 0, 0]

print(P)
```

```
[[0.25 0.25 0.5  0.   0.   0.  ]
 [0.   0.5  0.   0.5  0.   0.  ]
 [0.   0.   0.5  0.   0.5  0.  ]
 [0.   0.25 0.   0.5  0.   0.25]
 [0.25 0.   0.25 0.   0.5  0.  ]
 [0.   0.   0.   0.5  0.   0.5 ]]
```

In [86]:
```
qdata1=Queue(P,0,1000)
```

In [87]:
```
Qplot(qdata1)
```

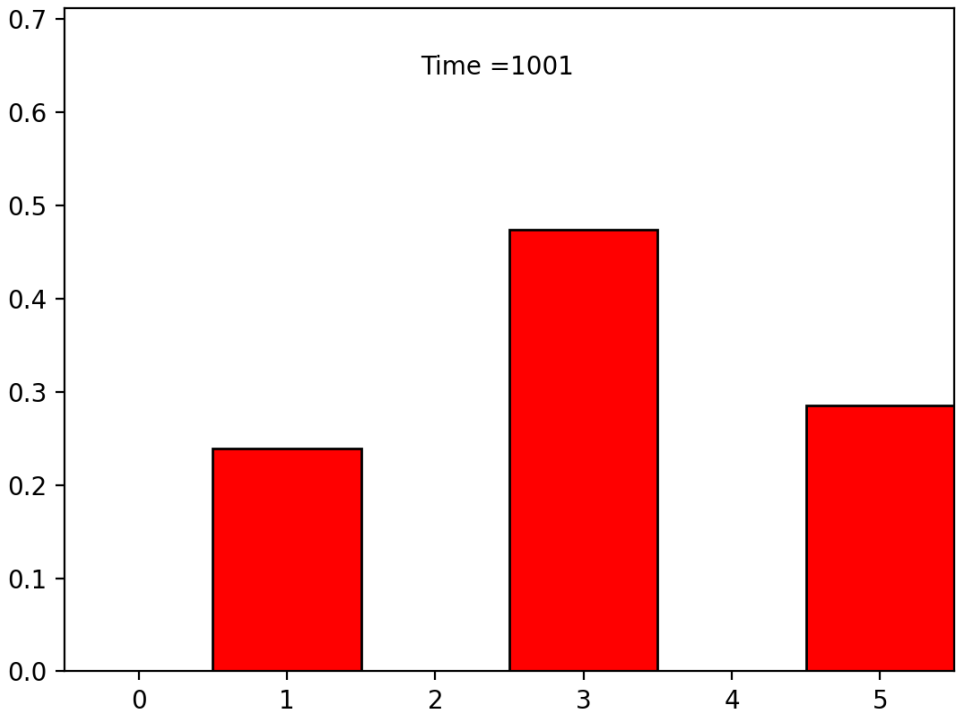




Out[87]: <matplotlib.animation.FuncAnimation at 0x7fd544cb28b0>

In [88]:
```
qdata1=Queue(P,1,1000)
```

In [89]:
```
Qplot(qdata1)
```



Out[89]: <matplotlib.animation.FuncAnimation at 0x7fd545ca7c10>

In [26]:
```
X, V = np.linalg.eig(P)
print(X)
```

```
[ 9.61366523e-01+0.j          1.44316739e-01+0.2769298j
  1.44316739e-01-0.2769298j -1.11022302e-16+0.j
  1.00000000e+00+0.j          5.00000000e-01+0.j        ]
```

From the eigenvalue of 1, we can see that P is now ergodic after changing the first row of the transition matrix. The equilibrium probabilites are given by the eigenvector corresponding to the eigenvalue of 1.

In [27]:
```
Equilibrium(P)
```

Out[27]:
```
array([1.20186574e-15, 2.50000000e-01, 3.34130169e-15, 5.00000000e-01,
       3.36535639e-15, 2.50000000e-01])
```

Examining the simulations and the transition matrix we see that an initial state of 0, 2 or 4 allows the queue to visit all other sizes from 0 to 5.

An initial state of 1, however, limits the queue to lengths 1, 3 and 5.

Q.2(*)

A queue is observed 1000 time intervals with data as given in the array `qdata` below. This is the same as Question 4 of Practical 1.

Compare the actual frequencies of events with a simulated one. Use the probabilities found in Practical 1.

Look at that question again for examples of do loops and if else statements in Python

In [92]:

```
qdata = [4, 5, 6, 6, 6, 7, 6, 7, 6, 5, 4, 4, 5, 6, 7, 6, 5, 4, 3, 4, 5, 6,
         2, 3, 4, 3, 2, 3, 2, 1, 1, 2, 2, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 0,
         8, 7, 6, 7, 6, 6, 5, 4, 5, 4, 3, 2, 3, 2, 3, 2, 3, 2, 1, 1, 2, 3,
         6, 5, 6, 5, 4, 5, 4, 3, 4, 3, 4, 3, 2, 1, 0, 0, 0, 0, 1, 2, 1, 0,
         3, 2, 1, 2, 3, 2, 3, 2, 1, 0, 0, 1, 1, 2, 3, 2, 1, 0, 0, 1, 0, 0,
         1, 2, 3, 4, 3, 4, 5, 6, 5, 6, 5, 6, 7, 7, 6, 5, 4, 3, 4, 3, 3, 4,
         2, 1, 0, 0, 0, 0, 1, 2, 3, 2, 2, 3, 2, 1, 0, 1, 2, 1, 0, 1, 0, 1,
         0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 2, 1, 1, 1, 0, 0, 1, 2, 3, 2, 3, 3,
         1, 0, 0, 0, 1, 2, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0,
         1, 0, 0, 1, 2, 3, 2, 3, 2, 2, 3, 4, 4, 5, 4, 3, 2, 3, 2, 3, 2, 3,
         1, 0, 1, 0, 1, 2, 3, 3, 2, 1, 1, 2, 1, 2, 1, 2, 1, 2, 3, 2, 2, 1,
         2, 2, 1, 2, 3, 4, 5, 4, 4, 5, 4, 3, 4, 3, 2, 1, 0, 0, 0, 0, 0, 0,
         1, 2, 2, 3, 2, 3, 4, 5, 6, 5, 6, 5, 4, 5, 6, 7, 6, 5, 6, 6, 7, 6,
         4, 5, 6, 6, 5, 6, 5, 6, 5, 6, 5, 5, 5, 4, 3, 4, 5, 6, 5, 4, 3, 3,
         2, 1, 0, 0, 0, 0, 0, 1, 2, 3, 2, 3, 4, 5, 6, 7, 7, 8, 9, 8, 9, 8,
         1, 0, 1, 2, 1, 2, 3, 4, 5, 6, 7, 7, 8, 7, 8, 9, 8, 7, 6, 5, 4, 3,
         2, 3, 2, 3, 3, 2, 1, 1, 0, 1, 2, 1, 2, 2, 3, 4, 5, 6, 5, 4, 3, 2,
         3, 2, 1, 0, 0, 1, 2, 3, 4, 3, 2, 3, 2, 2, 1, 0, 1, 0, 0, 1, 0, 1,
         6, 7, 8, 8, 9, 10, 9, 10, 9, 10, 11, 10, 9, 8, 9, 10, 11, 10, 10,
         9, 8, 7, 6, 7, 6, 7, 6, 5, 4, 4, 5, 4, 3, 2, 1, 1, 0, 1, 0, 1, 0,
         0, 0, 1, 0, 0, 1, 0, 1, 2, 3, 2, 3, 2, 3, 4, 3, 2, 1, 1, 2, 1, 2,
         2, 2, 1, 0, 1, 2, 3, 4, 5, 4, 5, 4, 3, 2, 3, 4, 5, 5, 4, 3, 4, 3,
         3, 4, 3, 2, 3, 4, 3, 2, 3, 2, 1, 0, 0, 0, 0, 0, 1, 0, 1, 2, 3, 4,
         6, 5, 4, 3, 2, 1, 0, 0, 1, 0, 0, 1, 0, 1, 2, 3, 4, 3, 4, 3, 2, 1,
         4, 3, 4, 4, 5, 6, 7, 6, 7, 7, 6, 5, 4, 3, 4, 3, 2, 2, 3, 4, 3, 4,
         1, 2, 3, 2, 1, 0, 0, 0, 0, 1, 1, 2, 3, 2, 3, 2, 3, 4, 5, 6, 5, 5,
         3, 2, 1, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1, 2, 1, 0, 0, 0, 1, 0, 0,
         3, 2, 3, 2, 1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 2, 1, 2, 1, 0, 1, 2, 1,
         0, 0, 1, 2, 2, 3, 3, 2, 3, 4, 3, 2, 1, 2, 3, 3, 2, 1, 0, 1, 2, 2,
         5, 4, 5, 4, 3, 2, 3, 4, 3, 2, 1, 2, 3, 4, 3, 2, 3, 2, 1, 0, 1, 2,
         0, 1, 2, 2, 3, 2, 1, 0, 1, 2, 3, 4, 4, 3, 4, 3, 2, 1, 0, 0, 0, 0,
         0, 0, 1, 0, 1, 2, 2, 3, 3, 4, 3, 2, 3, 2, 1, 0, 0, 1, 2, 1, 2, 3,
         1, 2, 1, 2, 3, 2, 3, 2, 1, 1, 0]
```

In [93]:

```
N = len(qdata)
```

In [94]:

```
nup=0 # nup counts no of up steps
ndown=0 # ndown counts no of down steps
n0 = 0

for i in range(1,N):
    if qdata[i]>qdata[i-1]:
        nup=nup+1
    elif qdata[i]<qdata[i-1]:
        ndown=ndown+1
    if qdata[i-1]==0:
        n0=n0+1
print(nup, "up steps", ndown, "down steps", n0, "steps at bottom")
```

```
417 up steps 421 down steps 170 steps at bottom
```

In [95]:

```
pup = nup/(N-1)
pdown = ndown/(N-1-n0)
print("pup:", pup,"pdown:",pdown)
```

```
pup: 0.4174174174174174 pdown: 0.5078407720144753
```

We can see that the probability of the queue increasing (pup) is 0.417, and the probability of the queue decreasing (pdown) is 0.508.

Since there is a higher probability of the queue decreasing, the expected behaviour is for the queue to be most commonly found at 0.

In [96]:

```
qsize = max(qdata) + 1 #Estimate max queue size based on the data given

P = Nearneigh(pup, pdown, qsize)
pi2 = Equilibrium(P)
```

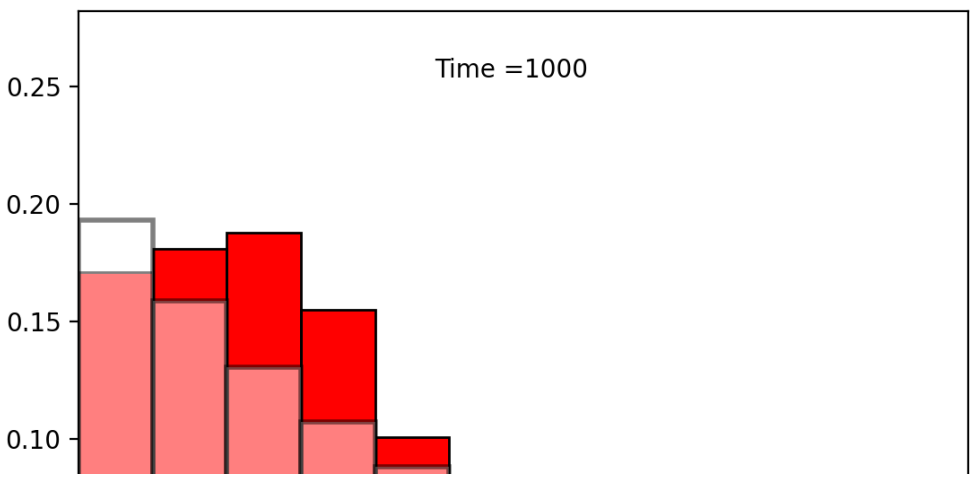
In [97]:

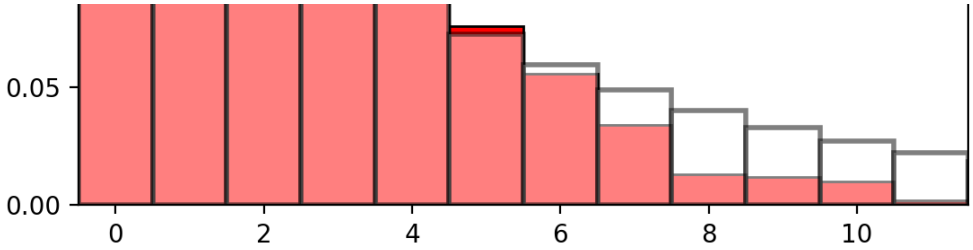
```
print("Equilbrium probabilities:\n", pi2)
```

```
Equilbrium probabilities:
 [0.19315028 0.15875899 0.13049123 0.10725668 0.08815914 0.072462
 0.05955982 0.04895492 0.04023827 0.03307367 0.02718475 0.02234438
 0.01836586]
```

In [98]:

```
Qplot(qdata, pi2)
```

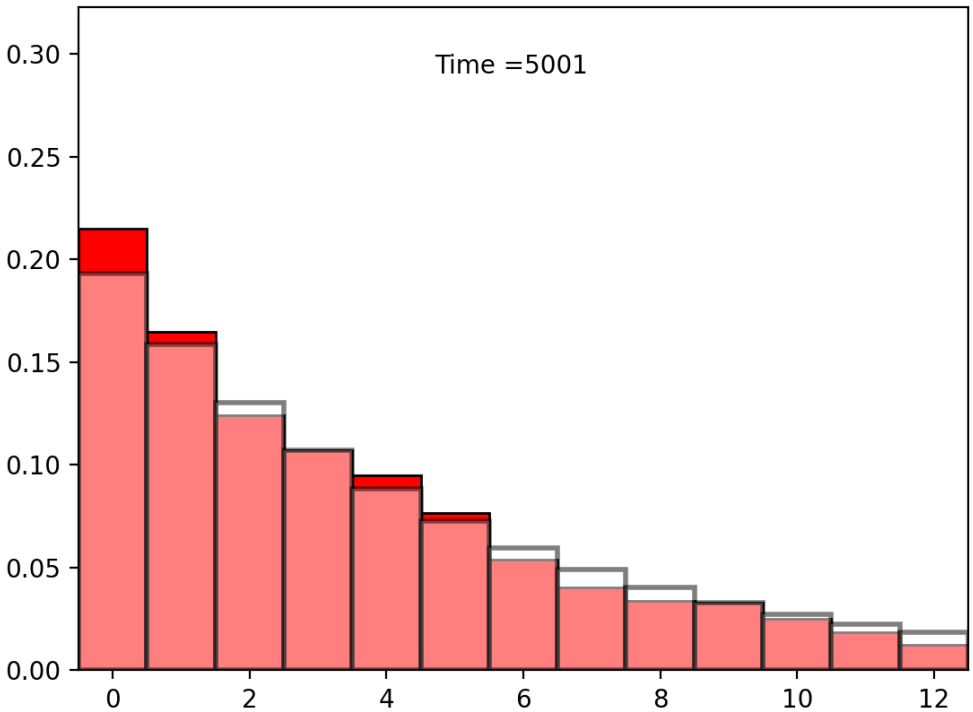




Out[98]: <matplotlib.animation.FuncAnimation at 0x7fd547156730>

In [99]:
```
qdata2 = Queue(P, 4, 5000)
```

In [100…
```
Qplot(qdata2, pi2)
```



Out[100… <matplotlib.animation.FuncAnimation at 0x7fd547d961f0>

Comparing the original qdata to the equilibrium probabilities, we can see that the observed queue sizes very roughly follow the expected queue sizes in the frequency graph.

Simulating a queue over 5000 time intervals and comparing the simulated frequencies to the equilibrium probability, we see that the more data points we have the more accurately the observed frequencies follow the expected values.

Q.3(*)

A queue is observed 10000 time intervals with data as given in the array `qdata` below.

This is the same as Question Question 5 of Practical 1.

(a) Construct a simple model for this queue as a Markov chain with only nearest neighbour interactions.

1. Estimate the transition probabilities.
2. What is the expected behaviour of the queue as time continues?
3. Is the system ergodic ?

(b) Suppose that the time step used is known to be 10 sec.

1. What is the average waiting time for customer service/arrival?
2. What is average number of servicings/arrivals per minute?

(c) Simulate this system and compare the actual frequencies of events with the simulated ones

In [101…

```
qdata = [5, 5, 5, 6, 5, 6, 6, 7, 6, 5, 6, 5, 4, 5, 6, 6, 6, 7, 8, 7, 6, 6,
```

In [102…

```
#(a)
N = len(qdata)
narrive = 0
nserve = 0
n0 = 0 #empty queue, can't be served

for i in range(1, N):
    if qdata[i] > qdata[i - 1]:
        narrive = narrive + 1
    elif qdata[i] < qdata[i - 1]:
        nserve = nserve + 1
    if qdata[i-1] == 0:
        n0=n0+1

print(narrive, "arrivals", nserve, "servings", n0, "empties")
```

```
3068 arrivals 3067 servings 2178 empties
```

In [103…

```
#1. Transition probabilities
p1 = narrive/(N-1)
p2 = nserve/(N-1-n0)
print("p1:", p1,"p2:",p2)
```

```
p1: 0.3068 p2: 0.39209920736384557
```

2. Expected behaviour / Ergodicity

Since p1 and p2 are neither both 0 or 1, the system is ergodic.

As the probability of being served is greater than the probability of customer arrival the expected behaviour of the queue is for it to be most likely found in an empty state.

This is reflected in the traffic density being less than 1:

In [104…

```
traffic_density = p1/p2
print(traffic_density)
```

```
0.7824550374959244
```

In [105…

```
qsize = max(qdata) + 1 #Estimate max queue size based on the data given

P = Nearneigh(p1, p2, qsize)
pi2 = Equilibrium(P)
```

In [106…

```
print("Equilibrium probabilities: ", pi2)
```

```
Equilibrium probabilities:  [0.21853494 0.17099377 0.13379494 0.10468852 0.
08191406 0.06409407
 0.05015073 0.03924069 0.03070408 0.02402456 0.01879814 0.0147087
 0.01150889 0.00900519 0.00704616 0.0055133  0.00431391 0.00337544
 0.00264113 0.00206657 0.001617   0.00126523]
```

In [107…

```
#(b)
#1. Average waiting time for customer arrival/service

average_arrive_time = ((N-1)*10)/narrive
print("Average customer arrival: ", average_arrive_time, " seconds")
```

```
Average customer arrival:  32.59452411994785  seconds
```

In [108…

```
average_serve_time = ((N-1-n0)*10)/nserve #taking into account times when q
print("Average waiting time for customer service: ", average_serve_time, "s
```

```
Average waiting time for customer service:  25.503749592435604 seconds
```

In [109…

```
#2.

average_arrivals_per_min = 60/average_arrive_time
print("Average arrivals per minute: ", average_arrivals_per_min)
```

```
Average arrivals per minute:  1.8408
```

In [110…

```
average_servings_per_min = 60/average_serve_time
print("Average servings per minute: ", average_servings_per_min)
```

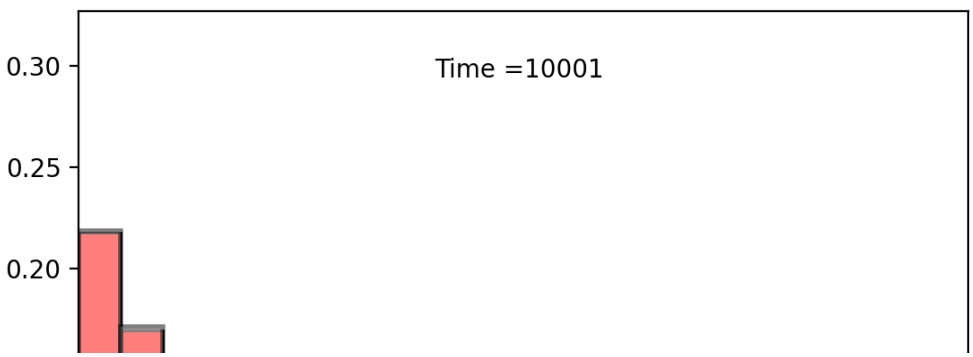
```
Average servings per minute:  2.3525952441830733
```

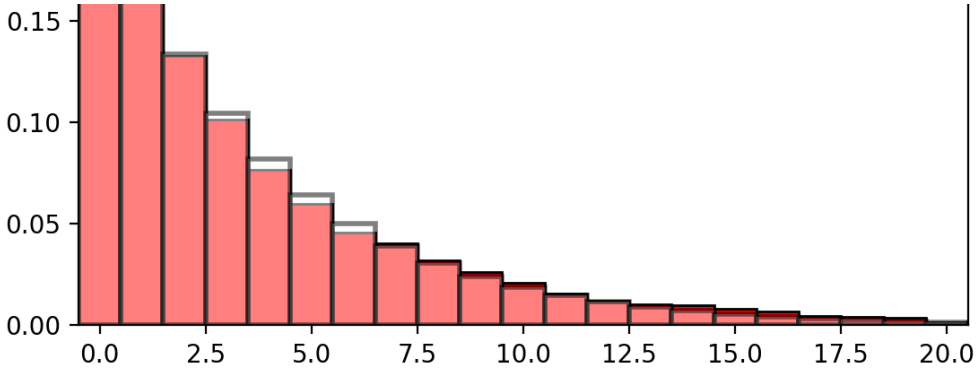
In [47]:

```
#(c)
```

In [111…

```
Qplot(qdata, pi2)
```

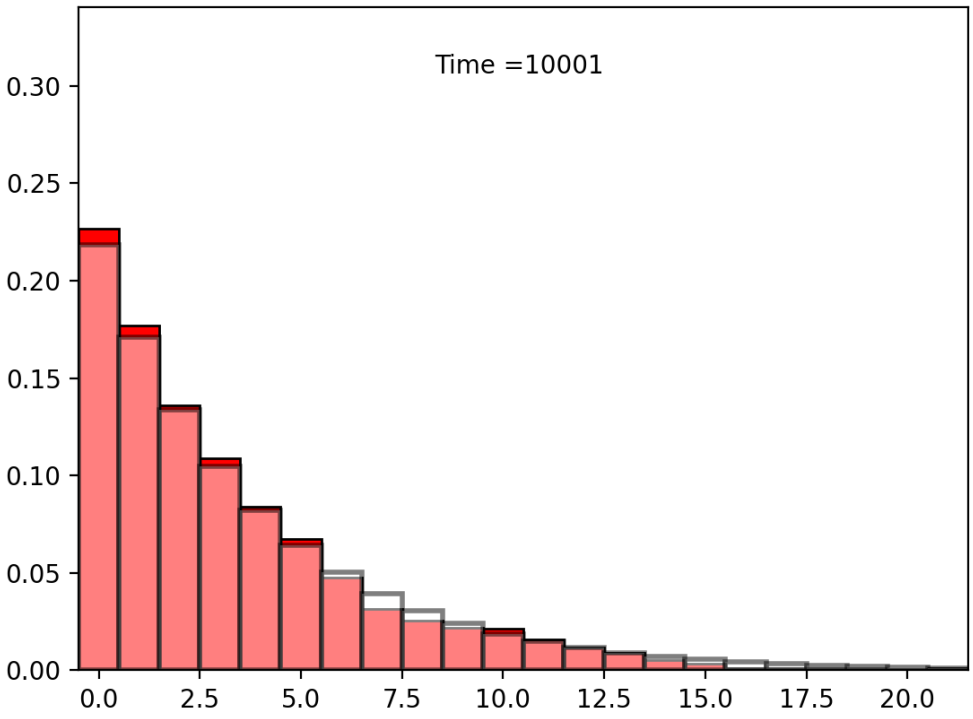




Out[111…
```
<matplotlib.animation.FuncAnimation at 0x7fd543d0dcd0>
```

In [112…
```
simulated_data = Queue(P, 20, 10000)
```

In [113…
```
Qplot(simulated_data, pi2)
```



Out[113…
```
<matplotlib.animation.FuncAnimation at 0x7fd5488a3df0>
```

We can see the simulated frequencies for this system are very close to what we expect from the equilibrium probabilities. The queue is most often found empty. We see higher than expected frequencies of queue size from 7.5 to 12.5 this may be because the queue started at size 20 and if we simulated over more time intervals the frequency graph would math the equilibrium probabilities even better.

Q.4 (*)

Telephone Exchange Queue.

A telephone exchange consists of N operators on N lines. Assume that the calling and servicing patterns are Poisson with parameters $\alpha_k = \alpha,\ \beta_k = k\beta$ respectively for queue size of $0 \leq k \leq N$ and for some $\alpha, \beta > 0$. The equilibrium distribution for the queue is:

$$\pi_k = \frac{\rho^k}{k!}\left(1 + \rho + \frac{\rho^2}{2!} + \ldots + \frac{\rho^N}{N!}\right)^{-1},\ \rho = \frac{\alpha}{\beta}$$

Of particular interest, in this model, is the frequency with which the exchange becomes saturated over a long period of time i.e. π_N. This should be a small number for an efficient exchange since it provides a measurement of the number of calls lost to the system.

1. A telephone exchange has 5 operators who can handle 40 calls an hour each on average. If calls arrive at the exchange on average every 40 secs, show that there is approximately a 5.5\% chance that the exchange will become saturated after a long time.

2. Simulate this system over 3 hours with 10 sec time intervals. For what proportion of your simulated events do you find all operators are busy?

3. Compare the results of your simulation to the equilibrium probabilities π

Hints

Calculate α, β based on the information given for the 5 operator exchange. Be consistent with use of time units. In part (2) choose 10 secs as the time interval size and construct a transition matrix P before simulating.

Calculate α, β

In [114…

```
#arrival rate is 1 caller every 40 seconds, 0.25 per 10 seconds
#service rate with 40 calls every hour per operator, 40/360 per 10 seconds
alpha = 1/4
beta = 1/9
```

In [115…

```
def Nearneigh2 (pup, pdown, r):
    #An updated version of Nearneigh that takes into account
    #a varying service rate
    P = np.zeros((r+1, r+1))
    P[0,0] = 1 - pup
    P[0,1] = pup
    P[r, r] = 1 - r*pdown
    P[r, r - 1] = r*pdown
    for i in range(1,r):
        P[i, i-1] = i*pdown
        P[i,i] = 1 - i*pdown - pup
        P[i, i + 1] = pup
    return (P)
```

In [116…

```
r = 5 #there are 5 operators described in the question
P = Nearneigh2(alpha, beta, r)
```

In [117…

```
pi4 = Equilibrium(P)
print("Equilibrium probabilities:\n", pi4)
```

```
Equilibrium probabilities:
 [0.10836466 0.24382049 0.27429805 0.20572354 0.11571949 0.05207377]
```

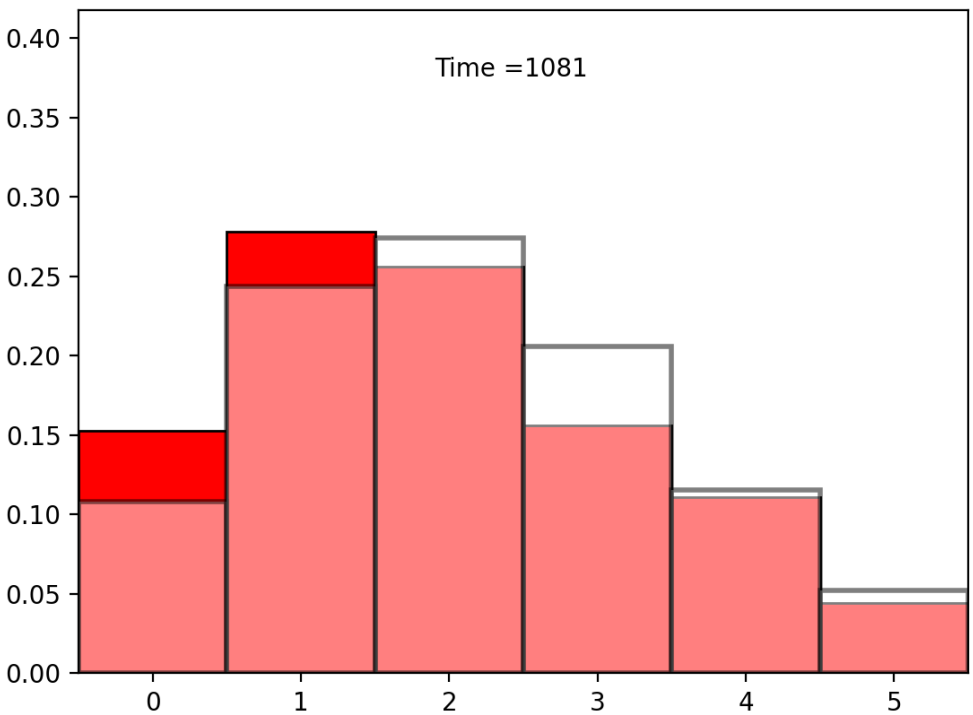
We can see from the equilibrium probabilities that there is a 5.207377% chance the exchange will be found saturated. There is approximately a 5.5% chance that the exchange will become saturated after a long time.

In [120…

```
#3 hours is 1080 time intervals of length 10 seconds
qdata4 = Queue(P, 0, 1080)
```

In [121…

```
Qplot(qdata4, pi4)
```



Out[121…

```
<matplotlib.animation.FuncAnimation at 0x7fd529792d60>
```

We can see that the simulated data closely follows our expected frequencies.

The queue in this simulated data is empty or has 1 operator busy more often than expected, and therefore is also found less often with 2, 3 or 4 operators busy. We can see that the telephone exchange was found saturated approximately 5% of the time.

MP307 Practical 1 Queueing Theory I

Name: Cian Mac Liatháin

ID: 11476078

In [2]:
```
# import a standard Python package for doing matrix theory below
import numpy as np
import matplotlib as plt
```

Q.1

Consider the two state telephone example considered in lecture 2 with transition matrix

$$P = \begin{bmatrix} 1/2 & 1/2 \\ 2/3 & 1/3 \end{bmatrix}$$

This can be stored in a Python array `P` as follows:

In [3]:
```
P=np.array([[1/2,1/2],[2/3,1/3]])
print(P)
```

```
[[0.5        0.5        ]
 [0.66666667 0.33333333]]
```

$\text{Prob}(i \to j)$ in 2 time steps is found by evaluating P^2.
We may multiply two matrices using the `np.matmul` Python function.

In [4]:
```
np.matmul(P,P)
```

Out[4]:
```
array([[0.58333333, 0.41666667],
       [0.55555556, 0.44444444]])
```

We may also take the power of a matrix using the `np.linalg.matrix_power` Python function

In [5]:
```
np.linalg.matrix_power(P,2)
```

Out[5]:
```
array([[0.58333333, 0.41666667],
       [0.55555556, 0.44444444]])
```

In [6]:
```
np.linalg.matrix_power(P,5)
```

Out[6]:
```
array([[0.57137346, 0.42862654],
       [0.57150206, 0.42849794]])
```

In [7]:
```
np.linalg.matrix_power(P,20)
```

Out[7]:
```
array([[0.57142857, 0.42857143],
       [0.57142857, 0.42857143]])
```

Notice that the rows are converging which suggests that the system is ergodic i.e. for $p_m(i,j)$ the probability of transtition from state i to state j in m steps then

$$\lim_{m\to\infty} p_m(i,j) = \pi_j$$

exists for equilibrium probability π_j independent of the initial state i.

In fact the above system is ergodic with equilibrium probabilities $\pi_0 = 4/7$ and $\pi_1 = 3/7$

In [8]:
```
print(4/7)
print(3/7)
```

```
0.5714285714285714
0.42857142857142855
```

The equilibrium probabilities are found by looking for the **left** eigenvector of P for eigenvalue 1.
We can find all eigenvectors and eigenvalues of P by use of the `np.linalg.eig` Python function as follows.

In [9]:
```
eigendata=np.linalg.eig(P)
print(eigendata)
```

```
(array([ 1.        , -0.16666667]), array([[ 0.70710678, -0.6       ],
       [ 0.70710678,  0.8       ]]))
```

The output is a tuple consisting of an array and a matrix. We may unpack the tuple as follows:

In [10]:
```
X, V=eigendata
```

In [11]:
```
print(X)
print(V)
```

```
[ 1.         -0.16666667]
[[ 0.70710678 -0.6       ]
 [ 0.70710678  0.8       ]]
```

The eigenvalues of P are given in the array `X` with eigenvectors given in the matrix `V`.
In particular the eigenvector for 1 is the first column vector with equal entries. We can read this off the entries of the first column as an array:

In [12]:
```
u=V[:,0] # Note that Python indices run as 0,1, ...
print(u)
```

```
[0.70710678 0.70710678]
```

A constant scalar multiple of an eigenvector is also an eigenvector so in fact we have confirmed that $Pu = u$ for $u = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$.

Thus the `np.linalg.eig` Python function returns the **right** eigenvectors of P.
Therefore we consider the right eigenvectors of the transpose of P, denoted by P^T, which provide us with the left eigenvectors of P.

In [13]:
```
PT=P.transpose()
print(PT)
```

```
[[0.5         0.66666667]
 [0.5         0.33333333]]
```

In [14]:

```
eigendata=np.linalg.eig(PT)
```

In [15]:

```
X, V=eigendata
print(X)
print(V)
```

```
[ 1.         -0.16666667]
[[ 0.8        -0.70710678]
 [ 0.6         0.70710678]]
```

Thus the left eigenvector of P for eigenvalue 1 is

In [16]:

```
u=V[:,0]
print(u)
print(u.real) # print real part of u
```

```
[0.8 0.6]
[0.8 0.6]
```

We normalize this eigenvector so that the entries sum to unity. We thus obtain the equilibrium probability vector π

In [17]:

```
pi=u/sum(u) # divide elements of u by sum of elements of u
print(pi.real)
print([4/7,3/7])
```

```
[0.57142857 0.42857143]
[0.5714285714285714, 0.42857142857142855]
```

Q.2

A finite queue of maximum size 3 is observed with the following transition matrix

$$\begin{bmatrix} 1/3 & 0 & 2/5 & 4/15 \\ 1/4 & 0 & 3/10 & 9/20 \\ 0 & 2/3 & 1/5 & 2/15 \\ 1/5 & 0 & 2/5 & 2/5 \end{bmatrix}$$

1. Find Prob($i \to j$ in 10 steps).

2. Find the equilibrium probabilities $\pi_0, \pi_1, \pi_2, \pi_3$.

In [18]:

```
P=np.array([
[1/3,0,2/5,4/15],
[1/4,0,3/10,9/20],
[0,2/3,1/5,2/15],
[1/5,0,2/5,2/5]
])
print(P)
```

```
[[0.33333333 0.         0.4        0.26666667]
 [0.25       0.         0.3        0.45      ]
 [0.         0.66666667 0.2        0.13333333]
 [0.2        0.         0.4        0.4       ]]
```

In [19]:
```
# 2.1 Find Prob(i->j in 10 steps)
np.linalg.matrix_power(P, 10)
```

Out[19]:
```
array([[0.17004087, 0.21052518, 0.31578941, 0.30364454],
       [0.17004046, 0.21052647, 0.315789  , 0.30364406],
       [0.17003993, 0.21052791, 0.31578988, 0.30364227],
       [0.17004086, 0.21052518, 0.31578941, 0.30364455]])
```

Repeat the sequence of steps followed Q.2.

In [20]:
```
#2.2 Find the equilibrium probabilities π0,π1,π2,π3

#Find left eigendata by first transposing matrix

PT = P.transpose()
print(PT)
```

```
[[0.33333333 0.25       0.         0.2        ]
 [0.         0.         0.66666667 0.         ]
 [0.4        0.3        0.2        0.4        ]
 [0.26666667 0.45       0.13333333 0.4        ]]
```

In [21]:
```
#Calculate left eigenvalues and eigenvectors
eigendata=np.linalg.eig(PT)
X, V=eigendata
print(X)
print(V)
```

```
[ 1.        +0.j         -0.1       +0.23804761j -0.1       -0.23804761j
  0.13333333+0.j        ]
[[ 3.30217662e-01+0.j         -2.50201620e-01-0.01669904j
  -2.50201620e-01+0.01669904j  7.07106781e-01+0.j        ]
 [ 4.08840914e-01+0.j          7.79444298e-01+0.j
   7.79444298e-01-0.j         -1.20682220e-16+0.j        ]
 [ 6.13261372e-01+0.j         -1.16916645e-01+0.27831728j
  -1.16916645e-01-0.27831728j -4.82728882e-17+0.j        ]
 [ 5.89674396e-01+0.j         -4.12326034e-01-0.26161825j
  -4.12326034e-01+0.26161825j -7.07106781e-01+0.j        ]]
```

In [23]:
```
u=V[:,0] # our unique eigenvalue of 1 corresponds to the first eigenvector

print("Equilibrium probabilities:", u.real)
```

```
Equilibrium probabilities: [0.33021766 0.40884091 0.61326137 0.5896744 ]
```

Q.3

Consider the random walk on 6 sites with the following transition matrix

$$\begin{bmatrix} 1/2 & 0 & 1/2 & 0 & 0 & 0 \\ 0 & 1/2 & 0 & 1/2 & 0 & 0 \\ 0 & 0 & 1/2 & 0 & 1/2 & 0 \\ 0 & 1/4 & 0 & 1/2 & 0 & 1/4 \\ 1/4 & 0 & 1/4 & 0 & 1/2 & 0 \\ 0 & 0 & 0 & 1/2 & 0 & 1/2 \end{bmatrix}$$

Is the system ergodic? Compare your result to that for the modified random walk with transition matrix below and explain the observed difference in behaviour.

$$\begin{bmatrix} 1/4 & 1/4 & 1/2 & 0 & 0 & 0 \\ 0 & 1/2 & 0 & 1/2 & 0 & 0 \\ 0 & 0 & 1/2 & 0 & 1/2 & 0 \\ 0 & 1/4 & 0 & 1/2 & 0 & 1/4 \\ 1/4 & 0 & 1/4 & 0 & 1/2 & 0 \\ 0 & 0 & 0 & 1/2 & 0 & 1/2 \end{bmatrix}$$

In [24]:

```
P=np.array([
[1/2,0,1/2,0,0,0],
[0,1/2,0,1/2,0,0],
[0,0,1/2,0,1/2,0],
[0,1/4,0,1/2,0,1/4],
[1/4,0,1/4,0,1/2,0],
[0,0,0,1/2,0,1/2]])
print(P)
```

```
[[0.5  0.   0.5  0.   0.   0.  ]
 [0.   0.5  0.   0.5  0.   0.  ]
 [0.   0.   0.5  0.   0.5  0.  ]
 [0.   0.25 0.   0.5  0.   0.25]
 [0.25 0.   0.25 0.   0.5  0.  ]
 [0.   0.   0.   0.5  0.   0.5 ]]
```

In [25]:

```
#Is this system ergodic?
X, V = np.linalg.eig(P)
print(X)
```

```
[ 1.00000000e+00+0.j    2.50000000e-01+0.25j  2.50000000e-01-0.25j
 -1.11022302e-16+0.j    1.00000000e+00+0.j    5.00000000e-01+0.j  ]
```

3.1 We can see that there are two eigenvalues of 1, so the matrix is not ergodic as there are different possible equilibrium probabilities depending on initial conditions

In [26]:

```
#3.2:

P2=np.array([
[1/4,1/4,1/2,0,0,0],
[0,1/2,0,1/2,0,0],
[0,0,1/2,0,1/2,0],
[0,1/4,0,1/2,0,1/4],
[1/4,0,1/4,0,1/2,0],
[0,0,0,1/2,0,1/2]])
print(P2)
```

```
[[0.25 0.25 0.5  0.   0.   0.  ]
 [0.   0.5  0.   0.5  0.   0.  ]
 [0.   0.   0.5  0.   0.5  0.  ]
 [0.   0.25 0.   0.5  0.   0.25]
 [0.25 0.   0.25 0.   0.5  0.  ]
 [0.   0.   0.   0.5  0.   0.5 ]]
```

In [27]:

```
X2, V2 = np.linalg.eig(P2)
print(X2)
```

```
[ 9.61366523e-01+0.j          1.44316739e-01+0.2769298j
  1.44316739e-01-0.2769298j -1.11022302e-16+0.j
  1.00000000e+00+0.j          5.00000000e-01+0.j        ]
```

3.2 This alternative system is ergodic since there is only a single eigenvalue of 1.

Q.4

A queue is observed over 1000 time intervals where the size of the queue after each time step is given. Construct a simple model for this queue as a Markov chain with only nearest neighbour interactions.

1. What is the expected behaviour of the queue as time continues?

2. Is the system ergodic?

In [28]:

```
qdata = [4, 5, 6, 6, 6, 7, 6, 7, 6, 5, 4, 4, 5, 6, 7, 6, 5, 4, 3, 4, 5, 6,
         2, 3, 4, 3, 2, 3, 2, 1, 1, 2, 2, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 0,
         8, 7, 6, 7, 6, 6, 5, 4, 5, 4, 3, 2, 3, 2, 3, 2, 3, 2, 1, 1, 2, 3,
         6, 5, 6, 5, 4, 5, 4, 3, 4, 3, 4, 3, 2, 1, 0, 0, 0, 0, 1, 2, 1, 0,
         3, 2, 1, 2, 3, 2, 3, 2, 1, 0, 0, 1, 1, 2, 3, 2, 1, 0, 0, 1, 0, 0,
         1, 2, 3, 4, 3, 4, 5, 6, 5, 6, 5, 6, 7, 7, 6, 5, 4, 3, 4, 3, 3, 4,
         2, 1, 0, 0, 0, 0, 1, 2, 3, 2, 2, 3, 2, 1, 0, 1, 2, 1, 0, 1, 0, 1,
         0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 2, 1, 1, 1, 0, 0, 1, 2, 3, 2, 3, 3,
         1, 0, 0, 0, 1, 2, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0,
         1, 0, 0, 1, 2, 3, 2, 3, 2, 2, 3, 4, 4, 5, 4, 3, 2, 3, 2, 3, 2, 3,
         1, 0, 1, 0, 1, 2, 3, 3, 2, 1, 1, 2, 1, 2, 1, 2, 1, 2, 3, 2, 2, 1,
         2, 2, 1, 2, 3, 4, 5, 4, 4, 5, 4, 3, 4, 3, 2, 1, 0, 0, 0, 0, 0, 0,
         1, 2, 2, 3, 2, 3, 4, 5, 6, 5, 6, 5, 4, 5, 6, 7, 6, 5, 6, 6, 7, 6,
         4, 5, 6, 6, 5, 6, 5, 6, 5, 6, 5, 5, 5, 4, 3, 4, 5, 6, 5, 4, 3, 3,
         2, 1, 0, 0, 0, 0, 0, 1, 2, 3, 2, 3, 4, 5, 6, 7, 7, 8, 9, 8, 9, 8,
         1, 0, 1, 2, 1, 2, 3, 4, 5, 6, 7, 7, 8, 7, 8, 9, 8, 7, 6, 5, 4, 3,
         2, 3, 2, 3, 3, 2, 1, 1, 0, 1, 2, 1, 2, 2, 3, 4, 5, 6, 5, 4, 3, 2,
         3, 2, 1, 0, 0, 1, 2, 3, 4, 3, 2, 3, 2, 2, 1, 0, 1, 0, 0, 1, 0, 1,
         6, 7, 8, 8, 9, 10, 9, 10, 9, 10, 11, 10, 9, 8, 9, 10, 11, 10, 10,
         9, 8, 7, 6, 7, 6, 7, 6, 5, 4, 4, 5, 4, 3, 2, 1, 1, 0, 1, 0, 1, 0,
         0, 0, 1, 0, 0, 1, 0, 1, 2, 3, 2, 3, 2, 3, 4, 3, 2, 1, 1, 2, 1, 2,
         2, 2, 1, 0, 1, 2, 3, 4, 5, 4, 5, 4, 3, 2, 3, 4, 5, 5, 4, 3, 4, 3,
         3, 4, 3, 2, 3, 4, 3, 2, 3, 2, 1, 0, 0, 0, 0, 0, 1, 0, 1, 2, 3, 4,
         6, 5, 4, 3, 2, 1, 0, 0, 1, 0, 0, 1, 0, 1, 2, 3, 4, 3, 4, 3, 2, 1,
         4, 3, 4, 4, 5, 6, 7, 6, 7, 7, 6, 5, 4, 3, 4, 3, 2, 2, 3, 4, 3, 4,
         1, 2, 3, 2, 1, 0, 0, 0, 0, 1, 1, 2, 3, 2, 3, 2, 3, 4, 5, 6, 5, 5,
         3, 2, 1, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1, 2, 1, 0, 0, 0, 1, 0, 0,
         3, 2, 3, 2, 1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 2, 1, 2, 1, 0, 1, 2, 1,
         0, 0, 1, 2, 2, 3, 3, 2, 3, 4, 3, 2, 1, 2, 3, 3, 2, 1, 0, 1, 2, 2,
         5, 4, 5, 4, 3, 2, 3, 4, 3, 2, 1, 2, 3, 4, 3, 2, 3, 2, 1, 0, 1, 2,
         0, 1, 2, 2, 3, 2, 1, 0, 1, 2, 3, 4, 4, 3, 4, 3, 2, 1, 0, 0, 0, 0,
         0, 0, 1, 0, 1, 2, 2, 3, 3, 4, 3, 2, 3, 2, 1, 0, 0, 1, 2, 1, 2, 3,
         1, 2, 1, 2, 3, 2, 3, 2, 1, 1, 0]
```

In [29]:

```
N=len(qdata)
```

In [30]:

```
N
```

Out[30]:

```
1000
```

N is the number of events recorded

We can easily compute the average with the following do loop

In [32]:
```
ans=0
for i in range(1,N):
    ans=ans+qdata[i-1]
#do loop complete
average=ans/N
```

In [33]:
```
average
```

Out[33]:
```
2.718
```

Apply the simplest model of an infinite nearest neighbour queue with p_1 = prob of jumping up and p_2 = prob of jumping down. These can be estimated by counting the number of jumps up vs the number of jumps down. Consider the following do loop with if statements.

In [34]:
```
nup=0 # nup counts no of up steps
ndown=0 # ndown counts no of down steps

for i in range(1,N):
    if qdata[i]>qdata[i-1]:
        nup=nup+1
    elif qdata[i]<qdata[i-1]:
        ndown=ndown+1

# if statements complete
print(nup,"up steps", ndown,"down steps")
```

```
417 up steps 421 down steps
```

In [35]:
```
p1=nup/(N-1)
p2=ndown/(N-1)
print(p1,p2)
```

```
0.4174174174174174 0.4214214214214214
```

Q: Do you consider this to be a good estimate? What is wrong?

A: #It is not possible for the queue size to get lower when it is at 0, but your estimate doesn't take this into account

In [36]:
```
# our for loop can be adapted to take that into account
# by counting the number of steps already at 0

nup=0 # nup counts no of up steps
ndown=0 # ndown counts no of down steps
n0 = 0

for i in range(1,N):
    if qdata[i]>qdata[i-1]:
        nup=nup+1
    elif qdata[i]<qdata[i-1]:
        ndown=ndown+1
    if qdata[i-1]==0:
        n0=n0+1
print(nup, "up steps", ndown, "down steps", n0, "steps at bottom")
```

```
417 up steps 421 down steps 170 steps at bottom
```

In [37]:

```
p1 = nup/(N-1)
p2 = ndown/(N-1-n0)
print("p1:", p1,"p2:",p2)
```

```
p1: 0.4174174174174174 p2: 0.5078407720144753
```

4.1

Since p2 > p1, the expected behaviour of the queue is to grow smaller until it empties

4.2 This system is ergodic as p1/p2 is not equal to 0 or 1

Q.5

A queue is observed over 10000 one-second time intervals with data as given below in the array `qdata` .

Construct a Poisson nearest neighbour model with a single arrival and servicing pattern and hence answer the following questions:

1. What is the average time taken for 1 customer to arrive?
2. What is the average number of customer servicings per second?
3. What is your estimate for the equilibrium probability $P(n \geq 4)$, where n is the queue size in this model?
4. Suppose that two equivalent servers are introduced. What would the equilibrium probability $P(n \geq 4)$ then be?

In [38]:

```
qdata = [5, 5, 5, 6, 5, 6, 6, 7, 6, 5, 6, 5, 4, 5, 6, 6, 6, 7, 8, 7, 6, 6,
```

In [39]:

```
N=len(qdata)
print(N)
```

```
10001
```

In [40]:

```
narrive = 0
nserve = 0
n0 = 0 #empty queue, can't be served

for i in range(1, N):
    if qdata[i] > qdata[i - 1]:
        narrive = narrive + 1
    elif qdata[i] < qdata[i - 1]:
        nserve = nserve + 1
    if qdata[i-1] == 0:
        n0=n0+1

print(narrive, "arrivals", nserve, "servings", n0, "empties")
```

```
3068 arrivals 3067 servings 2178 empties
```

In [42]:
```
#5.1
average_arrive = narrive/(N-1)
print(average_arrive, " seconds")
```

```
0.3068  seconds
```

In [43]:
```
#5.2
average_serve = nserve/(N-1-n0) #taking into account times when queue is em
print(average_serve, "customers served per second")
```

```
0.39209920736384557 customers served per second
```

In [44]:
```
#5.3
traffic_density = average_arrive/average_serve
print(traffic_density)
```

```
0.7824550374959244
```

In [47]:
```
#probabiliy that n >= 4
#this is equivalent to 1 - (P(n=0) + P(n=1) + P(n=2) + P(n=3)) = 1 -(1-rho

print("Equilibrium probability P(n>=4): ", traffic_density**4)
```

```
Equilibrium probability P(n>=4):  0.37483277974222884
```

In [52]:
```
#5.4 Suppose two equivalent servers are introduced
#Then average rate of serving would increase times 3

average_serve_3 = average_serve*3
print(average_serve_3, "customers served per second with 3 servers")
```

```
1.1762976220915367 customers served per second with 3 servers
```

In [53]:
```
# New traffic density with the increased serving rate
traffic_density_3 = average_arrive/average_serve_3
print(traffic_density_3)
```

```
0.26081834583197483
```

In [54]:
```
#Prob(n >=4)?
print("Equilibrium probability P(n>=4) with 3 servers:", traffic_density_3*
```

```
Equilibrium probability P(n>=4) with 3 servers: 0.004627565182002828
```